

МИНОБРНАУКИ РОССИИ

Юго-Западный государственный университет

В. В. Ефремов, И. Н. Ефремова, Е. А. Петрик

**КОМПЬЮТЕРНАЯ ГРАФИКА: ВВЕДЕНИЕ
В ОБРАБОТКУ ИЗОБРАЖЕНИЙ**

Практикум

Курск
ЮЗГУ
2024

УДК 004.932.4(071.8)

ББК 32.972.1я7

Е 92

Рецензенты:

Доктор педагогических наук, профессор кафедры программного обеспечения и администрирования информационных систем Курского государственного университета, профессор *В. А. Кудинов*

Доктор технических наук, профессор кафедры информатики, вычислительной техники и автоматизации Регионального открытого социального института *Д. Б. Борзов*

Е 92 **Ефремов, Владислав Владиславович**

Компьютерная графика: введение в обработку изображений : практикум / В. В. Ефремов, И. Н. Ефремова, Е. А. Петрик; Минобрнауки России, Юго-Западный гос. ун-т. – Курск : ЮЗГУ, 2024. – 154 с.

ISBN

В наглядной форме изложены способы компьютерной обработки изображений. Приведены необходимые теоретические сведения. Рассмотрены алгоритмы преобразования и примеры программ, реализующих эти алгоритмы. Представлены справочные данные. Предложены лабораторные работы.

Предназначено для студентов, обучающихся по направлению 09.03.04 «Программная инженерия», и для проектировщиков, чьи задачи связаны с компьютерной графикой.

УДК 004.932.4

ББК 32.972.1

ISBN

© Юго-Западный государственный университет, 2024

© Ефремов В. В., Ефремова И. Н., Петрик Е. А., 2024

Оглавление

Предисловие	4
Введение.....	5
Лабораторная работа № 1. Введение в компьютерную графику	7
Лабораторная работа № 2. Программа для формирования изображений.	23
Лабораторная работа № 3. Растеризация отрезков прямых	48
Лабораторная работа № 4. Растеризация окружностей.....	64
Лабораторная работа № 5. Попиксельные преобразования.....	85
Лабораторная работа № 6. Геометрические преобразования.	115
Лабораторная работа № 7. Введение в фильтрацию изображений	129
Заключение	158
Библиографический список	159

Предисловие

Целью практикума является закрепление знаний раздела «Компьютерная обработка изображений» учебной дисциплины «Компьютерная графика» путём формирования умений и навыков обработки цифровых изображений.

Практикум предназначен для студентов направлений, обучение по которым предполагает разработку программного обеспечения, и может быть полезен всем, кто желает получить практические навыки обработки изображений. Практикум может быть использован в качестве основного набора лабораторных работ при изучении раздела «Компьютерная обработка изображений» студентами дневной формы обучения, а также для самостоятельного освоения материала студентами других форм обучения. С учётом разнообразия сфер использования компьютерной обработки изображений материал практикума охватывает широкий спектр задач.

Практикум содержит лабораторные работы, достаточные для освоения навыков программной обработки изображений в рамках курсов «Компьютерная графика» высших учебных заведений.

Уровень математической сложности намеренно снижен для того, чтобы оставить возможность изучения компьютерной обработки изображений на младших курсах вузов. Для освоения материала достаточно понимание арифметики для целых и действительных чисел, алгебры и начала анализа в пределах исследования функций, полиномов и рядов, линейной алгебры работы с матрицами, булевой алгебры, двоичной арифметики и основ информатики. Способствовать восприятию материала пособия будет понимание разделов «Электричество» и «Оптика» в пределах школьной программы.

Для каждой лабораторной работы разработан пример выполнения с помощью алгоритмического языка программирования. Для успешного освоения материала требуется знание базовых алгоритмических структур, умение работать с многомерными массивами и с библиотеками объектов, а также хотя бы начальные навыки создания программ на языке программирования.

Авторы приносят извинения за возможные неточности и ошибки и с благодарностью примут отзывы и пожелания читателей по электронному адресу: i-am@swsu.ru.

Введение

В настоящее время оптическая информация в цифровом виде составляет значительную часть накапливаемой и передаваемой информации в компьютерных системах. Это связано с тем, что среди информации, воспринимаемой человеком, оптическая информация занимает ведущую роль. В связи с этим актуальность компьютерной обработки изображений не вызывает сомнения. Компьютерная обработка изображений может выполняться в различных целях, среди которых восстановление искажённых изображений, улучшение изображений для визуального восприятия, преобразование изображений для создания иллюстраций в книгах и презентациях, подготовка изображений для автоматизированного анализа, создание художественных эффектов, эффективное кодирование оптической информации и многое другое.

Значительное распространение получили программы для компьютерной обработки изображений, ориентированные на полиграфическую и художественную обработку. Среди них Adobe Photoshop, GIMP и им подобные. Обработка изображений, как правило, предусмотрена в программах для просмотра фотографий, таких как ACDSee, Xnview и др. Элементы обработки изображений входят в состав программ для обработки видео, причём как для видеомонтажа, так и для просмотра. Распространена обработка изображений и в онлайн-сервисах, предусматривающих услуги фото- и видеохостинга, таких как vk.com, а также с помощью чат-ботов. Обработка изображений практически без исключения применяется в программах автоматизации анализа для распознавания текстов, технических и медицинских изображений, в системах видеонаблюдения, радиолокации, и область применения таких программ расширяется с развитием техники и технологии.

В этой книге в качестве объекта обработки рассматриваются в первую очередь оптические цветные статические растровые цифровые двумерные изображения. Предлагаемые лабораторные работы можно распределить по категориям методов обработки:

Базовые манипуляции с изображением, позволяющие добавлять в заданную позицию созданного или считанного из файла изображения выделяющуюся цветом точку, графический примитив или фрагмент другого изображения, а результат выводить в файл и на экран, рассматриваются в работах № 1, № 2, № 3, № 4.

Поэлементные преобразования — изменение характеристик элементов раstra, включая изменения яркости и цвета с использованием различных цветовых моделей, — рассматриваются в работе № 5.

Координатные преобразования, позволяющие внести в изображение необходимые геометрические искажения или, наоборот, избавиться от них рассматриваются в работе № 6.

Фильтрация изображений, позволяющая, например, размыть изображение или, наоборот, сделать переходы между яркими и тёмными областями более чёткими. Среди фильтров наиболее подробно рассматриваются свёрточные и похожие на них — в работе № 7.

В первую очередь необходимо освоиться с базовыми методами обработки. Следом за ними следует изучить поэлементные преобразования и, соответственно, цветовые модели. Координатные преобразования и фильтрация изображений могут быть изучены независимо.

В качестве среды разработки для примеров данного пособия используется Lazarus с компилятором Free Pascal в связи с тем, что эта система программирования является свободным мультиплатформенным программным обеспечением, подобным известной среде разработки Delphi. В то же время это не должно ограничивать выполняющих лабораторные работы в выборе средств для создания программ обработки изображений. Достаточно, чтобы выбранные инструменты позволяли считывать и записывать файлы в графическом формате, предоставляли возможность считывания и записи отдельных элементов изображения и реализации алгоритмов обработки данных.

Освоение всего материала позволит самостоятельно написать программу, которая позволит создать изображение не хуже, чем графический редактор, а иногда позволит получить трудноосуществимый в известных программных продуктах результат за счёт большей гибкости программной обработки и возможности обработать изображение по любому сколь угодно необычному алгоритму.

Лабораторная работа № 1

Введение в компьютерную графику

Цель работы – освоение базовых навыков обработки растровых изображений с помощью алгоритмических языков программирования.

Краткие теоретические сведения

Для компьютерной графики изображения представляются в цифровом виде. Этого требуют устройства ввода, вывода, хранения и обработки данных цифровых компьютеров. Так что естественным способом представления графических данных на цифровом компьютере можно считать растр: изображение представлено совокупностью расположенных вплотную друг к другу элементов (пикселей), заданных яркостью.

Для представления растровой графики фундаментальное значение имеет формат пикселя, который определяет способ представления характеристик элемента изображения. Разные форматы различаются объёмом данных и интерпретацией этих данных.

1. **Монохромное изображение.** Данные описывают яркость одного канала. 8 бит на пиксель позволяют описать 256 оттенков одного цвета, чаще всего – серого. Важный частный случай – бинарное изображение с 1 битом на пиксель.

2. **Полноцветное изображение.** Так как человеческий глаз воспринимает цвет по трём цветовым каналам, чтобы задать любой воспринимаемый человеком цвет, достаточно задать яркость трёх цветовых составляющих. Как и у человека, используются красная, зелёная и голубая компоненты (Red, Green, Blue – RGB). В большинстве случаев на описание яркости каждой составляющей отводится по байту. 24 бита на пиксель позволяют задать 16777216 разных цветов. Часто к такому формату пикселя добавляется ещё один компонент, обозначающий дополнительные характеристики пикселей: дополнительный цвет, прозрачность и т. д.

3. **Индексированное изображение.** Применяемые на изображении цвета сводятся в палитру в виде таблицы и нумеруются. Каждый пиксель задаётся номером цвета. Количество цветов на изображении,

как правило, ограничено. Количество оттенков, которые можно представить в палитре, может быть достаточно большим.

Из регулярной совокупности пикселей составляется изображение произвольного размера. Как правило, пиксели располагаются в узлах прямоугольной решётки, и изображение будет тоже прямоугольным. Каждый пиксель такого изображения имеет целочисленные координаты, описывающие его позицию на изображении: номер строки и номер столбца — номер элемента в строке. В большинстве форматов описания изображений координаты отсчитываются от левого верхнего угла изображения. Регулярность структуры позволяет сохранять изображения в виде непрерывной совокупности битов (так называемого битмапа). Зная размер изображения и количество битов на пиксель, этот битмап легко декодировать или получить прямой доступ к любому пикселю, так что в простейшем случае формат растрового изображения предполагает наличие битмапа и информации о формате пикселя, размере изображения в пикселях по горизонтали и вертикали.

Все примеры в этой книге разработаны в IDE FPC / Lazarus. FPC — Free Pascal Compiler — свободный мультиплатформенный компилятор языка программирования Pascal. Lazarus — свободная мультиплатформенная IDE с набором библиотек для FPC, ориентированная на поддержку функциональных возможностей Delphi и повторяющая особенности её интерфейса. Lazarus понимает графические файлы следующих форматов: Bitmap (*.bmp), Pixmap (*.xpm, *.pbm, *.pgm, *.ppm), Portable Network Graphic (*.png), Icon (*.ico, *.icns), Cursor (*.cur), Joint Picture Expert Group (*.jpg, *.jpeg, *.jpe, *.jfif), Tagged Image File Format (*.tif, *.tiff), Graphics Interchange Format (*.gif). Полноцветные картинки любых размеров могут храниться в файлах BMP, PPM, PNG, JPEG, TIFF.

Минимальный набор инструментов для выполнения работы должен включать в себя:

- средства чтения изображения из файла и записи изображения в файл хотя бы одного стандартного формата;
- средства чтения и записи значения каждого пикселя изображения по каналам RGB;
- средства отображения изображения на мониторе.

Lazarus, как и Delphi, поддерживает следующие средства:

TImage — визуальный объект для работы с графической информацией. В первую очередь нас интересуют:

1. свойство **Proportional: boolean** – масштабировать, соблюдая пропорции;

2. свойство **Stretch: boolean** – вместить при отображении на форме;

3. свойство **Picture: TPicture** – контейнер для хранения графического объекта. Поддерживаются форматы битмапа (растровый формат), иконки и метафайла (векторный формат). Нас интересует растровая графика в виде битмапа. У **TPicture** есть полезные нам методы и свойства:

1) метод **LoadFromFile(filename: string)** – чтение графики из файла. Поддерживается ряд популярных графических форматов: jpeg, png, bmp и т. д. и т. п.;

2) метод **SaveToFile(filename: string)** – запись графики в файл;

3) свойство **Bitmap: TBitmap** — контейнер для хранения битмапа — изображения, представленного в виде массива пикселей. У **TBitmap** есть полезные нам методы и свойства:

а свойство **Canvas: TCanvas** – средство формирования изображения (холст). Среди большого количества реализованных в этом объекте методов в первую очередь нас интересует доступ к пикселям:

– свойство **Pixels[x: integer, y: integer]: TColor** – цвет пикселя с координатами (x, y);

б свойство **PixelFormat** описывает формат хранения пикселей. Нас интересует только формат **pf24bit**, в соответствии с которым каждый пиксель представлен в виде трёх байт, определяющих яркость составляющих RGB при описании цвета;

в свойство **Height: Integer** – количество пикселей по вертикали;

г свойство **Width: Integer** – количество пикселей по горизонтали.

TColor – тип данных для хранения цвета, с которым можно работать с помощью функций модуля **LCLIntf**:

1. **getrvalue(color: TColor): byte** – значение красной составляющей;

2. **getgvalue(color: TColor): byte** – значение зелёной составляющей;

3. **getbvalue(color: TColor): byte** – значение синей составляющей;

4. **RGB(R: byte, G: byte, B: byte): TColor** – получение цвета **TColor** из трёх составляющих.

TOpenPictureDialog – диалог для чтения графических файлов:

1. **Execute: boolean** – метод, отображающий диалоговое окно; возвращает значение **True**, когда пользователь нажал **ОК**;

2. **FileName: string** – свойство, содержащее полное имя файла.

TSavePictureDialog – диалог для чтения графических файлов:

1. **Execute: boolean** – метод, отображающий диалоговое окно; возвращает значение **True**, когда пользователь нажал **ОК**;

2. **FileName: string** – свойство, содержащее полное имя файла.

Логика применения этих объектов проста: объект **TImage** размещаем на форме для отображения обрабатываемого изображения. Сразу задаём нужные значения свойств **Stretch** и **Proportional**. Считывать изображение из файла будем в контейнер **TImage.TPicture** методом **TImage.TPicture.LoadFromFile**, имя файла берём из **TOpenPictureDialog**. Сохранять изображение в файл будем методом **TImage.TPicture.SaveToFile**, используя **TSavePictureDialog**. Растр будет храниться в контейнере **TImage.TPicture.TBitmap**. Доступ к отдельным пикселям будем получать через свойство **Pixels** объекта **Canvas** контейнера **TBitmap**. Координаты пикселей задаются целыми числами от 0 до M-1 для «x» и от 0 до N-1 для «y» для изображения размером MxN пикселей. Пиксель с координатами (0,0) находится в левом верхнем углу. Данные хранятся в формате **TColor**, считывать яркость каждой составляющей будем функциями **getrvalue**, **getgvalue**, **getbvalue**, записывать – с помощью функции **RGB**. Яркость каждой составляющей задаётся целым числом от 0 до 255.

Задание

1. Сфотографировать что-нибудь симпатичное на свой смартфон или цифровой фотоаппарат. Убедиться, что формат фотографии читается в системе программирования.

2. Разработать алгоритм и программу для обработки графической информации по варианту. Программа включает в себя:

- чтение изображения из файла;
- добавление точек заданного цвета в заданную позицию изображения;
- сохранение созданного изображения в файл на диске.

3. Обработать фотографию с помощью разработанной программы.

4. Найти изменения в результате и описать цвет точек.

5. Ответить на контрольные вопросы.
6. Оформить отчёт.

Содержание отчёта

1. Титульный лист.
2. Формулировка задачи.
3. Программа на алгоритмическом языке программирования.
4. Форма.
5. Исходное и преобразованное изображения.
6. Ответы на контрольные вопросы.

Порядок выполнения работы

1. Ознакомление с целью работы, заданием, содержанием отчёта порядком выполнения работы, индивидуальным вариантом, примером выполнения работы.
2. Ознакомление со средствами системы программирования для чтения, обработки и записи графики на примере и с помощью технической документации.
3. Получение фотографии и изучение свойств файла.
4. Разработка алгоритма и отладка программы для решения задачи.
5. Решение задачи обработки изображения.
6. Поиск добавленных в программе точек и описание их цвета.
7. Ответы на контрольные вопросы.
8. Оформление отчёта.

Индивидуальные варианты

Для каждого индивидуального варианта в таблице даны условия размещения трёх точек на изображении в соответствии с заданием.

1. Разместить точку цвета (0,0,0) в верхний левый угол, точку цвета (255,0,0) в центр верхней строки, точку цвета (0,255,0) в центр левого столбца.

2. Разместить точку цвета (0,0,255) в верхний левый угол, точку цвета (255,255,0) в центр верхней строки, точку цвета (255,0,255) в центр изображения.

3. Разместить точку цвета (0,255,255) в верхний левый угол, точку

цвета (255, 255,255) в центр верхней строки, точку цвета (127,127,127) в центр правого столбца.

4. Разместить точку цвета (255,127,127) в верхний левый угол, точку цвета (127,255,127) в центр верхней строки, точку цвета (127,127,255) в нижний левый угол.

5. Разместить точку цвета (255,255,127) в верхний левый угол, точку цвета (255,127,255) в центр верхней строки, точку цвета (127,255,255) в центр нижней строки.

6. Разместить точку цвета (0,0,127) в верхний левый угол, точку цвета (0,127,0) в центр верхней строки, точку цвета (127,0,0) в правый нижний угол.

7. Разместить точку цвета (0,127,127) в верхний левый угол, точку цвета (127,0,127) в правый верхний угол, точку цвета (127,127,0) в центр левого столбца.

8. Разместить точку цвета (0,127,255) в верхний левый угол, точку цвета (127,0,255) в правый верхний угол, точку цвета (127,255,0) в центр изображения.

9. Разместить точку цвета (64,64,64) в верхний левый угол, точку цвета (255,64,64) в правый верхний угол, точку цвета (64,255,64) в центр правого столбца.

10. Разместить точку цвета (64,64,255) в верхний левый угол, точку цвета (255,255,64) в правый верхний угол, точку цвета (255,64,255) в нижний левый угол.

11. Разместить точку цвета (64,255,255) в верхний левый угол, точку цвета (64,127,64) в правый верхний угол, точку цвета (127,64,64) в центр нижней строки.

12. Разместить точку цвета (64,64,127) в верхний левый угол, точку цвета (127,64,127) в правый верхний угол, точку цвета (127,127,64) в правый нижний угол.

13. Разместить точку цвета (64,127,127) в верхний левый угол, точку цвета (127,64,255) в центр левого столбца, точку цвета (127,255,64) в центр изображения.

14. Разместить точку цвета (64,127,255) в верхний левый угол, точку цвета (64,0,0) в центр левого столбца, точку цвета (0,64,0) в центр правого столбца.

15. Разместить точку цвета (0,0,64) в верхний левый угол, точку цвета (64,64,0) в центр левого столбца, точку цвета (64,0,64) в центр нижней строки.

16. Разместить точку цвета (0,64,64) в верхний левый угол, точку цвета (64,64,64) в центр левого столбца, точку цвета (127,127,64) в правый нижний угол.

17. Разместить точку цвета (64,127,127) в верхний левый угол, точку цвета (127,64,127) в центр изображения, точку цвета (127,64,64) в центр правого столбца.

18. Разместить точку цвета (64,64,127) в верхний левый угол, точку цвета (64,127,64) в центр изображения, точку цвета (127,64,0) в левый нижний угол.

19. Разместить точку цвета (0,127,64) в верхний левый угол, точку цвета (127,0,64) в центр изображения, точку цвета (191,191,191) в центр нижней строки.

20. Разместить точку цвета (255,191,191) в верхний левый угол, точку цвета (191,255,191) в центр изображения, точку цвета (191,191,255) в правый нижний угол.

Контрольные вопросы

1. Что такое графика?
2. Что такое оптическое изображение?
3. Что такое сцена в компьютерной графике?
4. Что такое визуализация?
5. Зачем визуализировать то, что нереально воплотить физически?
6. Что такое рендеринг?
7. Что понимается под 3D-графикой на 2D-мониторе?
8. Как получить движущееся изображение?
9. Как компьютерная графика различается по назначению?
10. Каковы особенности векторной и растровой графики?
11. Как отобразить 4D-объекты?
12. Как кодируется цвет?
13. Чем стереоизображения отличаются от трёхмерных?
14. Что такое фрактал?
15. Что такое разрешение?
16. В каких единицах измеряется разрешающая способность?

Пример выполнения работы

Разместить точку цвета (254,111,94) в центр изображения, точку цвета (0,51,153) в центр правого столбца, точку цвета (221,72,20) в центр нижней строки.

Исходное изображение (рис. 1) получаем с помощью веб-камеры.



Рис. 1. Изображение, полученное с веб-камеры

Изучаем свойства файла (рис. 2) и выясняем, что формат файла поддерживается системой программирования.

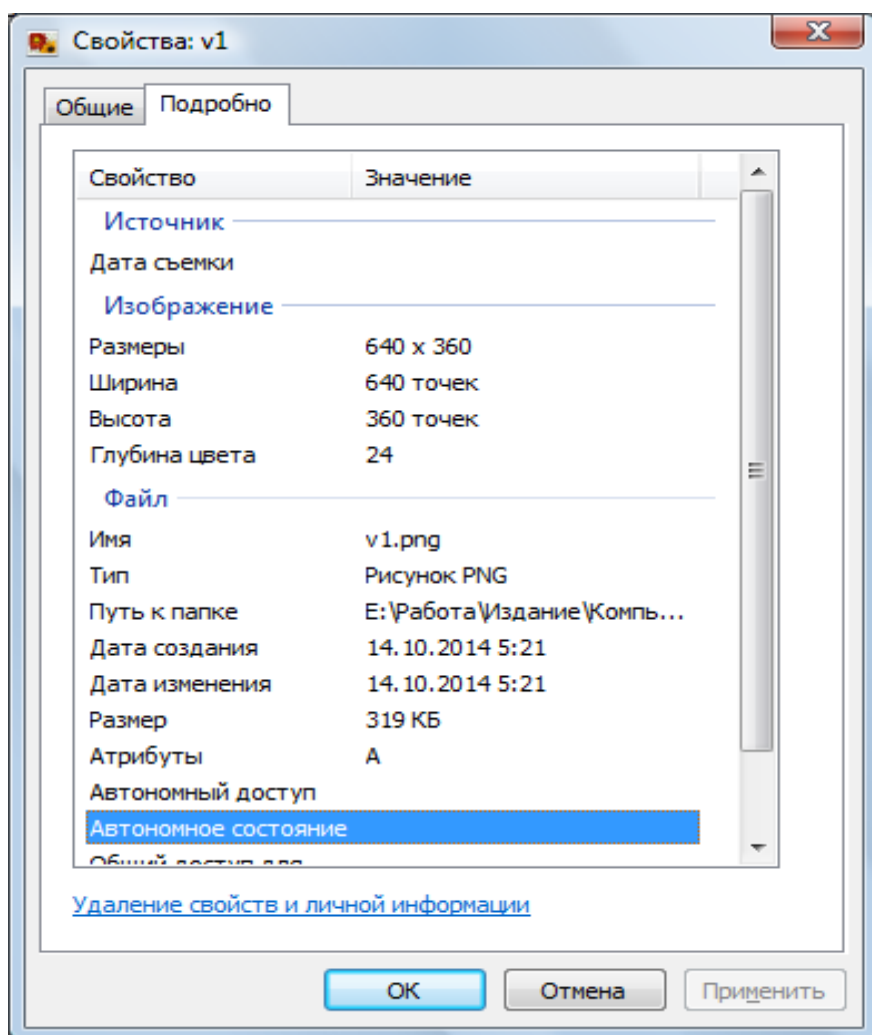


Рис. 2. Свойства исходного изображения

Разработку программы начинаем с разработки формы.

На форме (рис. 3) размещаем:

- три кнопки — для чтения, обработки и записи файла;
- объекты `OpenPictureDialog` и `SavePictureDialog` для организации цивилизованного чтения и записи файлов;
- объект `TImage` для хранения графического объекта.

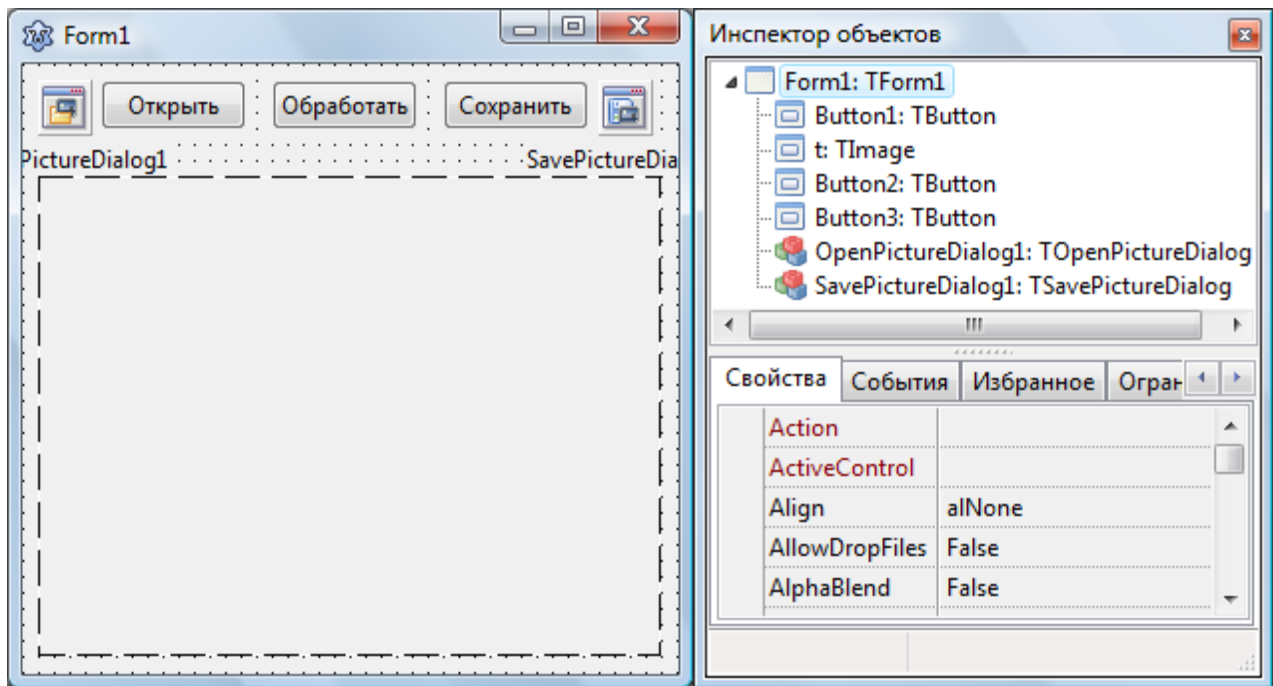


Рис. 3. Разработка формы для программы

Текст программы (весь модуль).

```
unit Unit1;
```

```
{ $mode objfpc } { $H+ }
```

```
interface
```

```
uses
```

```
Classes, SysUtils, FileUtil, Forms, Controls, Graphics, Dialogs, StdCtrls,  
GraphType, IntfGraphics, LCLType, LCLProc, LCLIntf, ExtCtrls,  
ExtDlgs, fpImage;
```

```
type
```

```
{ TForm1 }
```

```
TForm1 = class(TForm)
```

```
Button1: TButton;
```

```
Button2: TButton;
```

```
Button3: TButton;
```

```
OpenPictureDialog1: TOpenPictureDialog;
```

```

SavePictureDialog1: TSavePictureDialog;
t: TImage;
procedure Button1Click(Sender: TObject);
procedure Button2Click(Sender: TObject);
procedure Button3Click(Sender: TObject);
procedure FormClose(Sender: TObject; var CloseAction:
TCloseAction);
  procedure FormCreate(Sender: TObject);
private
  { private declarations }
public
  { public declarations }
end;

var
  Form1: TForm1;

implementation

{$R *.lfm}

{ TForm1 }

procedure TForm1.Button1Click(Sender: TObject);
begin
  if OpenPictureDialog1.Execute then //если файл выбран
    t.Picture.LoadFromFile(OpenPictureDialog1.FileName);//читаем его
end;

procedure TForm1.Button2Click(Sender: TObject);
begin
  with t.Picture.Bitmap do begin //обращаемся к битмапу
//середина изображения имеет координаты (width div 2,height div 2)
//width и height – свойства битмапа
    Canvas.Pixels[width div 2,height div 2]:=rgb(254,111,94);
//последний столбец – width - 1
    Canvas.Pixels[width-1,height div 2]:=rgb(0,51,153);
//последняя строка – height -1

```

```

    Canvas.Pixels[width div 2,height-1]:=rgb(221,72,20);
end;
end;

procedure TForm1.Button3Click(Sender: TObject);
begin
    if SavePictureDialog1.Execute then //если имя файла выбрано
        t.Picture.SaveToFile(SavePictureDialog1.FileName); //сохраняем
end;

procedure TForm1.FormClose(Sender: TObject; var CloseAction:
TCloseAction);
begin
    t.Free;//при закрытии формы освобождаем память от TImage
end;

procedure TForm1.FormCreate(Sender: TObject);
begin
end;
end.

```

Запускаем программу (рис. 4).

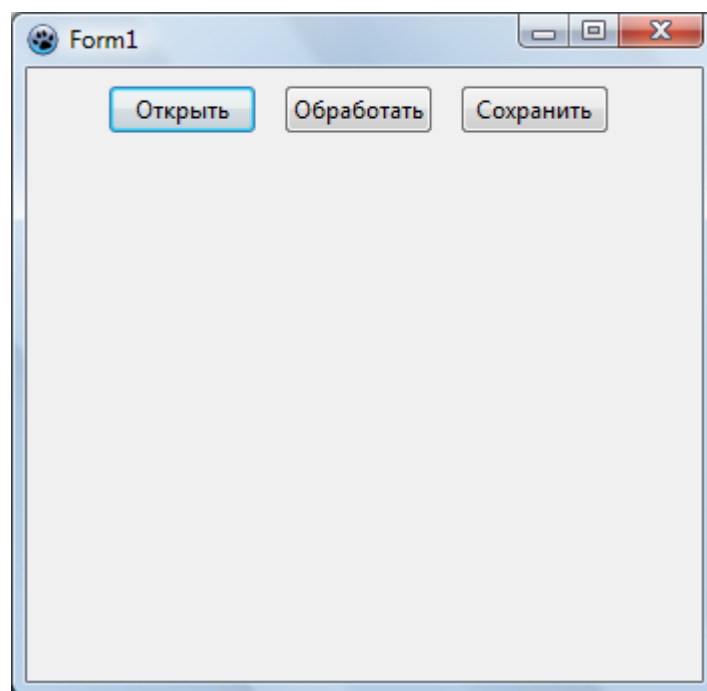


Рис. 4. Интерфейс запущенной программы

Выбираем файл, сохранённый с веб-камеры (рис.5) и открываем его.

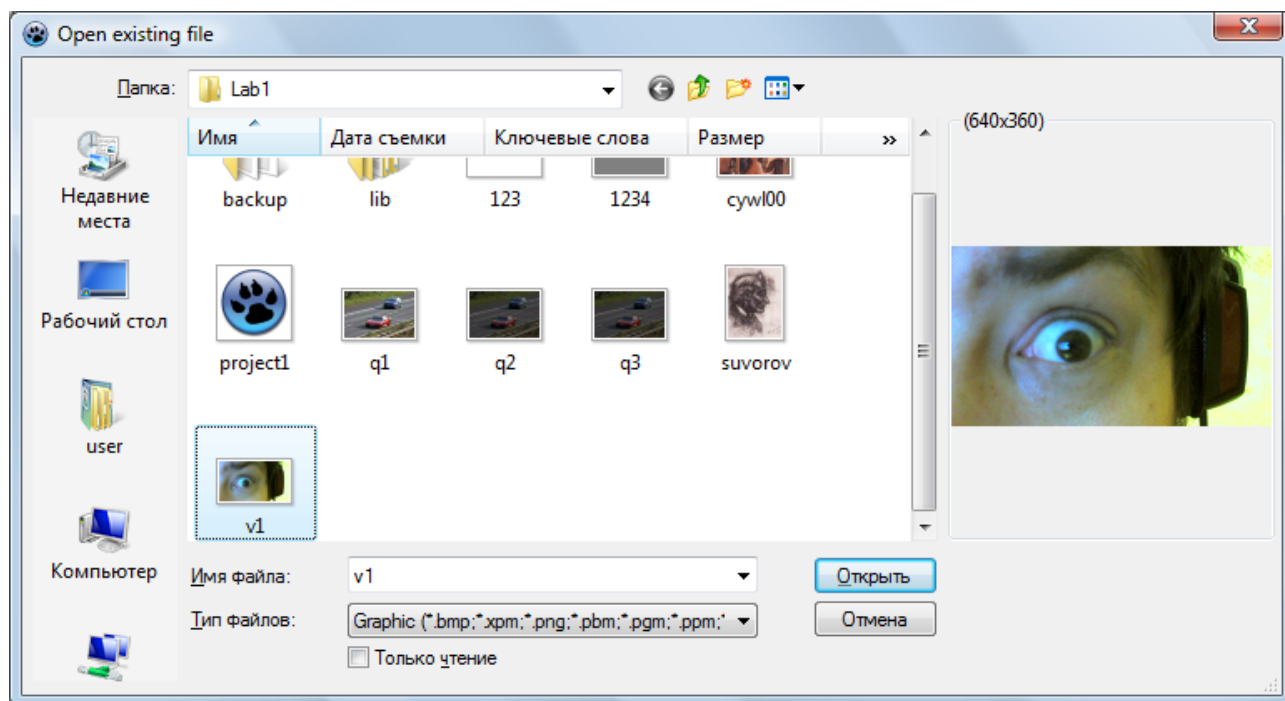


Рис. 5. Диалог выбора файла с исходным изображением

Изображение отображается на форме (рис. 6).



Рис. 6. Форма с изображением из файла

Обрабатываем изображение по индивидуальному заданию. На изображении (рис. 7) появляются три едва заметные точки.



Рис. 7. Форма с результатом обработки

Для сохранения результата нажимаем кнопку «Сохранить» на форме и задаём имя файла в диалоговом окне (рис. 8).

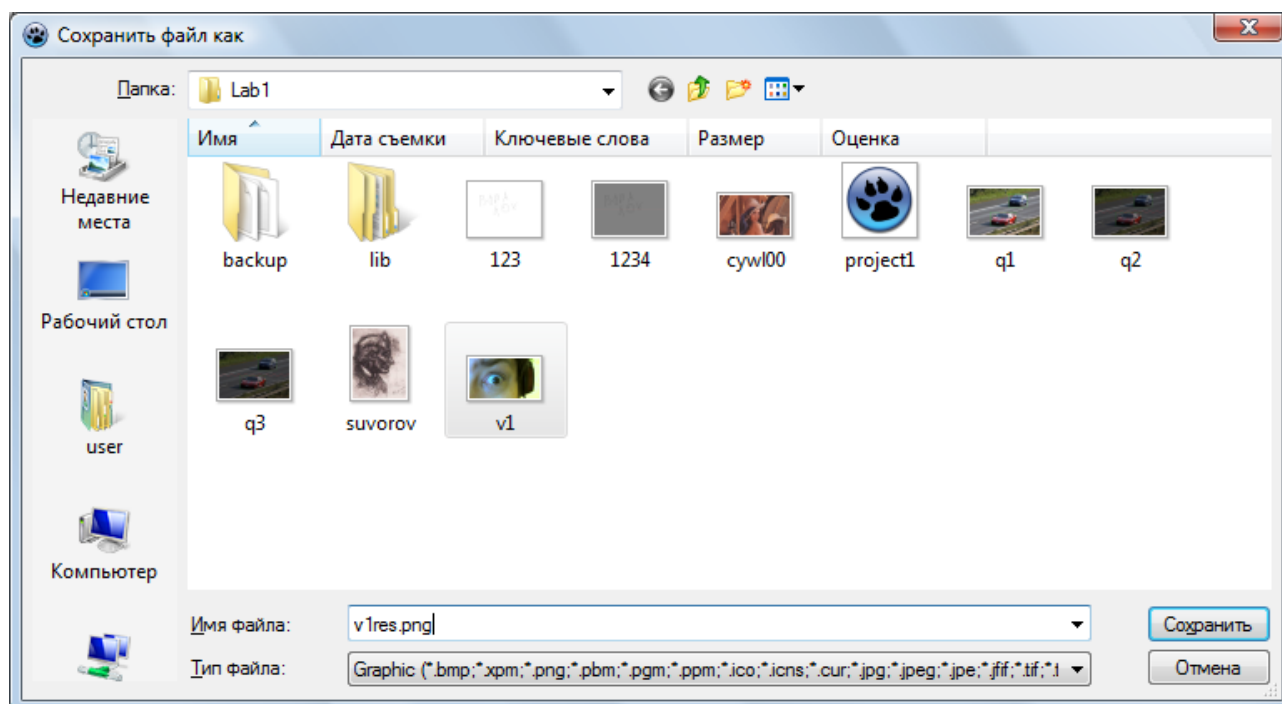


Рис. 8. Диалог сохранения файла с результатом

Находим три точки. Для этого открываем сохранённое изображение встроенной в операционную систему программой просмотра и увеличиваем области их размещения. Первая точка должна быть в центре изображения (рис. 9).

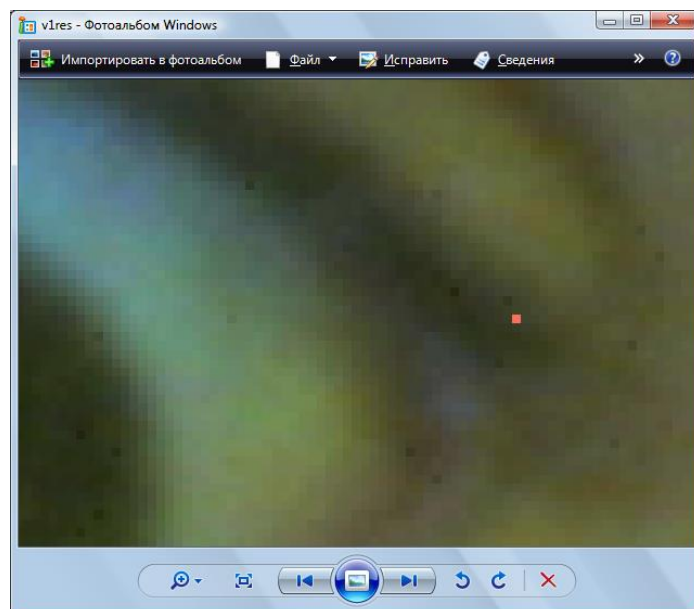


Рис. 9. Увеличенная центральная часть сохранённого изображения.

В центре изображения наблюдаем точку цвета горькой радости (254,111,94). Вторая точка должна быть в центре правого столбца (рис. 10).

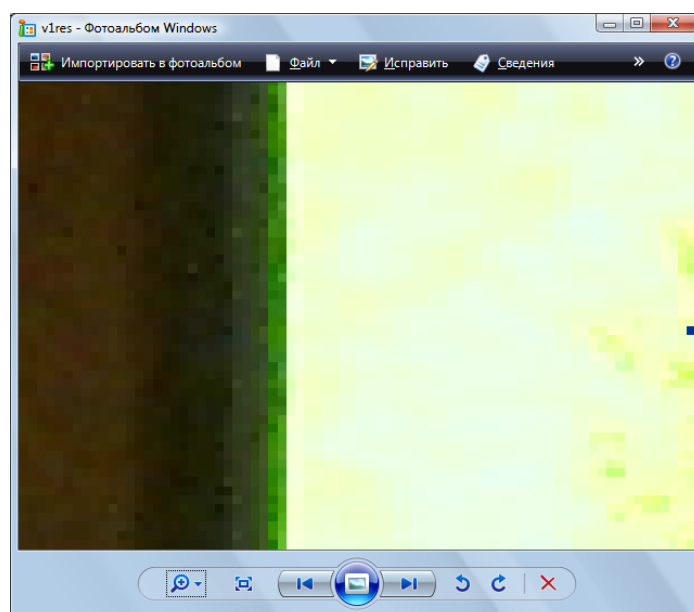


Рис. 10. Центр увеличенной правой части сохранённого изображения.

В центре правой части изображения наблюдаем точку цвета «пороховой синий» (0,51,153). Третья точка цвета «оранжевый Ubuntu» (221,72,20) должна быть в центре нижней строки (рис. 11), что мы и можем наблюдать.

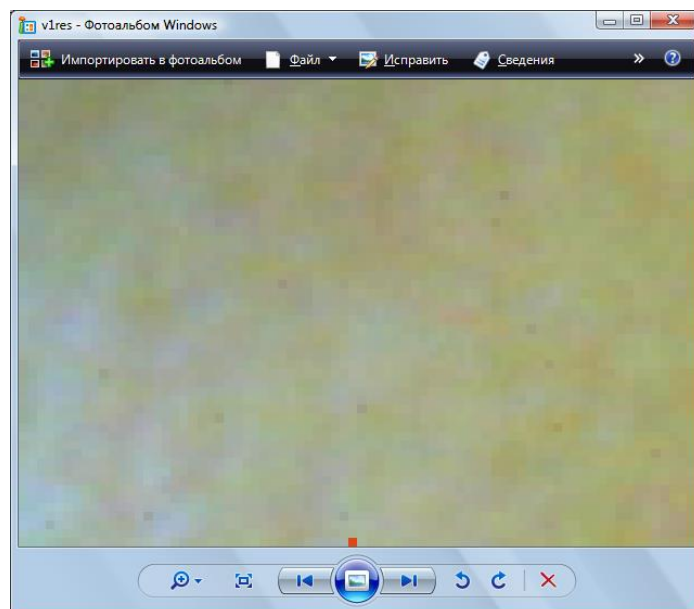


Рис. 11. Центр увеличенной нижней части сохранённого изображения.

Лабораторная работа № 2

Программа для формирования изображений

Цель работы – совершенствование базовых навыков обработки растровых изображений с помощью алгоритмических языков программирования.

Краткие теоретические сведения

В любом визуальном компоненте Lazarus, будь то форма или управляющий элемент, существует специальный объект, средствами которого выполняется рисование видимых частей компонента. Он называется *холстом* и оформлен в виде свойства **Canvas**. В частности, этот объект присутствует в объектах **TImage** и **TBitmap**. Объект **Canvas** имеет пять главных свойств.

1. **Pen: TPen** – объект для рисования линий и границ геометрических фигур:

1) **Color: TColor** – цвет вычерчиваемых пером линий;

2) **Style: TpenStyle** – стиль линий (рис. 1). Учитывается только для толщины линий 1 пиксель. Для толстых линий стиль всегда *psSolid* (сплошная);

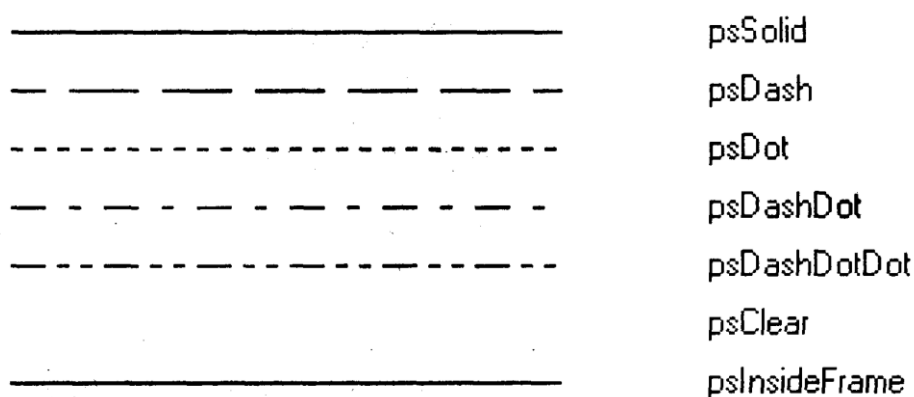


Рис. 1. Стили линий TPen

3) **Width: Integer** – толщина линий в пикселях экрана;

4) **Mode: TPenMode** – способ взаимодействия линий с фоном:

- PmBlack – линии всегда черные. Свойства *Color* и *Style* игнорируются;
- PmWhite – линии всегда белые. Свойства *Color* и *Style* игнорируются;
- PmNor – цвет фона не меняется (линии не видны);
- PmNot – инверсия цвета фона. Свойства *Color* и *Style* игнорируются;
- PmCory – цвет линий определяется свойством *Color* пера;
- PmNotCory – инверсия цвета пера. Свойство *Style* игнорируется;
- PmMergePenNot – комбинация цвета пера и инверсионного цвета фона;
- PmMaskPenNot – комбинация общих цветов для пера и инверсионного цвета фона. Свойство *Style* игнорируется;
- PmMergeNotPen – комбинация инверсионного цвета пера и фона;
- PmMaskNotPen – комбинация общих цветов для инверсионного цвета пера и фона. Свойство *Style* игнорируется;
- PmMerge – комбинация цветов пера и фона;
- PmNotMerge – инверсия цветов пера и фона. Свойство *Style* игнорируется;
- PmMask – общие цвета пера и фона;
- PmNotMask – инверсия общих цветов пера и фона;
- PmXor – объединение цветов пера и фона операцией *XOR*;
- PmNotXor – инверсия объединения цветов пера и фона операцией *XOR*.

2. **Brush: TBrush** – объект для заполнения фигур:

1) **Bitmap: TBitmap** – растровое изображение, которое будет использоваться кистью для заполнения. Если это свойство определено, свойства *Color* и *Style* игнорируются;

2) **Color: TColor** – цвет кисти;

3) **Style: TBrushStyle** – стиль кисти (рис. 2).

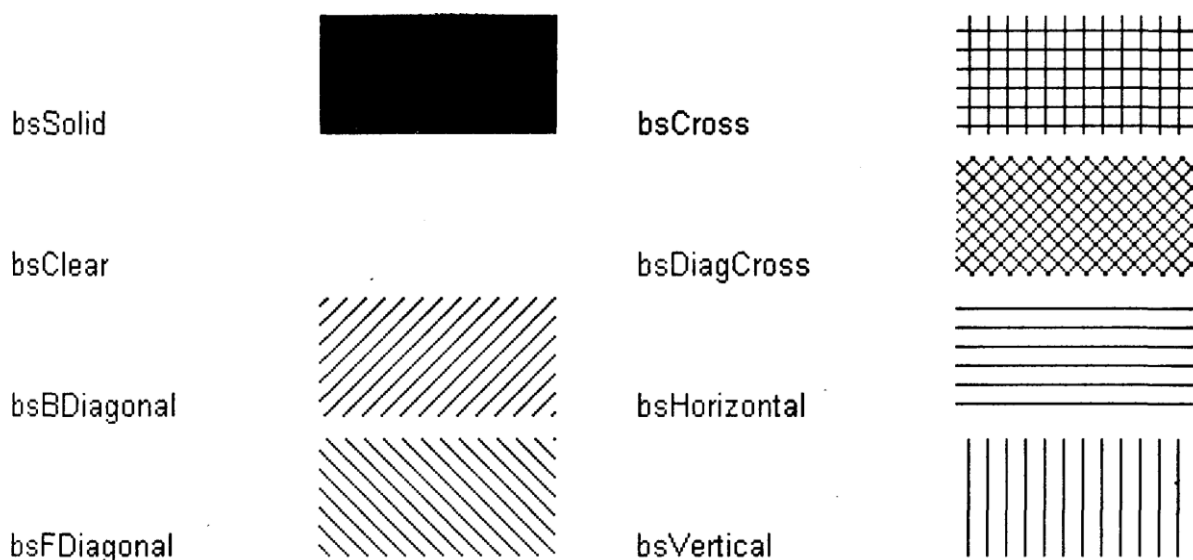


Рис. 2. Стили заполнения TBrush

3. **Font: TFont** – объект для вывода текста:

1) **Color: TColor** – цвет шрифта;

2) **Height: Integer** – высота шрифта в пикселях экрана;

3) **Name: TFontName** – имя шрифта. По умолчанию имеет значение *MS Sans Serif*;

4) **Pitch: TFontPitch** – способ расположения букв в тексте: значение *fpFixed* задаёт моноширинный текст, при котором каждая буква имеет одинаковую ширину; значение *fpVariable* определяет пропорциональный текст, при котором ширина буквы зависит от её начертания; *fpDefault* определяет ширину, принятую для текущего шрифта;

5) **PixelPerInch: Integer** – количество пикселей экрана на один дюйм реальной длины. Это свойство не следует изменять, т. к. оно используется системой для обеспечения соответствия экранного шрифта шрифту принтера;

6) **Size: Integer** – высота шрифта в пунктах (1/72 дюйма). Изменение этого свойства автоматически изменяет свойство *Height*, и наоборот;

7) **Style: TFontStyles** – стиль шрифта. Может принимать значение как комбинацию следующих признаков: *fsBold* (жирный), *fsItalic* (курсив), *fsUnderline* (подчёркнутый), *fsStrikeOut* (перечёркнутый).

4. **PenPos: TPoint** – объект для хранения текущей позиции рисования.

5. **Pixels[X, Y: Integer]: TColor** – объект для хранения пикселей холста.

Свойства **TPen**, **TBrush**, **TFont**, представляют собой инструменты для рисования. Сами по себе эти объекты не рисуют, но хранят в своих свойствах необходимые холсту параметры рисования.

Холст поддерживает такое понятие, как текущая позиция рисования. Текущая позиция хранится в свойстве **PenPos** и используется при рисовании прямых (прямая рисуется от текущей позиции до заданной). Выражение **PenPos.X** возвращает горизонтальную позицию, а **PenPos.Y** – вертикальную.

Благодаря свойству **Pixels** холст интерпретируется как двумерная матрица пикселей. Элемент, стоящий на пересечении столбца **X** и строки **Y** матрицы **Pixels**, кодирует цвет пикселя, который задаётся объектом типа **TColor**. Левый верхний пиксел изображения имеет координаты [0,0], ось **X** направлена вправо, а ось **Y** – вниз.

Все методы можно разбить на несколько групп в соответствии с их назначением.

1. Рисование прямых, ломаных и кривых линий;

1) **MoveTo(X, Y: Integer)** – перемещает указатель текущей позиции в заданную точку.

2) **LineTo(X, Y: Integer)** – рисует прямую линию от текущей позиции рисования (**PenPos.X**, **PenPos.Y**) до заданной (**X**, **Y**) и перемещает указатель текущей позиции в точку с координатами (**X**, **Y**).

3) **Polyline(Points: array of TPoint)** – рисует ломаную линию, соединяя точки массива **Points**. Для получения элемента массива по двум координатам может использоваться функция **Point(X, Y: Integer): TPoint**. Пример: с помощью метода **Polyline** можно нарисовать контур треугольника:

with Canvas do

begin

Pen.Style:=psDot; //Использовать пунктирную линию

Brush.Color:= RGB(0,255,255); // васильковый цвет

//Нарисовать треугольник)

Polyline([Point(20,90),Point(90,20),Point(180,90),Point(20,90)]);

end;

4) **Arc(X1, Y1, X2, Y2, X3, Y3, X4, Y4: Integer)** – рисует дугу эллипса, вписанного в прямоугольник с координатами (**X1**, **Y1**) и (**X2**,

Y2). Дуга определяется двумя радиусами эллипса, приходящимися через точки **(X3, Y3)** и **(X4, Y4)**. Дуга рисуется против часовой стрелки от точки пересечения эллипса с первым радиусом до точки пересечения со вторым радиусом (рис. 3);

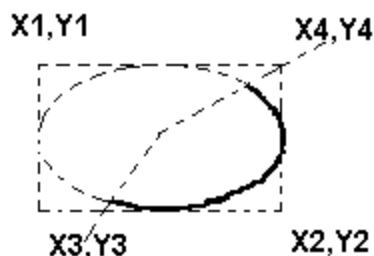


Рис. 3. Схема рисования дуги эллипса

5) **Chord(X1, Y1, X2, Y2, X3, Y3, X4, Y4: Integer)** – рисует хорду эллипса, вписанного в прямоугольник с координатами **(X1, Y1)** и **(X2, Y2)** Хорда лежит на прямой, проходящей через точки **(X3, Y3)** и **(X4, Y4)** (рис. 4).

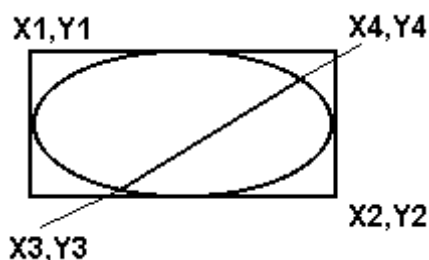


Рис. 4. Схема рисования хорды эллипса

2. Рисование геометрических фигур:

1) **Rectangle(X1, Y1, X2, Y2: Integer)** – рисует прямоугольник с левым верхним углом в точке **(X1, Y1)** и нижним правым углом в точке **(X2, Y2)**. Прямоугольник рисуется текущими атрибутами кисти и пера;

2) **RoundRect(X1, Y1, X2, Y2, X3, Y3: Integer)** – рисует прямоугольник с закруглёнными углами. Углы рисуются как четверти эллипса с шириной **X3** и высотой **Y3**. (рис. 5);

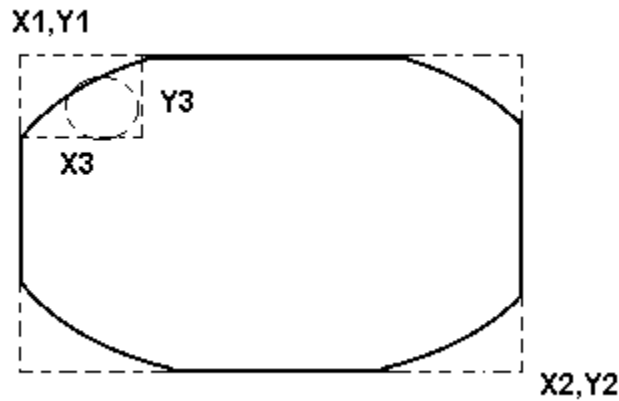


Рис. 5. Схема рисования прямоугольника с закруглёнными углами

3) **Ellipse(X1, Y1, X2, Y2: Integer)** – рисует эллипс, вписанный в прямоугольник с левым верхним углом в точке **(X1, Y1)** и нижним правым углом в точке **(X2, Y2)** (рис. 6). Рисуются текущими пером и кистью;

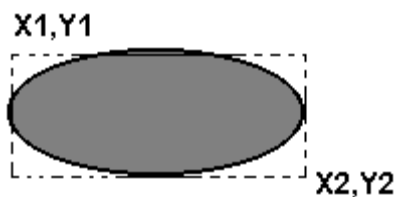


Рис. 6. Схема рисования эллипса

4) **Polygon(Points: array of TPoint)** – рисует ломаную линию, соединяя точки массива **Points**. Ломаная замыкается отрезком из последней точки в первую, и полученная фигура заполняется текущей кистью;

5) **Pie(X1, Y1, X2, Y2, X3, Y3, X4, Y4: Integer)** – рисует сектор эллипса, вписанного в прямоугольник с координатами **(X1, Y1)** и **(X2, Y2)** (рис. 7). Сектор определяется двумя радиусами эллипса, проходящими через точки **(X3, Y3)** и **(X4, Y4)**.

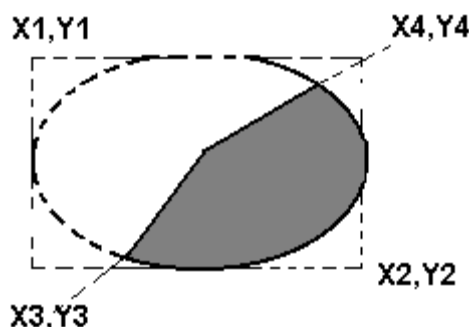


Рис. 7. Схема рисования сектора эллипса

Пример: эллипс, вписанный в холст.

with Canvas do

begin

Pen.Width:=10; //Использовать очень толстое перо

Pen.Style:=psInsideFrame; //и вписать фигуру

Ellipse(0,0,Width,Height); //Нарисовать вписанный эллипс

end;

3. Заполнение внутреннего пространства фигур:

1) FillRect(const Rect: TRect) – заполняет прямоугольник, используя текущую кисть. Для получения прямоугольника по его координатам может использоваться функция **Rect(X1, Y1, X2, Y2: Integer): TRect**;

2) FloodFill(X, Y: Integer; Color: TColor; FillStyle: TFillStyle) – заливает область экрана, используя атрибуты кисти. Заливка начинается в точке **(X, Y)** и продолжается во всех направлениях до достижения цветовой границы. Способ заполнения области определяется параметром **FillStyle**. Если параметр **FillStyle** равен *fsBorder*, заливка заполняется до тех пор, пока не обнаружится граница с цветом, указанным в параметре **Color**. Если же параметр **FillStyle** равен *fsSurface*, заливка выполняется, пока не обнаруживается цвет, указанный в параметре **Color**.

4. Вывод текста:

1) TextOut(X, Y: Integer; const Text: string) – выводит текущим шрифтом строку текста **Text** в прямоугольнике с левым верхним углом в точке **(X, Y)**;

2) **TextRect(Rect: TRect; X, Y: Integer; const Text: string)** – выводит текст в прямоугольнике **Rect**. Вывод за границы прямоугольника отсекается (см. метод **TextOut**);

3) **TextHeight(const Text: string): Integer** – возвращает высоту (в пикселах) строки **Text** при выводе её текущим шрифтом;

4) **TextWidth(const Text: string): Integer** – возвращает ширину (в пикселах) строки **Text** при выводе её текущим шрифтом.

5. Вывод картинок:

1) **Draw(X, Y: Integer; Graphic: TGraphic)** – выводит картинку **Graphic** в точке (X, Y). **TGraphic** – это абстрактный базовый класс для классов **TBitmap**, **TIcon**, **TMetaFile**, реализующих картинки конкретных форматов. Класс **TGraphic** как контейнер **Graphic** включен, в частности, в класс **TPicture**, так что, если есть объект **P: TPicture**, можно получить **P.Graphic: TGraphic**;

2) **StretchDraw(const Rect; TRect; Graphic: TGraphic)** – выводит картинку **Graphic**, масштабируя её до прямоугольника **Rect**;

3) **CopyRect(Dest: TRect; Canvas: TCanvas; Source: TRect)** – копирует прямоугольную область **Source** с холста **Canvas**. Результирующая картинка масштабируется до прямоугольника **Dest**;

4) Способ комбинации точек исходной картинки с точками холста определяется свойством холста **CopyMode**. Возможные значения этого свойства (**FinalPixel** означает цвет результирующего пиксела; **SrcPixel** – цвет пиксела копируемой картинки; **DestPixel** – цвет исходного пиксела полотна; **BrushPixel** – цвет кисти полотна):

- **CmBlackness** – всё результирующее изображение чёрное;
- **CmWhiteness** – всё результирующее изображение белое;
- **CmSrcCopy** – **FinalPixel** := **SrcPixel**;
- **CmSrcPaint** – **FinalPixel** := **SrcPixel** or **DestPixel**;
- **CmSrcAnd** – **FinalPixel** := **SrcPixel** and **DestPixel**;
- **CmSrcInvert** – **FinalPixel** := **SrcPixel** xor **DestPixel**;
- **CmSrcErase** – **FinalPixel** := **SrcPixel** and (not **DestPixel**);
- **CmNotSrcCopy** – **FinalPixel** := not **SrcPixel**;
- **CmNotSrcErase** – **FinalPixel** := not (**SrcPixel** or **DestPixel**);
- **CmDstInvert** – **FinalPixel** := not **DestPixel**;
- **CmMergePaint** – **FinalPixel** := (not **SrcPixel**) or **DestPixel**;
- **CmMergeCopy** – **FinalPixel** := **BrushPixel** and **SrcPixel**;
- **CmPatCopy** – **FinalPixel** := **BrushPixel**;

- CmPatPaint – FinalPixel := BrushPixel or (not SrcPixel) or DestPixel;
- CmPatInvert – FinalPixel := BrushPixel xor DestPixel;

Задание

1. Разработать алгоритм и программу для чтения из файла, преобразования и записи в файл графической информации по варианту. Программа включает в себя:

- создание нового изображения;
 - чтение изображения из файла;
 - добавление фрагмента заданной формы из заданной позиции изображения, загруженного из файла в заданную позицию созданного нового изображения попиксельно;
 - добавление осей системы координат и графика функции;
 - сохранение созданного изображения в файл на диске.
2. Ответить на контрольные вопросы.
3. Оформить отчёт.

Содержание отчёта

1. Титульный лист.
2. Формулировка задачи.
3. Схема алгоритма.
4. Программа на алгоритмическом языке программирования.
5. Форма.
6. Исходное и преобразованное изображения.
7. Ответы на контрольные вопросы.

Порядок выполнения работы

1. Ознакомление с целью работы, заданием, содержанием отчёта, порядком выполнения работы, индивидуальным вариантом, примером выполнения работы.

2. Ознакомление со средствами системы программирования для чтения, обработки и записи графики на примере и с помощью технической документации.

3. Разработка алгоритма в соответствии с вариантом задания.

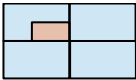
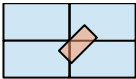
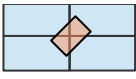
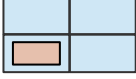
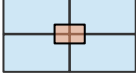
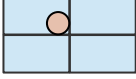
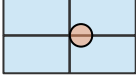
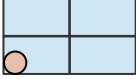
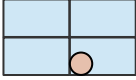
4. Разработка и отладка программы для решения задачи.
5. Обработка изображения.
6. Ответы на контрольные вопросы.
7. Оформление отчёта.

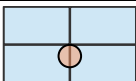
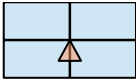
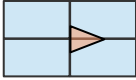
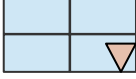
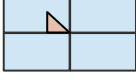
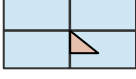
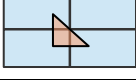
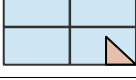
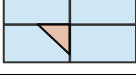
Индивидуальные задания

Каждый вариант индивидуальных заданий (табл. 1) включает в себя графическое изображение формы и позиции фрагмента на исходном изображении, размеченном проходящими через центр вертикальной и горизонтальной линиями, позицию для размещения фрагмента на новом изображении и функцию, график которой нужно нарисовать на новом изображении. Конкретные числовые параметры, детально определяющие форму и положение фрагментов, выбираются произвольно, но так чтобы не нарушить содержание условия задания.

Таблица 1

Варианты индивидуальных заданий

№	Форма и позиция фрагмента на исходном изображении	Регион размещения на новом изображении	Функция
1		Левый верхний угол	$\sin(x)$
2		Левый нижний угол	$\cos(x)$
3		Правый верхний угол	$\tan(x)$
4		Правый нижний угол	$x \cdot \sin(x)$
5		Центр	$\exp(x)$
6		Левый край по центру	$\ln(x)$
7		Правый край по центру	x^2
8		Верхний край по центру	x^3
9		Нижний край по центру	$x^{(1/2)}$

№	Форма и позиция фрагмента на исходном изображении	Регион размещения на новом изображении	Функция
10		В заданную позицию	$x^{(1/3)}$
11		Левый верхний угол	$1/x$
12		Левый нижний угол	$(1/x)^2$
13		Правый верхний угол	$1/(x-1)^2$
14		Правый нижний угол	$\sin(x^2)$
15		Центр	$1/\exp(x)$
16		Левый край по центру	$ x-1 $
17		Правый край по центру	$x*\cos(x)$
18		Верхний край по центру	$x^{(3/7)}$
19		Нижний край по центру	$\sin^2(x)$
20		В заданную позицию	2^x

Контрольные вопросы

1. В чём основные отличия свойств оптического излучения от других видов излучения?
2. Почему зрение так важно для человека? Чем его можно заменить для восприятия той же информации об окружающем мире?
3. Почему человек воспринимает излучение в пределах именно этой небольшой части оптического диапазона, называемой видимым светом?
4. Опишите характеристики трёх самосветящихся источников, которые чаще всего попадают на глаза.

5. В каких случаях имеет смысл корректировать спектр источника излучения?
6. Как получить анизотропный диффузный рефлектор и как сделать его изотропным?
7. Почему функция яркости непрерывна, в то время как количество излучаемых фотонов дискретно?
8. Опишите достоинства и недостатки каждого из способов разделения света на зоны.
9. Какой из способов размещения чувствительных элементов фотоприёмника предпочтителен – регулярный (как на матрице) или хаотический (как на сетчатке глаза)?
10. Почему оптическая система глаза, состоя из небольшого количества элементов, обеспечивает сравнительно высокое качество изображения?
11. Какими параметрами должен обладать искусственный датчик излучения, чтобы не уступать глазу?
12. Какие параметры, влияющие на дискретизацию и квантование, следует улучшать в первую очередь?
13. Какова система координат холста?
14. Как с помощью программы вывести на холст движущиеся изображения?

Пример выполнения работы

Задание:

1. Создание нового изображения.
2. Чтение изображения из файла.
3. Добавление фрагмента формы круг из заданной позиции изображения, загруженного из файла в заданную позицию созданного нового изображения попиксельно.
4. Добавление осей системы координат и графика функции $\text{sinc}(x)$.
5. Сохранение созданного изображения в файл на диске.

В соответствии с логикой выполнения действий, указанных в задании, пользователь должен выполнить следующие действия:

1. Задать ширину и высоту создаваемого изображения в пикселях.

2. Создать новое изображение заданного размера.
3. Открыть файл с имеющимся изображением.
4. Задать координаты центра и радиус круга, который нужно скопировать на новое изображение.
5. Задать координаты размещения фрагмента на новом изображении.
6. Скопировать фрагмент.
7. Нарисовать оси системы координат.
8. Нарисовать график функции, определив масштаб.
9. Сохранить результат в файл на диске.

В соответствии с составом и последовательностью действий, ожидаемых от пользователя, разместим на форме (рис. 8) следующие компоненты: **TOpenPictureDialog**, **TSavePictureDialog**, **2 TImage**, **6 Button**, **7 TEdit**, **11 Label**. Для **TImage** сразу зададим нужные значения свойствам **Proportional** и **Stretch**. Добавим текст на кнопки, изменив соответствующие свойства. Разработаем алгоритмы обработчиков событий для нажатия кнопок и закрытия формы. Рассчитывая эту программу для чтения любых типов графических файлов и получения полноцветного результата, добавим функцию **ConvertTo24Bit** преобразования в формат пикселей RGB.

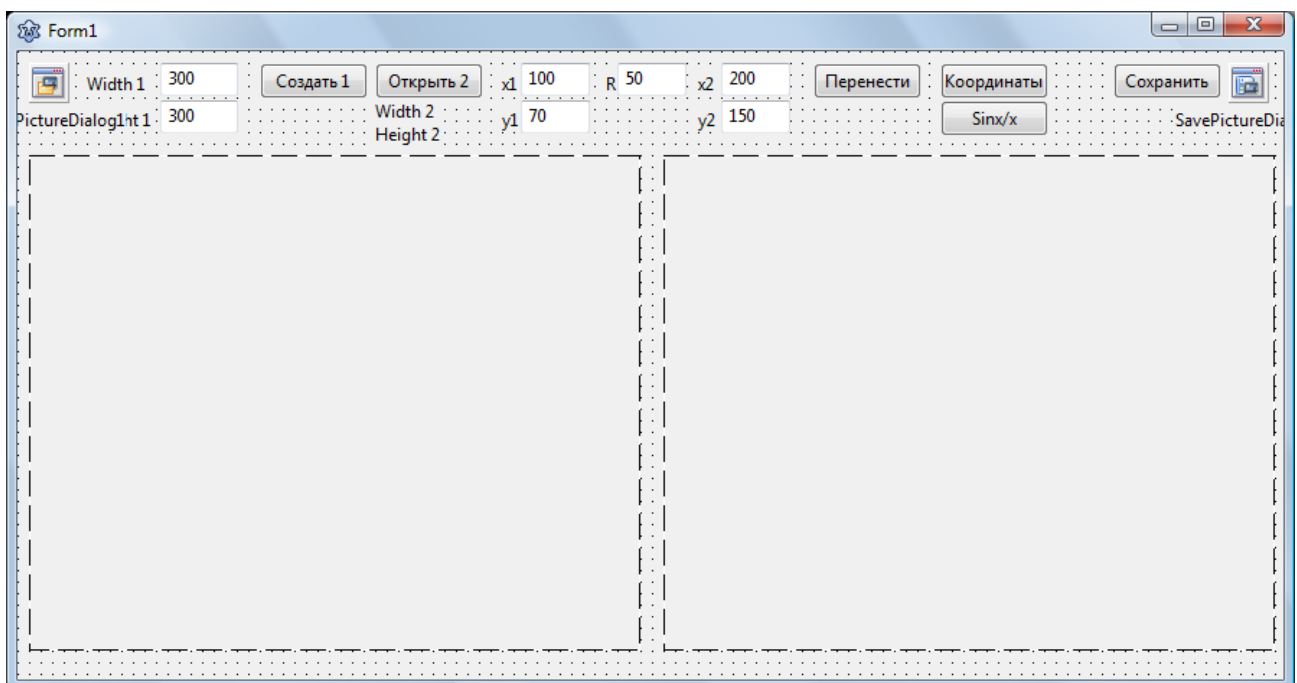


Рис. 8. Форма программы

При нажатии на кнопку «Создать 1» для битмапа Image1 выполняется очистка, задаётся формат пикселей pf24bit, из соответствующих компонентов Edit считывается ширина и высота, выполняется заполнение белым фоном.

При нажатии на кнопку «Открыть 2» выполняется чтение изображения из файла. Любое считанное изображение переводится в полноцветный формат. Если формат считанного изображения неполноцветный, вызывается процедура преобразования и выводится сообщение.

Для копирования круга считываем координаты размещения центра круга на новом изображении x_1 , y_1 , координаты центра круга на источнике x_2 , y_2 и радиус R . Вычисляем разницу между координатами центра dx , dy . Рассматривая в цикле все точки нового изображения, проверяем условие попадания внутрь нового круга. Если оно выполняется, с учётом разности координат проверяем условие попадания в исходное изображение. Если не выходим за пределы нового изображения, считываем три составляющих пикселя исходного изображения и записываем их в результат, иначе в результат записываем пиксели чёрного цвета. Если попадаем не внутрь круга, а на границу, выводим точку определённого цвета в новое и исходное изображение.

График будем рисовать в правой полуплоскости. Функция принимает положительные и отрицательные значения, так что ось ОХ располагаем по центру. Деления намечаем через каждые десять пикселей. Выводим заголовок. Цвет графика задаём светло-синий. Выбираем масштаб по ОХ: единица – каждое деление, т. е. десять пикселей. Для соответствия этому масштабу при вычислении значения аргумента функции в позиции каждого пикселя номер пикселя нужно разделить на 10. Максимальное значение функции $\sin(x)/x - 1$ при $x=0$. Выбираем масштаб по ОУ – единица на верхнем делении. Верхнее деление находится в позиции (10,20). Так как ось ОХ (и 0 на ОУ) расположены на уровне середины графика ($\text{height} \div 2$), а верхнее деление – на 20 пикселей ниже верхней границы, единица будет занимать $\text{height} \div 2 - 20$ пикселей, и для соответствия выбранному масштабу значение функции нужно умножить на это число.

Сохраняется изображение как в предыдущей работе.

Кодируем. Получаем модуль примерно следующего содержания:

```
unit Unit1;
```

```
{ $mode objfpc } { $H+ }
```

```
interface
```

```
uses
```

```
Classes, SysUtils, FileUtil, Forms, Controls, Graphics, Dialogs, StdCtrls,  
GraphType, IntfGraphics, LCLType, LCLProc, LCLIntf, ExtCtrls,  
ExtDlgs, fpImage, crt, math;
```

```
type
```

```
{ TForm1 }
```

```
TForm1 = class(TForm)
```

```
    Button1: TButton;
```

```
    Button2: TButton;
```

```
    Button3: TButton;
```

```
    Button4: TButton;
```

```
    Button5: TButton;
```

```
    Button6: TButton;
```

```
    Edit1: TEdit;
```

```
    Edit2: TEdit;
```

```
    Edit3: TEdit;
```

```
    Edit4: TEdit;
```

```
    Edit5: TEdit;
```

```
    Edit6: TEdit;
```

```
    Edit7: TEdit;
```

```
    Image2: TImage;
```

```
    Label1: TLabel;
```

```
    Label10: TLabel;
```

```
    Label11: TLabel;
```

```
    Label12: TLabel;
```

```
    Label13: TLabel;
```

```
    Label3: TLabel;
```

```

Label5: TLabel;
Label6: TLabel;
Label7: TLabel;
Label8: TLabel;
Label9: TLabel;
OpenPictureDialog1: TOpenPictureDialog;
SavePictureDialog1: TSavePictureDialog;
Image1: TImage;
procedure Button1Click(Sender: TObject);
procedure Button2Click(Sender: TObject);
procedure Button3Click(Sender: TObject);
procedure Button4Click(Sender: TObject);
procedure Button5Click(Sender: TObject);
procedure Button6Click(Sender: TObject);
procedure FormClose(Sender: TObject; var CloseAction:
TCloseAction);
private
  { private declarations }
public
  { public declarations }
end;

var
  Form1: TForm1;

implementation

{$R *.lfm}

{ TForm1 }
//Функция преобразования в pf24bit
function ConvertTo24Bit(aValue: TBitmap;BackColor: TColor): TBitmap;
begin
  Result := Tbitmap.Create;//Преобразовывается bitmap avalue в result
  with Result do
    if aValue.PixelFormat <> pf24bit then //если исходный bitmap не 24bit
    begin
      PixelFormat:= pf24bit; //формат результата 24 бита
    end;
  end;
end;

```

```

    SetSize(aValue.Width,aValue.Height); //размер result как у avalue
    Canvas.FloodFill(0,0,BackColor,fsBorder); //заливаем result цветом
    Canvas.Draw(0,0,aValue); //выводим в result картинку из avalue
end
else //если исходный битмап 24bit
    Assign(AValue); //результат полагаем соответствующим исходнику
end;

procedure TForm1.Button1Click(Sender: TObject); //Кнопка Создать 1
begin
//С битмапом расположенного на форме TImage
with Image1.Picture.Bitmap do
begin
    Clear; //а вдруг там мусор
    PixelFormat:=pf24bit; //формат пикселей
    Width:=StrToInt(Edit6.Text); //Ширина из Edit
    Height:=StrToInt(Edit7.Text); //Высота из Edit
    Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder); //Белый фон
end;
end;

procedure TForm1.Button4Click(Sender: TObject); //Кнопк Открыть 2
begin
    if OpenPictureDialog1.Execute then
        Image2.Picture.LoadFromFile(OpenPictureDialog1.FileName);
    Label6.Caption:=IntToStr(Image2.Picture.Width); //Для справки
    Label8.Caption:=IntToStr(Image2.Picture.Height); //выводим размер
//Если формат пикселей не такой как надо
    if Image2.Picture.bitmap.PixelFormat <> pf24bit then
        begin
            showmessage('invalid pixel color format. trying to convert');
            image2.picture.bitmap.assign(convertto24bit(image2.Picture.bitmap,RGB(
255,255,255))); //воспользуемся функцией преобразования
        end;
end;

procedure TForm1.Button2Click(Sender: TObject); //Перенести
var

```

```

cr,cb,cg:byte;
px, py,x1,y1,x2,y2,r,dx,dy: Integer;
begin
x1:=strtoint(edit1.text);//Считываем параметры окружности
x2:=strtoint(edit4.text);//и координат переноса
y1:=strtoint(edit2.text);
y2:=strtoint(edit5.text);
r:=strtoint(edit3.text);
dx:=x1-x2;
dy:=y1-y2;
for py:=0 to Image1.picture.bitmap.Height-1 do begin//для всех строк
  for px:=0 to Image1.picture.bitmap.Width-1 do begin//для пикселей
//Если попадает в окружность
    if (round(sqrt(power(py-y1,2)+power(px-x1,2))) < r) then
//Если не выходит за пределы изображения
      if ((px-dx)>0) and ((py-dy)>0) and ((px-dx)<image2.Picture.Width)
and ((py-dy)<image2.Picture.Height) then
        begin //Считываем три составляющих
          cr:=getrvalue(Image2.Picture.bitmap.canvas.pixels[px-dx,py-dy]);
          cg:=getgvalue(Image2.Picture.bitmap.canvas.pixels[px-dx,py-dy]);
          cb:=getbvalue(Image2.Picture.bitmap.canvas.pixels[px-dx,py-dy]);
//Записываем куда надо
          Image1.Picture.bitmap.Canvas.pixels[px,py]:=RGB(cr,cg,cb);
        end
//Если не откуда считать
      else Image1.Picture.bitmap.Canvas.pixels[px,py]:=RGB(0,0,0)
      else if (round(sqrt(power(py-y1,2)+power(px-x1,2))) = r) then
        begin
//Обводим контуром то, что скопировали для красоты.-)
          Image1.Picture.bitmap.Canvas.pixels[px,py]:=RGB(127,255,127);
          with Image2.Picture.Bitmap do
            if ((px-dx)>0)and((py-dy)>0)and((px-dx)<Width)and((py-
dy)<Height) then //Обводим контуром исходный фрагмент для справки
              Canvas.pixels[px-dx,py-dy]:=RGB(255,127,127);
            end;
          end;
        end;
      end;
    end;
  end;
end;
end;

```

```

procedure TForm1.Button6Click(Sender: TObject); //Координаты
var i:integer;
begin //Рисуем декартову сетку из отрезков
  With Image1.Picture.bitmap do //на битмапе новой картинки
    begin
      h:=height; //сохраняем высоту
      w:=width; //сохраняем ширину
      with canvas do //на холсте битмапа новой картинки
        begin
          Polyline([point(10,h-10),point(10,10)]); //ось OY
          Polyline([point(5,20),point(10,10),point(15,20)]); //стрелочка OY
          Polyline([point(8,h div 2),point(w - 10, h div 2)]); //ось OX
          Polyline([point(w-20,h div 2-5),point(w-10,h div 2),point(w-20,h
div 2+5)]); //стрелочка OX
          for i:=0 to (h - 40) div 20 do //рисуем деления на OY
            begin //относительно центра оси, размером 2 с шагом 10 каждое
              Polyline([point(10,i*10 + h div 2),point(8, i*10 + h div 2)]); //вниз
              Polyline([point(10,h div 2 - i*10),point(8,h div 2 - i*10)]); //вверх
            end;
          for i:=0 to (w - 40) div 10 do //рисуем деления на OX с шагом 10
            Polyline([point((i+1)*10,h div 2),point((i+1)*10,h div 2 + 2)]);
            textout(1,h div 2 + 1,'0'); //подписываем 0
            TextOut(w - 20, h div 2 + 6, 'x'); //подписываем O
            textout(20,1,'y'); //подписываем ось OY
          end;
        end;
      end;
    end;
  end;
end;

```

```

procedure TForm1.Button5Click(Sender: TObject);
var i:integer;
begin
  With Image1.Picture.bitmap do //рисуем на новом битмапе
    begin
      canvas.textout(width div 2,1,'y=sin(x)/x'); //заголовок графика
      canvas.Pen.Color:=RGB(127,127,255); //цвет точек
      canvas.moveto(10,20 ); //первая точка (с учётом масштаба) на оси
      for i:=1 to width - 20 do //оси OY до конца оси OX со сдвигом

```

```

//рисуем отрезками с учётом масштаба по вертикали и горизонтали
  canvas.lineto(i+10,height div 2-round(sin(i/10)/(i/10)*(height div 2-20)));
  end;
end;

procedure TForm1.Button3Click(Sender: TObject);//Сохранить
begin
  if SavePictureDialog1.Execute then
    Image1.Picture.SaveToFile(SavePictureDialog1.FileName);
end;

procedure TForm1.FormClose(Sender: TObject; var CloseAction:
TCloseAction);
begin
  Image1.Free; //освобождаем память
  Image2.Free;
end;

end.

```

Запускаем программу (рис. 9).

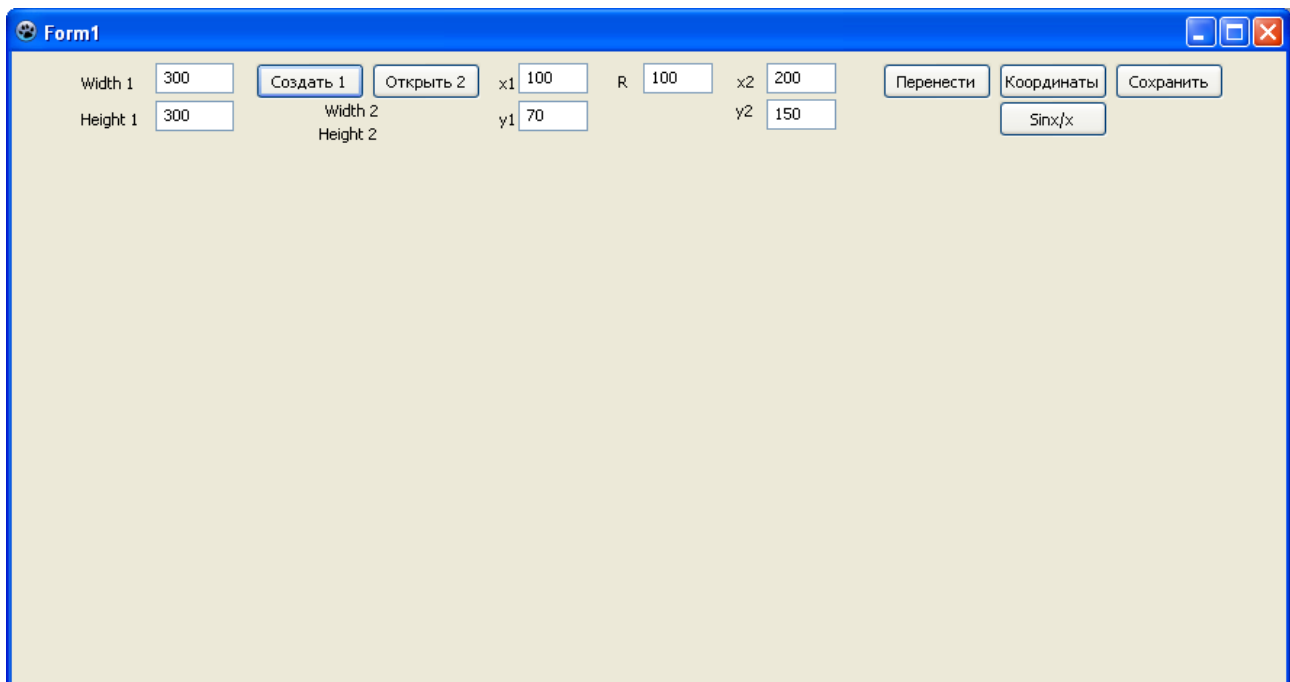


Рис. 9. Окно программы после запуска

Создаём изображение 300x300 пикселей (оно заливается белым) (рис. 10).

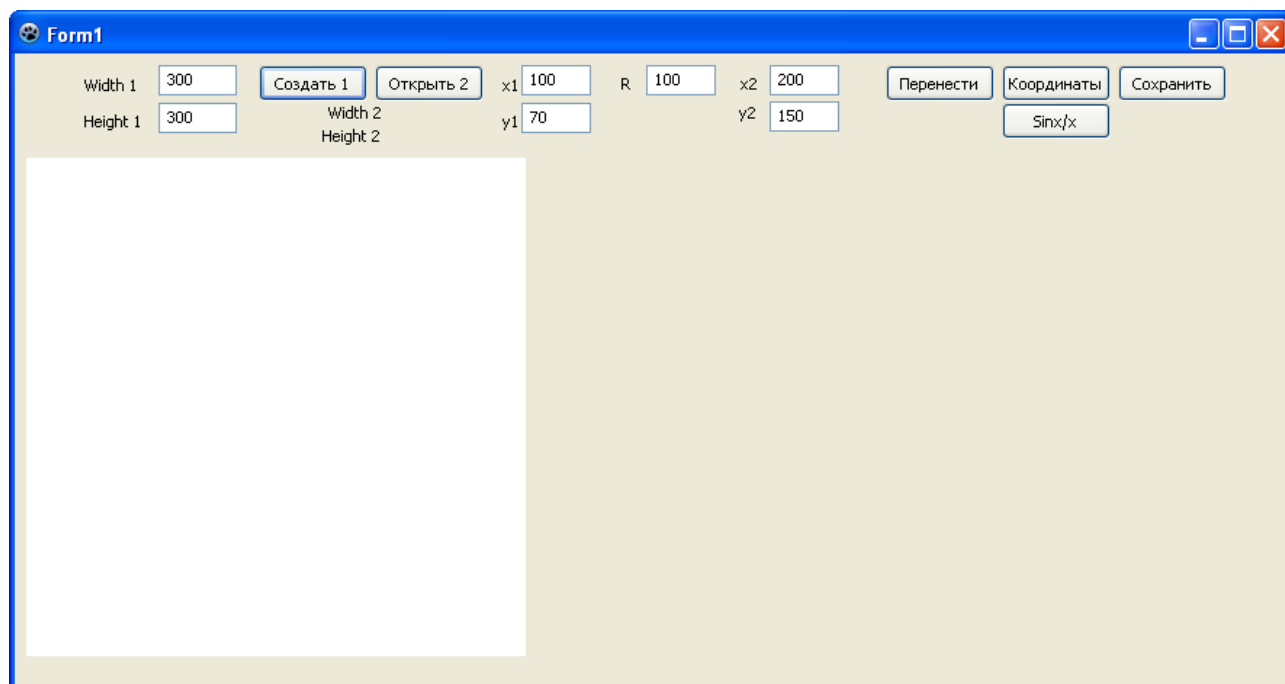


Рис. 10. Окно с созданным новым изображением

Открываем изображение из файла (рис. 11).

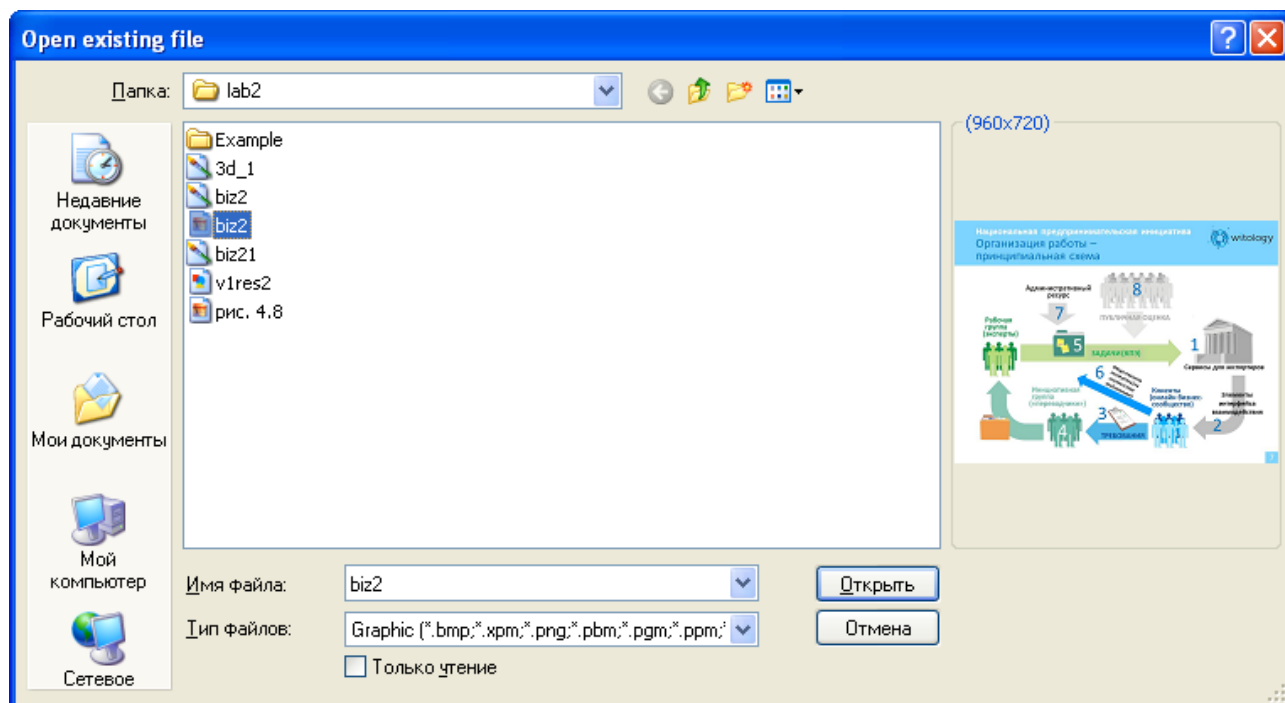


Рис. 11. Диалоговое окно выбора файла изображения

Так как оно не 24–битное, выдаётся сообщение о преобразовании (рис. 12).

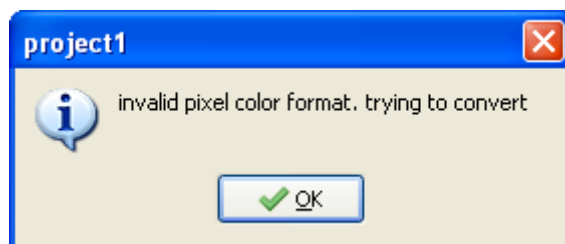


Рис. 12. Сообщение о преобразовании

Соглашаемся. Изображение считывается на форму (рис. 13).

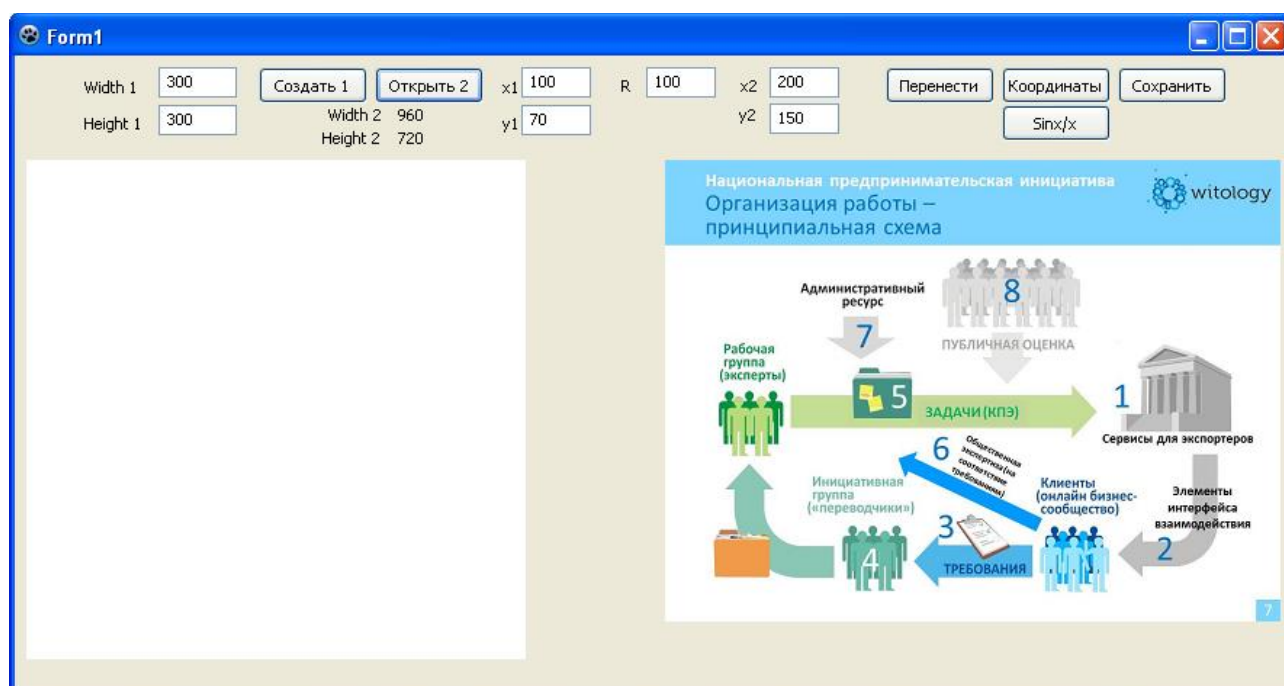


Рис. 13. Окно с загруженным из файла изображением

Переносим круговой фрагмент изображения радиусом 100 пикселей из позиции (200,150) в позицию (100,70) нового изображения (рис. 14).

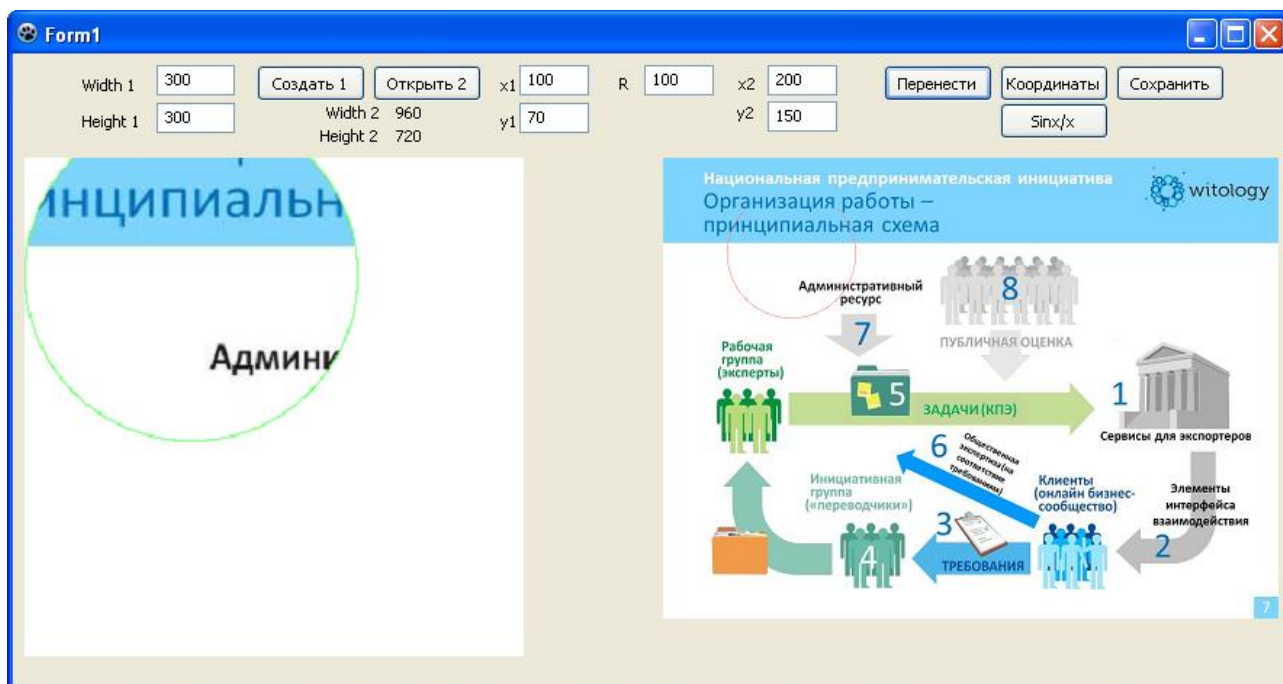


Рис. 14. Перенос фрагмента изображения

Рисуем координатные оси (рис. 15).

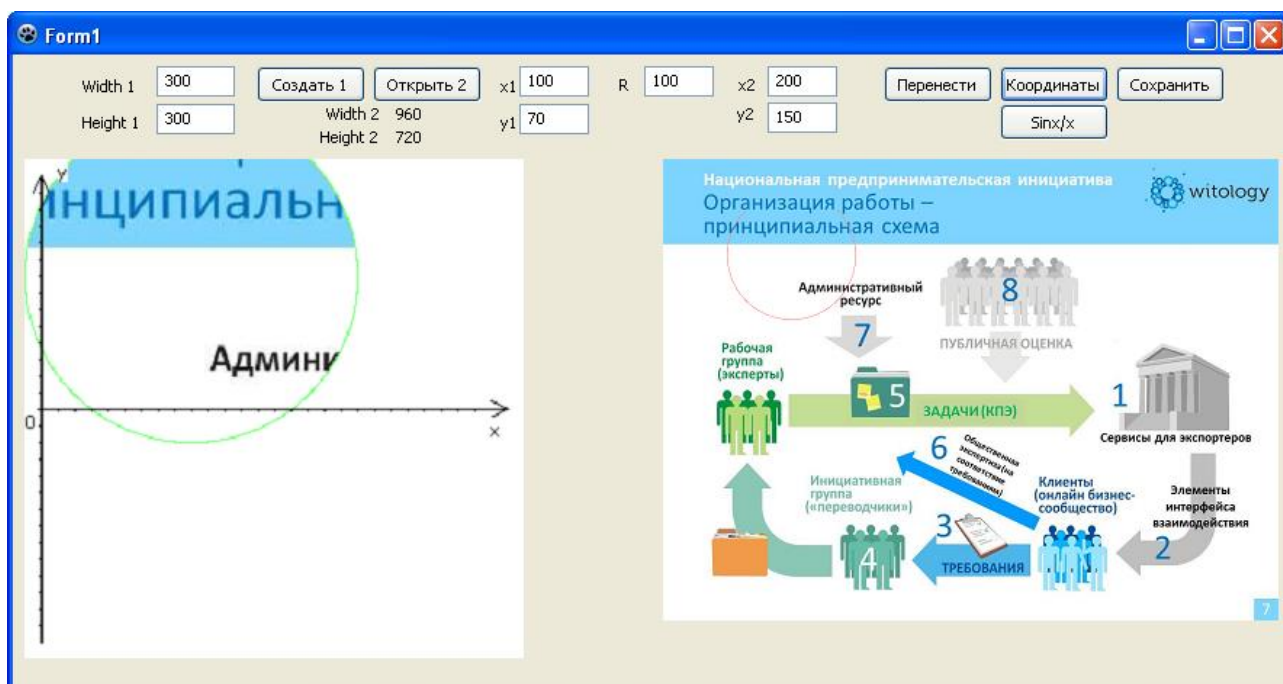


Рис. 15. Вывод координатных осей

Рисуем график функции (рис. 16).

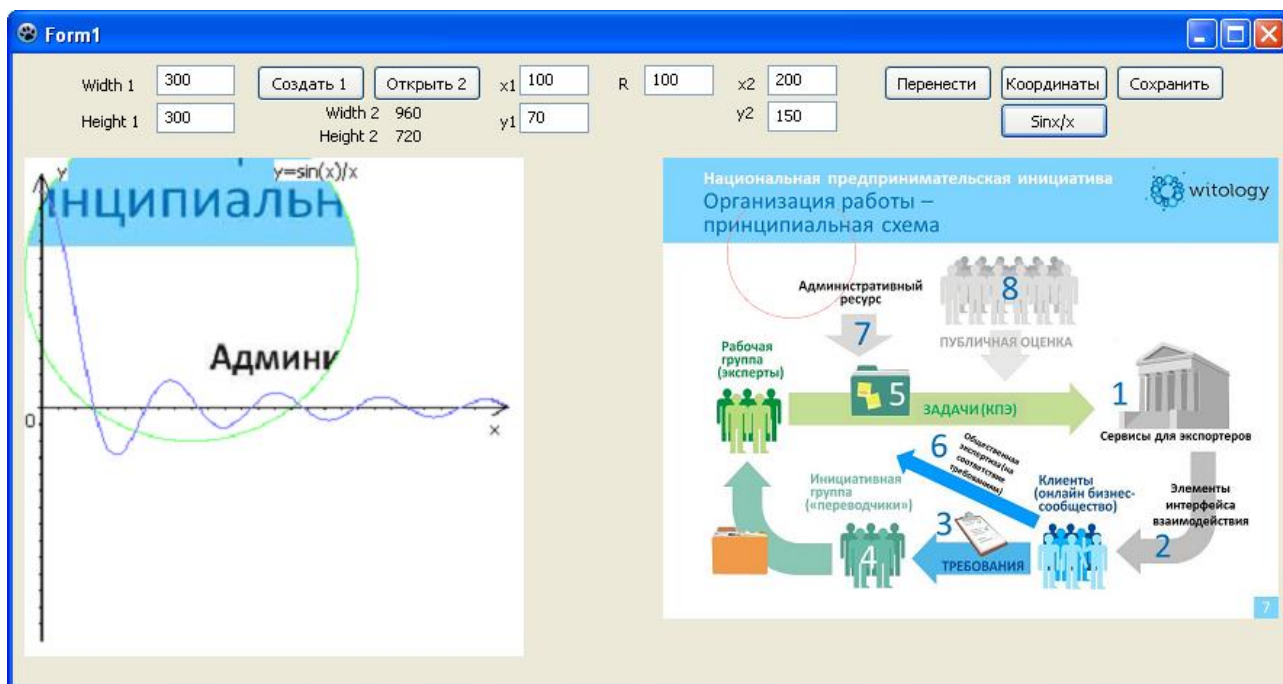


Рис. 16. Вывод графика функции

Сохраняем результат в файл (рис. 17).

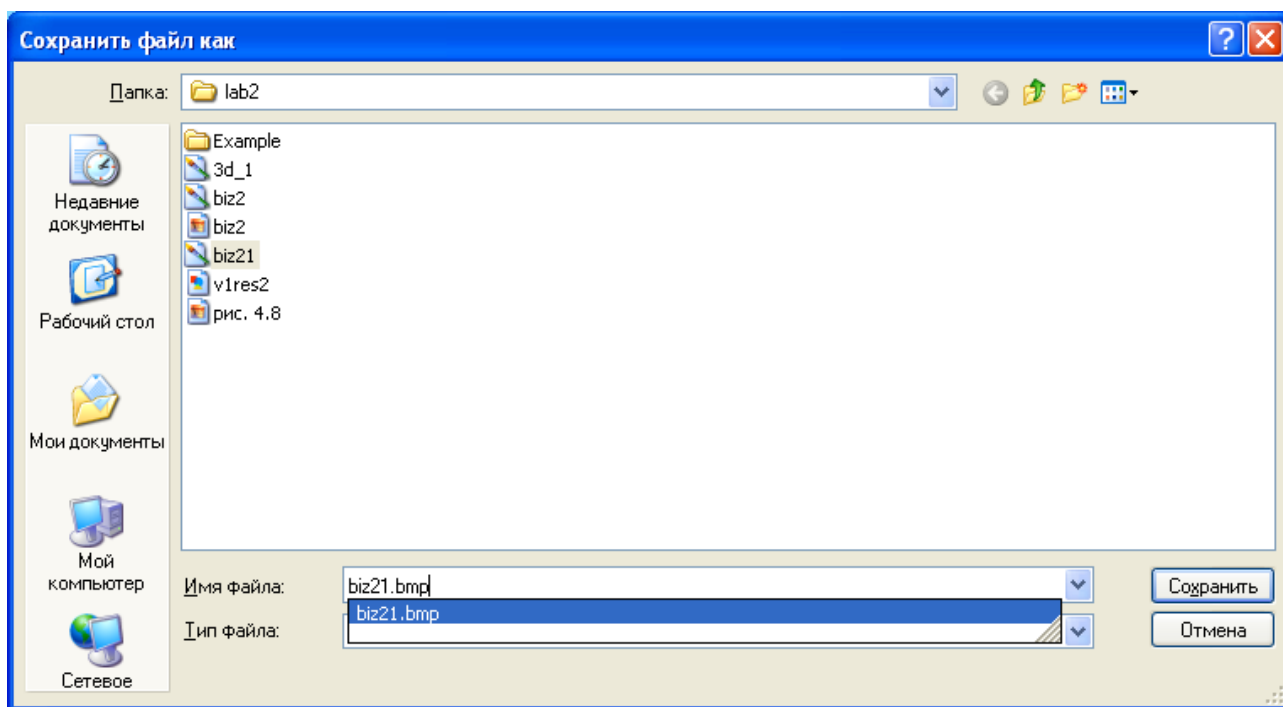


Рис. 17. Диалоговое окно сохранения файла изображения

Смотрим на результат (рис. 18).

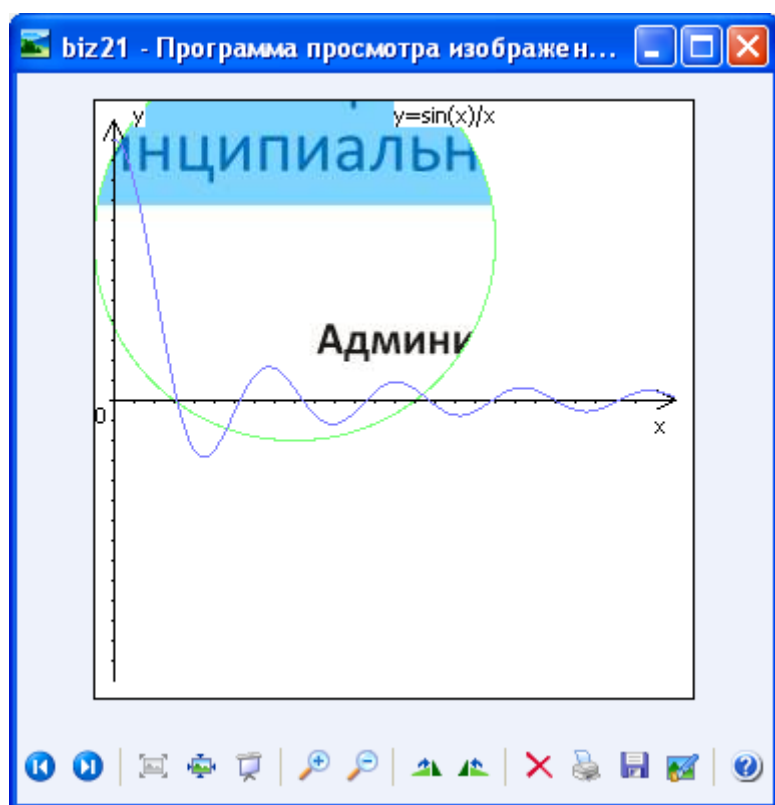


Рис. 18. Сохранённый файл изображения во встроенной программе просмотра изображений

Лабораторная работа № 3

Растеризация отрезков прямых

Цель работы – изучение алгоритмов растеризации отрезков, создание программы для визуализации работы алгоритмов.

Краткие теоретические сведения

Дисплей накладывает технические ограничения на представление графической информации. Изображение представляется в виде точек, и каждой точке соответствует вполне определённый атрибут – цвет и интенсивность. Таким образом, графические (а точнее – дисплейные) примитивы хранятся в памяти в виде совокупности образующих их точек, называемых *пикселями*.

В процессе вывода изображение (кадр) помещается в битовую карту (видеопамять), которая организована по линейному принципу и расположена в адресном пространстве центрального процессора. Но прежде чем будет сформирован растровый образ в видеопамяти, файл должен быть преобразован в соответствующий формат вывода. Чтобы преобразовать двумерный точечный образ, необходимо проанализировать и преобразовать значение каждой точки, что резко снижает производительность видеоподсистемы и системы в целом. Когда изображение может быть представлено в виде совокупности графических примитивов, т. е. в векторном формате, то процесс преобразования сводится к растеризации отдельных отрезков, окружностей, областей заливки и т. д., следовательно, количество анализируемых и синтезируемых точек намного меньше, чем при обработке растровых изображений. К тому же файлы векторного формата в среднем на порядок меньше по объёму растровых файлов, поскольку последние хранят описание каждого пикселя и объём резко увеличивается с увеличением разрешения.

Базовые алгоритмы предназначены для реализации (растеризации) графических примитивов, таких как отрезки, окружности, эллипсы и т. д. (Графические примитивы, описанные на языке высокого уровня, – это векторная форма изображения.) Кроме того, в эту группу попадают такие операции машинной графики, как закраска сплошных областей и отсечение отрезков и многоугольников.

Необходимость разработки специальных алгоритмов построения геометрических объектов объясняется тем, что сами эти объекты имеют непрерывную (аналоговую) природу, а изображение строится, как правило, на экране растрового дисплея, т. е. получаемое изображение носит дискретный характер. Это означает, например, что нельзя построить непосредственно отрезок, соединяющий две заданные точки экрана. Решение поставленной задачи очевидно лишь для трёх типов отрезков: горизонтальных, вертикальных и под углом 45° (рис. 1). Процесс нахождения пикселей, наилучшим образом аппроксимирующих заданный отрезок, называется разложением отрезка в растр.

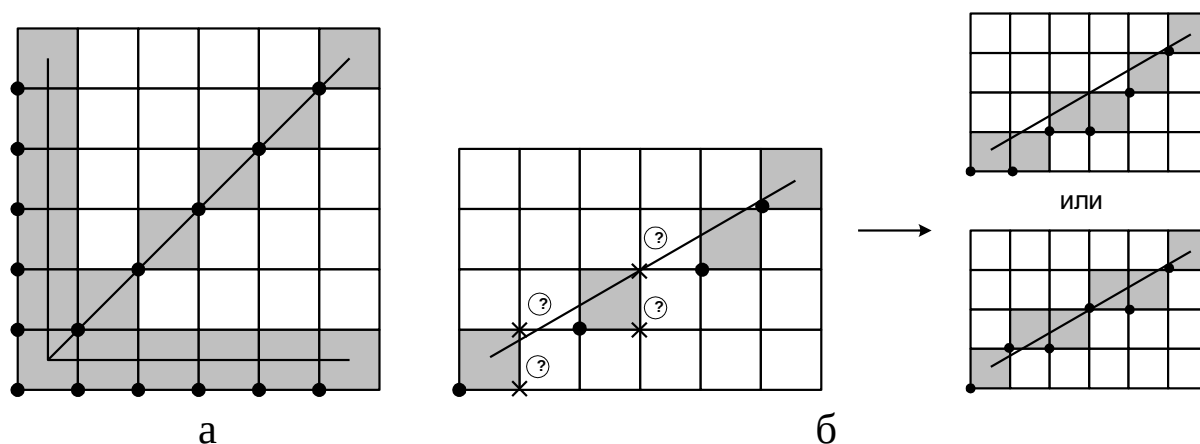


Рис. 1. Разложение отрезка в растр: а – тривиальный случай; б – поиск наилучшего приближения произвольного отрезка пикселями

Требования к алгоритмам растеризации отрезков.

Во-первых, отрезки должны выглядеть прямыми, начинаться и заканчиваться в заданных точках. Во-вторых, яркость вдоль отрезка должна быть постоянной и не зависеть от длины и наклона. В-третьих, алгоритмы должны работать быстро.

Первое требование в силу дискретной природы растрового дисплея выполнено всегда быть не может. Можно лишь добиться того, что визуально (человеческим глазом) отрезок будет восприниматься прямым. Решение этой задачи может достигаться путём увеличения разрешающей способности экрана дисплея и применения методов устранения ступенчатости.

Второму требованию удовлетворяют также только горизонтальные, вертикальные и наклонённые под углом в 45° отрезки. Однако вертикальные и горизонтальные отрезки по сравнению с отрезками,

расположенными под 45° , будут выглядеть ярче, так как расстояние между соседними пикселями у них меньше, чем у наклонных отрезков. Обеспечение постоянной яркости вдоль отрезка требует высвечивания очередного пикселя яркостью, зависящей от расстояния между пикселями, вычисление которого производится с использованием операций извлечения квадратного корня и умножения. Использование этих операций существенно замедляет работу алгоритма, поэтому второе требование остаётся, как правило, невыполненным.

Удовлетворение третьего требования достигается путём сведения к минимуму вычислительных операций, использования операций над целочисленными данными, а также реализацией алгоритмов на аппаратном или микропрограммном уровне.

Цифровой дифференциальный анализатор.

Многие алгоритмы вычерчивания отрезков и кривых используют пошаговый принцип, суть которого состоит в том, что результат вычисления координат высвечиваемого пикселя зависит от результатов, полученных на предыдущем шаге. Алгоритм вычерчивания отрезков по методу цифрового дифференциального анализатора, кроме этого, основан также на достаточно общем принципе, известном в математике: изучение какого-либо явления на основе дифференциального уравнения или системы таких уравнений, описывающей это явление.

Поскольку прямая линия на плоскости описывается уравнением вида

$$ax + by + c = 0,$$

где a , b , c – коэффициенты этого уравнения, производная dy/dx является постоянной.

Заменив дифференциалы конечными разностями, получим

$$\frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1},$$

где x_1, y_1 и x_2, y_2 – координаты начальной и конечной точек отрезка.

Ордината очередного пикселя y_{i+1} может быть вычислена по известной ординате предыдущего пикселя y_i следующим образом:

$$y_{i+1} = y_i + \Delta y.$$

Подставляя Δy , получим

$$y_{i+1} = y_i + \frac{y_2 - y_1}{x_2 - x_1} \Delta x.$$

Остаётся определить величину приращения Δx . Как правило, большее из приращений (Δx или Δy) выбирается в качестве единицы раstra, а приращение вдоль другой координатной оси подлежит определению. Если же поступить по-другому (меньшее из приращений взять равным единице), то отрезок на экране может получиться «дырявым», т. е. состоящим из отдельных точек, не расположенных вплотную друг к другу.

Алгоритм разложения отрезка в растр по методу ЦДА может быть записан следующим образом:

1. Ввод исходных данных: x_1, y_1, x_2, y_2 .
2. Проверка вырожденности отрезка. Если отрезок вырожден, то вывод точки и переход к п.7.
3. Вычисление $L:=|x_2-x_1|$, если $|x_2-x_1|\geq|y_2-y_1|$ или $L:=|y_2-y_1|$, если $|y_2-y_1|>|x_2-x_1|$.
4. Вычисление $dx:=(x_2-x_1)/L$, $dy:=(y_2-y_1)/L$.
5. Присвоение координатам текущей точки начальных значений: $x:=x_1+0.5*\text{sign}(dx)$, $y:=y_1+0.5*\text{sign}(dy)$, где sign – функция, возвращающая -1, 0, 1 для отрицательного, нулевого и положительного аргумента соответственно, использование этой функции делает алгоритм пригодным для всех квадрантов.
6. Цикл от $i:=1$ до $i=L+1$ с шагом 1:
 - вывод точки с текущими координатами ($\text{floor}(x), \text{floor}(y)$), где floor – функция округления до ближайшего меньшего целого;
 - вычисление координат следующей точки: $x:=x+dx$, $y:=y+dy$.
7. Конец.

Вещественный алгоритм Брезенхема

Работа алгоритма Брезенхема основывается на использовании понятия ошибка. Ошибкой здесь называется расстояние между действительным положением отрезка и ближайшим пикселем сетки раstra, который аппроксимирует отрезок на очередном шаге.

На каждом шаге вычисляется величина ошибки и в зависимости от полученного значения выбирается пиксель, ближе расположенный к идеальному отрезку. Если отрезок (рис.2) начинается в пикселе (0,0) и пересекает прямую $x=1$ между пикселями (1,0) и (1,1), то выбор одного из этих пикселей для аппроксимации отрезка зависит от величины тан-

генса угла наклона отрезка $\Delta y/\Delta x$. Поскольку при реализации алгоритма на ЭВМ удобнее анализировать не само значение ошибки, а её знак, то истинное значение ошибки смещается на $-0,5$.

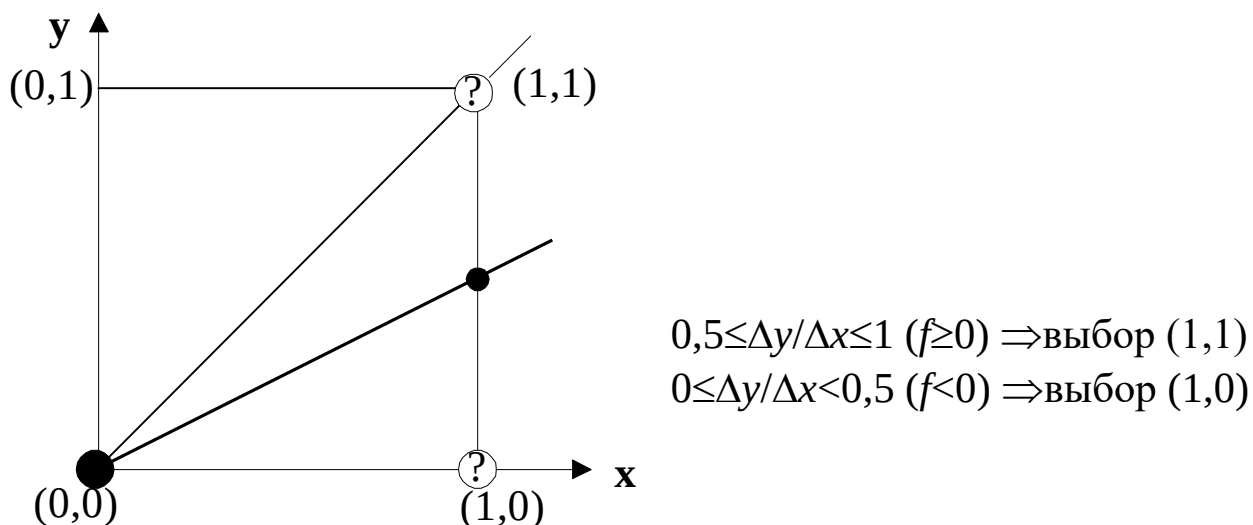


Рис. 2. Основная идея алгоритма Брезенхема

Пусть f_i – значение ошибки на i -м шаге, Y_{ui} – ордината идеального отрезка, Y_i – ордината пикселя, выбранного для аппроксимации отрезка на том же i -м шаге, $\Delta y/\Delta x$ – тангенс угла наклона отрезка. Поскольку на первом шаге высвечивается пиксель с начальными координатами, то для него $f=0$, поэтому задаваемое предварительно значение $f=\Delta y/\Delta x-0,5$ является фактически ошибкой для следующего шага. Ошибка на очередном шаге вычисляется как

$$f_{i+1} = Y_{ui+1} - Y_{i+1} = Y_{ui} + \Delta y/\Delta x - Y_{i+1} = f_i + \Delta y/\Delta x \text{ (если } Y_{i+1} = Y_i).$$

(Поскольку в алгоритме и программе не надо сохранять значения ошибок для всех шагов, то последнее выражение можно записать как $f = f + \Delta y/\Delta x$).

В зависимости от полученного значения ошибки выбирается пиксель с той же ординатой ($Y_{i+1} = Y_i$ при $f < 0$) или пиксель с ординатой, на единицу большей, чем у предыдущего пикселя ($Y_{i+1} = Y_i + 1$ при $f \geq 0$).

Поскольку предварительное значение ошибки вычисляется заранее, то есть $f + \Delta y/\Delta x$ вычислено на предыдущем шаге, то во втором случае останется только вычесть единицу из значения ошибки:

$$f = f - 1.$$

Алгоритм Брезенхема, работающий с действительными величинами, можно записать в следующем виде.

1. Ввод исходных данных: x_1, y_1, x_2, y_2 .

2. Проверка вырожденности отрезка. Если отрезок вырожденный, то выводится точка и осуществляется переход к п.10.
3. Вычисление приращений: $dx:=x_2-x_1$ и $dy:=y_2-y_1$.
4. Вычисление шага изменения каждой координаты пикселя: $sx:=\text{sign}(dx)$, $sy:=\text{sign}(dy)$.
5. Вычисление модулей приращения координат: $dx:=|dx|$, $dy:=|dy|$.
6. Обмен значений dx и dy в зависимости от углового коэффициента наклона отрезка:
 - 1) если $dy > dx$, то выполнить $temp:=dx$, $dx:=dy$, $dy:=temp$, $flag:=1$;
 - 2) иначе $flag:=0$ ($flag$ – флаг, определяющий факт обмена местами координат).
7. Инициализация начального значения ошибки: $f:=dy/dx-0.5$.
8. Инициализация начальных значений координат текущего пикселя: $x:=x_1$, $y:=y_1$.
9. Цикл от 0 до dx делать:
 - 1) вывод точки с координатами (x,y) ;
 - 2) вычисление координат и корректировка ошибки для следующего пикселя:
 - если $f \geq 0$, то если $flag=1$, то $x:=x+sx$, иначе $y:=y+sy$;
 - корректировка ошибки $f:=f-1$;
 - 3) переход к следующему пикселю: если $flag=1$, то $y:=y+sy$ иначе $x:=x+sx$;
 - 4) вычисление ошибки $f:=f+dy/dx$.
10. Конец.

Целочисленный алгоритм Брезенхема

Алгоритм Брезенхема в том виде, как он представлен выше, требует использования арифметики с плавающей точкой и деления (для вычисления углового коэффициента и оценки ошибки). Быстродействие алгоритма можно увеличить, если использовать только целочисленную арифметику и исключить деление. Для этого выражение для вычисления ошибки запишем в виде $f=\Delta y/\Delta x-0,5$ и, умножив обе части этого равенства на $2\Delta x$, получим $2\Delta x f=2\Delta y-\Delta x$. Обозначив $f_u=2\Delta x f$, окончательно получим $f_u=2\Delta y-\Delta x$. В соответствии с этим выражением начальное значение ошибки в п.7 алгоритма будет вычисляться в виде $f:=2*dy-dx$.

Тогда в п.9 алгоритма корректировка ошибки будет производиться по формуле: $f:=f-2*dx$, а вычисление ошибки — $f:=f+2*dy$.

Оптимизация алгоритма Брезенхема – прорисовка линии отрезками переменной длины.

Идея алгоритма Брезенхема – это движение вдоль основной оси по одному пикселю и поддержание текущего отклонения от идеальной прямой в заданных пределах. Если отклонение превышает норму, то делается шаг по неосновной оси, чтобы уменьшить отклонение. Главный момент здесь – это то, что вычисление и проверка производятся на каждом шаге.

Для контроля размещения точек необходимо периодически принимать решения относительно их расстановки, и делать это желательно как можно быстрее. Можно принимать решения для каждой точки как в стандартном алгоритме Брезенхема, но большинство проверок в этом случае излишни.

Идея оптимизации (алгоритм с переменной длиной отрезков) – это движение с каждой итерацией по неосновной оси и проверка текущего отклонения горизонтальной координаты.

Посмотрим на задачу так: когда программа проводит линию в 35 пикселей по X и 10 пикселей по Y, то уже есть много информации, часть которой игнорируется стандартным алгоритмом Брезенхема. Например, поскольку наклон линии находится в пределах $1/3-1/4$, то уже можно определённо сказать, что составляющие линию ряды точек состоять из 3 или 4 пикселей (рис. 3). Вычислений становится на 70% меньше.

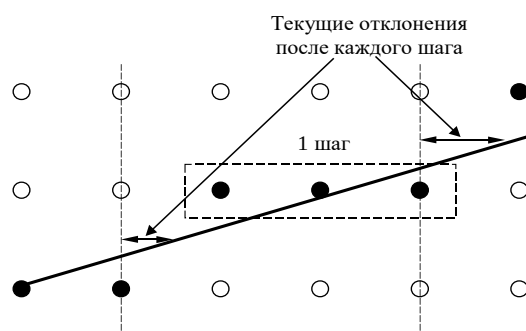


Рис. 3 – Алгоритм с переменной длиной отрезков

Задание

1. Разработать программу, реализующую алгоритмы растеризации отрезков с их последующей прорисовкой. Вывести рисунок в соответствии с вариантом. Для вывода отрезков использовать:

- алгоритм ЦДА;
- алгоритм Брезенхема;
- целочисленный алгоритм Брезенхема;
- встроенные средства языка программирования.

2. Проиллюстрировать разницу между результатами или её отсутствием.

3. Ответить на контрольные вопросы.

4. Оформить отчёт.

Содержание отчёта

1. Титульный лист.
2. Задание.
3. Схемы алгоритмов.
4. Программа на алгоритмическом языке программирования.
5. Пример работы программы.
6. Ответы на контрольные вопросы.

Порядок выполнения работы

1. Ознакомление с целью работы, заданием, содержанием отчёта, порядком выполнения работы, индивидуальным вариантом, примером выполнения работы, примером оформления отчёта.

2. Ознакомление со средствами системы программирования для чтения, обработки и записи графики на примере и с помощью технической документации.

3. Ознакомление с теоретическими материалами по теме.

4. Разработка алгоритма в соответствии с вариантом задания. Алгоритм должен предусматривать:

- 1) чтение файла изображения;
- 2) вывод отрезков в соответствии с заданием по:
 - алгоритму ЦДА;
 - алгоритму Брезенхема;

- целочисленному алгоритму Брезенхема;
- встроенными средствами языка программирования;
- 3) запись полученного изображения в файл;
- 4) иллюстрацию разницы между результатами или её отсутствие.
- 5. Разработка и отладка программы для решения задачи.
- 6. Ответы на контрольные вопросы.
- 7. Оформление отчёта.

Индивидуальные задания

Задание включает в себя вывод фигуры из нескольких отрезков под разными углами.

1. Вывести прямоугольник по заданным левому верхнему и правому нижнему углам.
2. Вывести пентаграмму по заданному радиусу описанной окружности и центральному углу одной из вершин.
3. Вывести звезду Давида по заданному радиусу описанной окружности и центральному углу одной из вершин.
4. Вывести ромб по заданным координатам одной диагонали и длине другой диагонали.
5. Вывести пифагоров квадрат по заданному радиусу описанной окружности и координатам начала центрального отрезка.
6. Вывести полилинию по заданным координатам четырёх вершин.
7. Вывести параллелограмм по заданным координатам большой диагонали, высоте и углу.
8. Вывести трапецию по заданным координатам трёх вершин и длине средней линии.
9. Вывести правильный семиугольник по заданному радиусу описанной окружности.
10. Вывести связный звёздчатый восьмиугольник по заданному радиусу описанной окружности.
11. По заданным вершинам вывести треугольник и его высоты.
12. По заданным вершинам вывести треугольник и его биссектрисы.
13. По заданным вершинам вывести треугольник и его медианы.
14. По заданным координатам отрезка, углу и количеству звеньев вывести зигзаг.

15. По заданным вершинам вывести треугольник и его срединный треугольник.
16. По двум длинам сторон нарисовать спидрон.
17. По заданным координатам вершин бедра вывести золотой треугольник.
18. По заданным вершинам вывести треугольник и его ортотреугольник.
19. По заданным вершинам вывести треугольник и его подёрный треугольник.
20. По заданным вершинам вывести треугольник и треугольник, построенный по основаниям его биссектрис.

Контрольные вопросы

1. Что такое пиксель?
2. Что такое растровое изображение?
3. Для чего нужны алгоритмы растеризации отрезков?
4. Какие ещё алгоритмы растеризации отрезков существуют?

Пример выполнения работы

Взять n точек, равномерно распределённых по окружности радиуса R и соединить отрезком каждую k -ю с $2*k$ -й по модулю n .

Для растеризации отрезков, заданных координатами концов, реализуем алгоритмы ЦДА, Брезенхема и целочисленный Брезенхема в виде отдельных функций, куда будем передавать координаты вершин и холст.

На форме (рис. 4) предусматриваем 2 объекта `Tbutton`: для нового изображения и сохранения результата, диалог `TsavePictureDialog` для цивилизованного сохранения файла, объект `TImage` для хранения и отображения картинок, 2 объекта `Tedit` для ввода радиуса и количества точек, 2 компонента `Tlabel` для подписи `Tedit`.

При нажатии на кнопку «Создать» из компонентов `Tedit` считываются радиус и количество точек, для битмапа `Image1` выполняется очистка, задаётся формат пикселей `pf24bit`, задаются ширина и высота величиной $2 \text{ радиуса} + 10$, чтобы поместилась окружность, обрамлённая пустотой, выполняется заполнение белым фоном.

Далее вычисляются координаты точек на окружности, и разным цветом прорисовываются отрезки алгоритмами ЦДА, Брезенхема и целочисленным Брезенхема и встроенным методом Tcanvas.line().

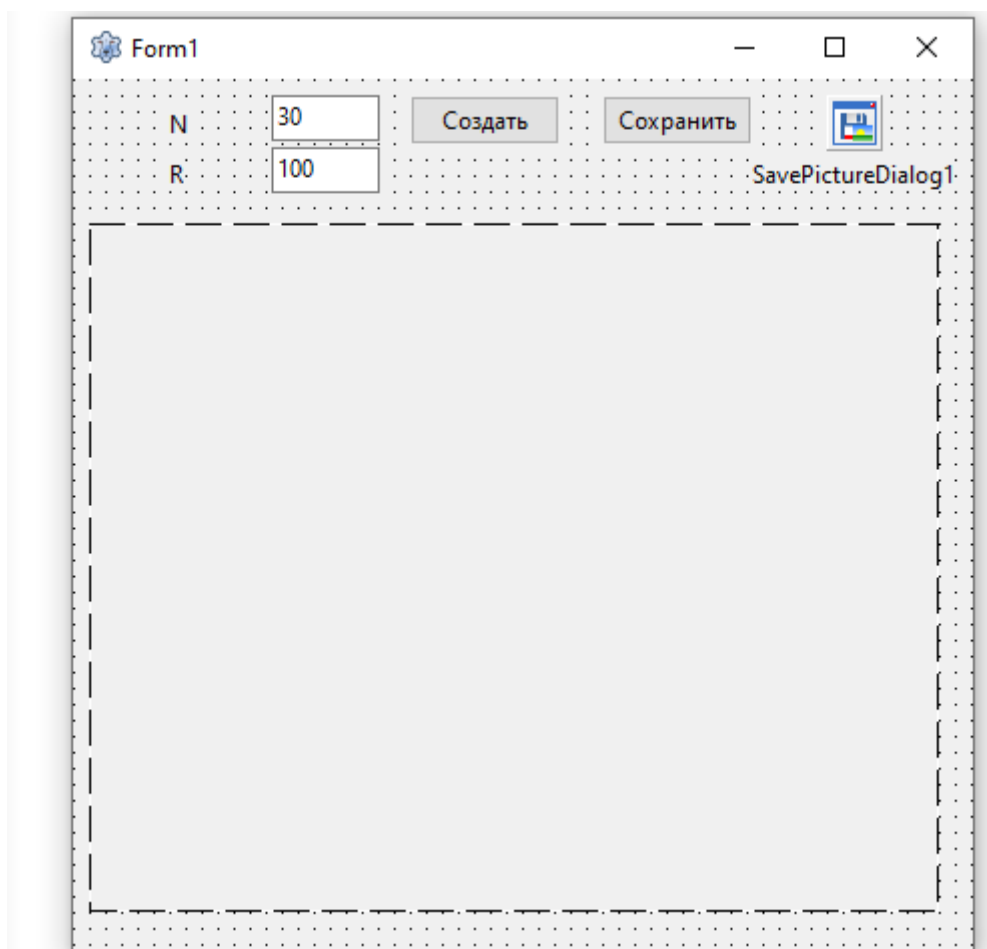


Рис. 4. Форма программы прорисовки отрезков

Получаем модуль приблизительно следующего содержания:

```
unit Unit1;
```

```
{ $mode objfpc } { $H+ }
```

```
interface
```

```
uses
```

```
Classes, SysUtils, FileUtil, Forms, Controls, Graphics, Dialogs,  
StdCtrls, GraphType, IntfGraphics, LCLType, LCLProc, LCLIntf, ExtCtrls,  
ExtDlgs, fpImage, crt, math;
```

type

```
{ TForm1 }
```

```
TForm1 = class(TForm)
```

```
  Button1: TButton;
```

```
  Button3: TButton;
```

```
  Edit6: TEdit;
```

```
  Edit7: TEdit;
```

```
  Label1: TLabel;
```

```
  Label3: TLabel;
```

```
  SavePictureDialog1: TSavePictureDialog;
```

```
  Image1: TImage;
```

```
  procedure Button1Click(Sender: TObject);
```

```
  procedure Button3Click(Sender: TObject);
```

```
private
```

```
  { private declarations }
```

```
public
```

```
  { public declarations }
```

```
end;
```

var

```
  Form1: TForm1;
```

implementation

```
{ $R *.lfm }
```

```
{ TForm1 }
```

```
procedure cda(x1,y1,x2,y2:real; canvas:TCanvas);
```

```
var i: integer;
```

```
  l,dx,dy,x,y:real;
```

```
begin
```

```
  if (x1=x2) and (y1=y2) then
```

```
    begin
```

```
      canvas.Pixels[floor(x1),floor(y1)]:=canvas.Pen.Color;
```

```
    exit
```

```

        end;
    if abs(x2-x1)>=abs(y2-y1) then l:=abs(x2-x1) else l:=abs(y2-y1);
    dx:=(x2-x1)/l;
    dy:=(y2-y1)/l;
    x:=x1+0.5*sign(dx);
    y:=y1+0.5*sign(dy);
    for i:=1 to round(l)+1 do
        begin
            canvas.Pixels[floor(x),floor(y)]:=canvas.Pen.Color;
            x:=x+dx;
            y:=y+dy;
        end;
    end;
end;

```

```

procedure brezf(x1,y1,x2,y2:real; canvas:TCanvas);
var i,sx,sy,f: integer;
    e,dx,dy,x,y,t:real;
begin
    if (x1=x2) and (y1=y2) then
        begin
            canvas.Pixels[floor(x1),floor(y1)]:=canvas.Pen.Color;
            exit
        end;
    sx:=sign(x2-x1);
    sy:=sign(y2-y1);
    dx:=abs(x2-x1);
    dy:=abs(y2-y1);
    x:=x1;
    y:=y1;
    f:=0;
    if dy>dx then
        begin
            t:=dx;
            dx:=dy;
            dy:=t;
            f:=1
        end;
    e:=dy/dx-0.5;

```

```

for i:=0 to round(dx) do
  begin
    canvas.Pixels[floor(x),floor(y)]:=canvas.Pen.Color;
    if e>=0 then
      begin
        if f=1 then x:=x+sx else y:=y+sy;
        e:=e-1;
      end;
    if f=1 then y:=y+sy else x:=x+sx;
    e:=e+dy/dx
  end;
end;

```

```

procedure brezi(x1,y1,x2,y2:integer; canvas:TCanvas);
var t,i,sx,sy,f,dx,dy,x,y,e: integer;
begin
  if (x1=x2) and (y1=y2) then
    begin
      canvas.Pixels[x1,y1]:=canvas.Pen.Color;
      exit
    end;
  sx:=sign(x2-x1);
  sy:=sign(y2-y1);
  dx:=abs(x2-x1);
  dy:=abs(y2-y1);
  x:=x1;
  y:=y1;
  f:=0;
  if dy>dx then
    begin
      t:=dx;
      dx:=dy;
      dy:=t;
      f:=1
    end;
  e:=2*dy-dx;
  for i:=0 to dx do
    begin

```

```

        canvas.Pixels[x,y]:=canvas.Pen.Color;
    if e>=0 then
        begin
            if f=1 then x:=x+sx else y:=y+sy;
            e:=e-2*dx;
        end;
        if f=1 then y:=y+sy else x:=x+sx;
        e:=e+2*dy
    end;
end;

procedure TForm1.Button1Click(Sender: TObject); //Кнопка Создать
var i,n,r:integer;
    alph,x1,x2,y1,y2:real;
begin // С битмапом расположенного на форме TImage
    with Image1.Picture.Bitmap do
        begin
            Clear; //а вдруг там мусор
            PixelFormat:=pf24bit; //формат пикселей
            r:=StrToInt(Edit7.Text);
            Width:=r*2 + 10; //Ширина
            Height:=r*2 + 10; //Высота
            n:=StrToInt(Edit6.Text);
            Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder); //Белый фон
            alph:=2*3.1415926/n;
            for i:=1 to n do
                begin //рисуем отрезками
                    x1:=height div 2 + r*cos(i*alph);
                    y1:=width div 2 + r*sin(i*alph);
                    x2:=height div 2 + r*cos((i*2 mod n)*alph);
                    y2:=width div 2 + r*sin((i*2 mod n)*alph);
                    canvas.Pen.Color:=RGB(127,127,255); //цвет точек
                    cda(x1,y1,x2,y2,canvas);
                    canvas.Pen.Color:=RGB(255,127,127); //цвет точек
                    brezf(x1,y1,x2,y2,canvas);
                    canvas.Pen.Color:=RGB(127,255,127); //цвет точек
                    brezi(round(x1),round(y1),round(x2),round(y2),canvas);
                    canvas.Pen.Color:=RGB(127,127,127); //цвет точек
                end;
            end;
        end;
    end;
end;

```

```

        canvas.line(round(x1),round(y1),round(x2),round(y2));
    end;
end;
end;

procedure TForm1.Button3Click(Sender: TObject);
begin
    if SavePictureDialog1.Execute then
        Image1.Picture.SaveToFile(SavePictureDialog1.FileName);
    end;

end.

```

Смотрим на результат (рис. 5). Можем сохранить.

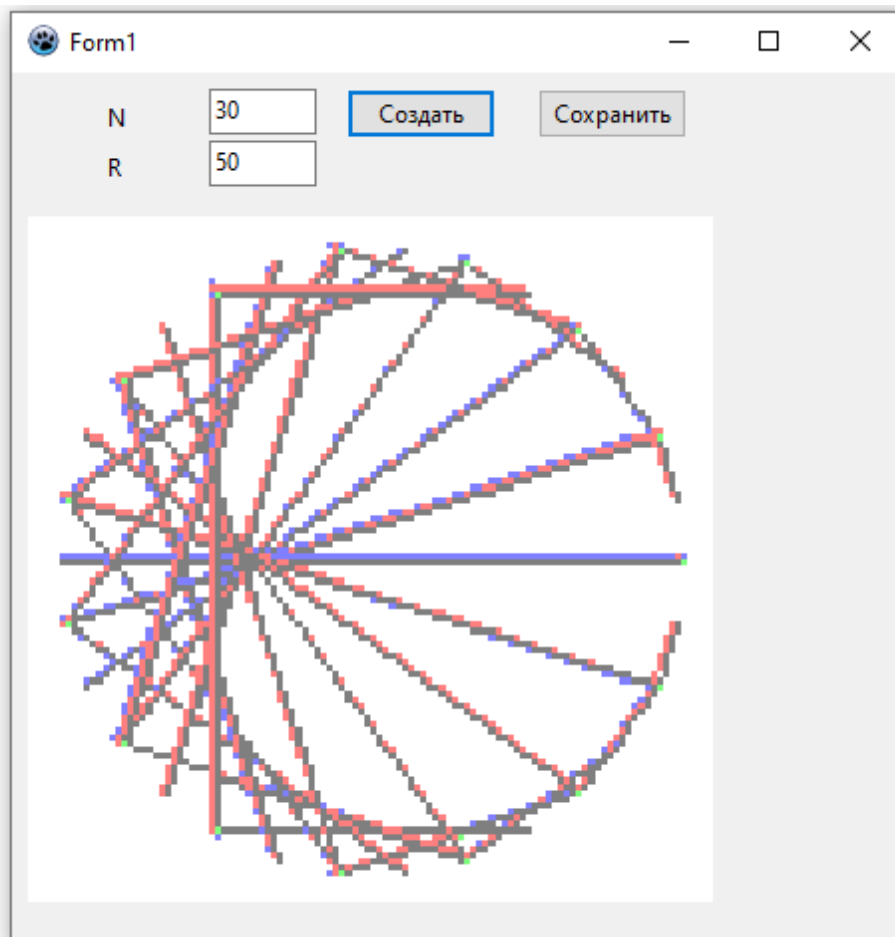


Рис. 5. Результат работы программы прорисовки отрезков

Лабораторная работа № 4.

Растеризация окружностей

Цель работы – изучение алгоритмов растеризации окружностей, создание программы для визуализации работы алгоритмов.

Краткие теоретические сведения

Существует несколько простых, но неэффективных способов преобразования окружностей в растровую форму.

Рассмотрим окружность с центром в начале координат, для которой $x^2 + y^2 = R^2$.

Решая это уравнение относительно y , получим $y = \pm\sqrt{R^2 - x^2}$.

Чтобы изобразить четвертую часть окружности, можно увеличивать x с единичным шагом от 0 до R и на каждом шаге вычислять y и делать округление полученного значения (остальные четверти изображаются симметрично). Этот метод работоспособен, но не эффективен, поскольку в него входят операции умножения и извлечения квадратного корня. Более того, при значениях x , близких к R , в окружности появляются заметные незаполненные промежутки, так как при этих значениях x тангенс угла наклона касательной к окружности стремится к бесконечности (рис. 1).

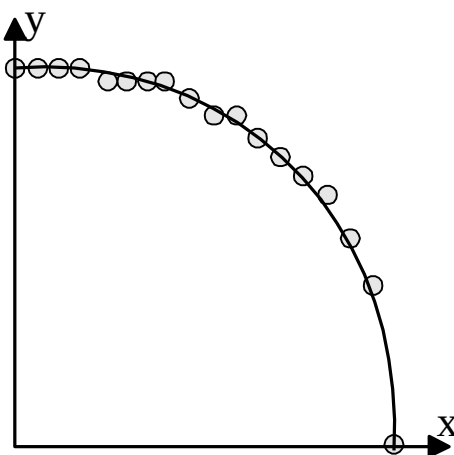


Рис.1. Четверть окружности, построенная с использованием единичных шагов по x и вычисленная округлённых значений y

Ещё один простой подход к построению окружности также заключается в использовании её параметрического уравнения:

$$\begin{cases} x = x_0 + R \cos \tau, \\ y = y_0 + R \sin \tau, \end{cases}$$

где x_0, y_0 – координаты центра;

R - радиус окружности;

τ - параметр.

При этом, учитывая тот факт, что шаг изменения аргумента должен составлять величину $\tau=1/R$, потребуется рассчитать для каждого значения параметра τ значения координат соответствующих точек окружности и соединить их затем отрезками прямых. Такой путь также требует довольно большого количества вычислений.

Другим, более простым и эффективным путём является использование алгоритма Брезенхема для генерации окружности. Чтобы построить полную окружность, достаточно сгенерировать её одну восьмую часть. Остальные части получаются затем симметричным отражением относительно определённой прямой. Так, отражая одну восьмую часть, построенную в первом октанте для углов в диапазоне $0-45^\circ$, относительно прямой с уравнением $y=x$, получим одну четвертую часть, лежащую в первом квадранте. Отразив эту четверть относительно прямой $x=0$, получим одну вторую часть, лежащую выше оси абсцисс, наконец, отразив эту полуокружность относительно прямой $y=0$, получим полную окружность (рис. 2).

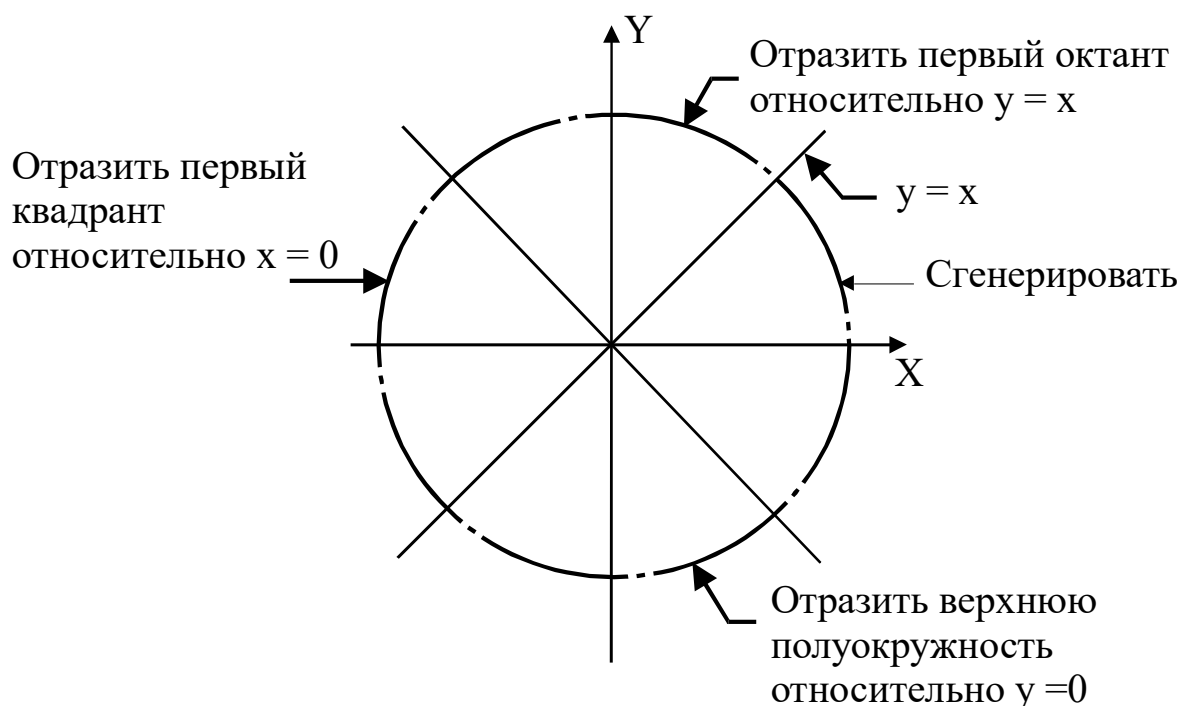


Рис. 2. Генерация полной окружности из дуги в первом октанте

Рассмотрим построение части окружности, лежащей в первом квадранте. Предположим, что центр окружности лежит в начале координат, её радиус равен R , ось абсцисс направлена вправо, ось ординат — вверх. Пусть алгоритм начинает работу в точке с координатами $x=0$, $y=R$, тогда он заканчивает работу в точке $x=R$, $y=0$. При таком направлении построения окружности y как функция аргумента x будет являться монотонно убывающей ($y = \sqrt{R^2 - x^2}$).

Для любого пикселя при выбранном направлении генерации окружности существует только три варианта выбора следующего пикселя, наилучшим образом аппроксимирующего окружность:

- горизонтальный пиксель вправо;
- диагональный вниз и вправо;
- вертикальный вниз.

В качестве критерия выбора очередного пикселя используется минимум модуля разности квадратов расстояний от центра окружности до пикселя и до идеальной окружности. Эти величины имеют следующие значения:

$$m_h = |(x_i + 1)^2 + y_i^2 - R^2|,$$

$$m_d = |(x_i + 1)^2 + (y_i - 1)^2 - R^2|,$$

$$m_v = |x_i^2 + (y_i - 1)^2 - R^2|,$$

где m_h , m_d , m_v – искомые величины расстояний для горизонтального, диагонального и вертикального направлений соответственно; x_i , y_i – координаты текущего пикселя.

Вычисления можно упростить, если заметить, что в окрестности текущей точки возможны пять вариантов прохождения окружности (рис. 3):

- 1) между горизонтальным $(x_i + 1, y_i)$ и диагональным $(x_i + 1, y_i - 1)$ пикселями;
- 2) между горизонтальным $(x_i + 1, y_i)$ и вторым диагональным $(x_i + 1, y_i + 1)$ пикселями;
- 3) между вертикальным $(x_i, y_i - 1)$ и диагональным $(x_i + 1, y_i - 1)$;
- 4) между вертикальным $(x_i, y_i - 1)$ и третьим диагональным $(x_i - 1, y_i - 1)$ пикселями;
- 5) прохождение точно через диагональный $(x_i + 1, y_i - 1)$ пиксель.

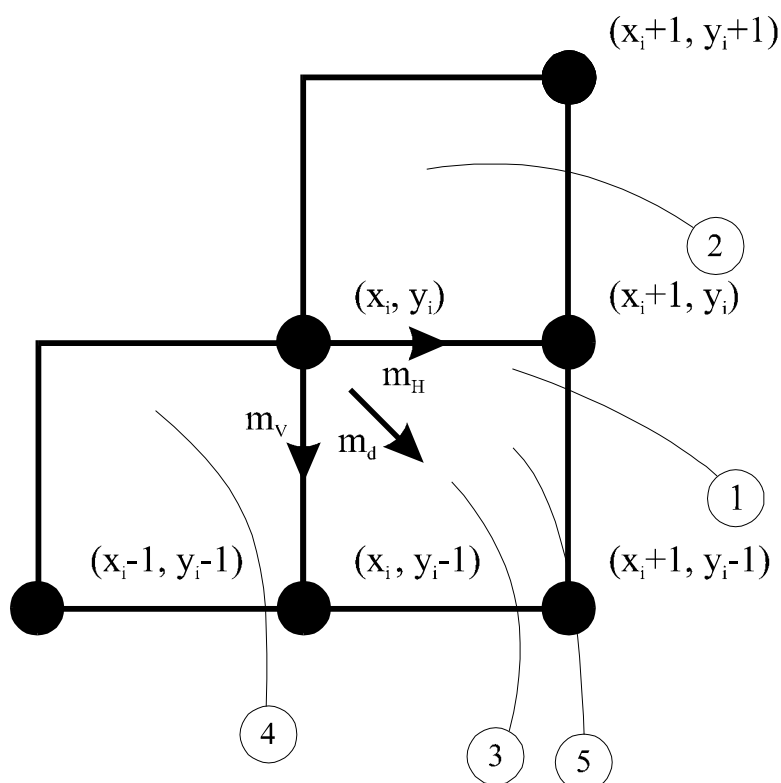


Рис. 3. Пересечение окружности и сетки раstra

По аналогии с ошибкой в алгоритме построения отрезков выберем и здесь некоторую величину, анализ знака которой позволит выбрать нужный пиксель.

Такой величиной Δ является разность квадратов расстояний от центра окружности до диагонального (x_i+1, y_i-1) пикселя и до идеальной окружности:

$$\Delta_i = (x_i+1)^2 + (y_i-1)^2 - R^2.$$

Отрицательное значение $\Delta_i < 0$ может быть только в том случае, когда диагональный пиксель (x_i+1, y_i-1) лежит внутри окружности, т. е. это случаи 1 и 2, и следует выбирать горизонтальный (x_i+1, y_i) или диагональный (x_i+1, y_i-1) пиксель.

Рассмотрим случай 1 и вычислим разность Δ_1 расстояний, т. е.

$$\Delta_1 = m_h - m_d = |(x_i+1)^2 + y_i^2 - R^2| - |(x_i+1)^2 + (y_i-1)^2 - R^2|.$$

При $m_h < m_d$ $\Delta_1 < 0$, и в этом случае расстояние до горизонтального пикселя меньше, чем до диагонального, поэтому выбирается горизонтальный пиксель. При $m_h \geq m_d$ $\Delta_1 \geq 0$, и в этом случае расстояние до диагонального пикселя меньше, чем до горизонтального, поэтому выбирается диагональный пиксель. При $m_h = m_d$ $\Delta_1 = 0$ и можно выбирать либо горизонтальный, либо диагональный пиксель. Для определённости выберем горизонтальный пиксель.

Для сокращения вычислений упростим выражение. При этом следует учесть, что для случая 1 $m_h > 0$, а $m_d < 0$, так как горизонтальный пиксель лежит вне окружности, а диагональный внутри окружности.

Таким образом, можно записать:

$$\Delta_1 = (x_i + 1)^2 + y_i^2 - R^2 + (x_i + 1)^2 + (y_i - 1)^2 - R^2$$

Дополнение до полного квадрата члена y_i^2 с помощью добавления и вычитания $-2y_i + 1$ даёт

$$\Delta_1 = 2[(x_i + 1)^2 + (y_i - 1)^2 - R^2] + 2y_i - 1.$$

В квадратных скобках стоит по определению Δ_i , и его подстановка существенно упрощает выражение

$$\Delta_1 = 2(\Delta_i + y_i) - 1.$$

Во втором случае выбирается горизонтальный пиксель, второй диагональный пиксель $(x_i + 1, y_i + 1)$ выбран быть не может в силу монотонного убывания функции. Для этого случая $m_h < 0$ и $m_d < 0$, для первого квадранта $1 < y_i < R$, поэтому $\Delta_1 = 1 - 2y_i < 0$. Таким образом, и в случае 2, если $\Delta_1 < 0$, то выбирается горизонтальный пиксель.

Рассмотрим случай $\Delta_i > 0$. Это будет справедливо только в том случае, если диагональный пиксель лежит вне окружности, т. е. это случаи 3 и 4, и надо выбирать либо диагональный $(x_i + 1, y_i - 1)$, либо вертикальный $(x_i, y_i - 1)$ пиксель.

Рассмотрим случай 3 и вычислим разность расстояний:

$$\Delta_2 = m_d - m_v = |(x_i + 1)^2 + (y_i - 1)^2 - R^2| - |x_i^2 + (y_i - 1)^2 - R^2|$$

При $\Delta_2 < 0$ расстояние до диагонального пикселя меньше, чем до вертикального, поэтому выбирается диагональный пиксель. При $m_d > m_v$ $\Delta_2 > 0$ – расстояние до вертикального пикселя меньше, чем до диагонального, поэтому выбирается вертикальный пиксель. При $m_d = m_v$ $\Delta_2 = 0$, и можно выбирать либо диагональный, либо вертикальный пиксель. Для определённости выберем диагональный пиксель.

Для упрощения вычисления выражения учтём, что для случая 3 $m_d > 0$, а $m_v < 0$, так как диагональный пиксель лежит вне окружности, а вертикальный внутри окружности:

$$\Delta_2 = (x_i + 1)^2 + (y_i - 1)^2 - R^2 + x_i^2 + (y_i - 1)^2 - R^2.$$

Дополнение до полного квадрата члена x_i^2 с помощью добавления и вычитания $2x_i + 1$ даёт

$$\Delta_2 = 2[(x_i + 1)^2 + (y_i - 1)^2 - R^2] - 2x_i - 1.$$

В квадратных скобках стоит по определению Δ_i , и его подстановка существенно упрощает выражение:

$$\Delta_2 = 2(\Delta_i - x_i) - 1.$$

В четвёртом случае выбирается вертикальный пиксель, третий диагональный пиксель (x_i-1, y_i-1) выбран быть не может в силу монотонного убывания функции. Для этого случая $m_d > 0$ и $m_v > 0$, для первого квадранта $0 < x_i < R$, поэтому $\Delta_2 = 2x_i + 1 > 0$. Таким образом, и в случае 4, если $\Delta_2 > 0$, то выбирается вертикальный пиксель.

Остаётся проверить случай 5, для которого $\Delta_i = 0$, то есть диагональный пиксель лежит точно на окружности.

Вычислим значение $\Delta_1 = m_h - m_d$, учитывая, что $m_h > 0$, а $m_d = 0$:

$$\Delta_1 = (x_i + 1)^2 + y_i^2 - R^2 > 0,$$

следовательно, выбирается диагональный пиксель.

Вычислим значение $\Delta_2 = m_d - m_v$, учитывая, что $m_d = 0$, $m_v < 0$:

$$\Delta_2 = x_i^2 + (y_i - 1)^2 - R^2 < 0,$$

следовательно, выбирается диагональный пиксель.

Таким образом, при $\Delta_i = 0$ выполняются те же условия для Δ_1 и Δ_2 , как и в ранее рассмотренных случаях.

Подведём итог полученных результатов:

$\Delta_i < 0$:

$\Delta_1 \leq 0$ выбираем пиксель $(x_i + 1, y_i)$,

$\Delta_1 > 0$ выбираем пиксель $(x_i + 1, y_i - 1)$,

$\Delta_i > 0$:

$\Delta_2 \leq 0$ выбираем пиксель $(x_i + 1, y_i - 1)$,

$\Delta_2 > 0$ выбираем пиксель $(x_i, y_i - 1)$,

$\Delta_i = 0$: выбираем пиксель $(x_i + 1, y_i - 1)$.

Теперь остаётся записать формулы для вычисления координат очередного пикселя и критерия для каждого из трёх возможных направлений движения. Обозначим новое положение пикселя как $(i+1)$ в индексе (x, y) координат и критерия Δ .

При переходе к горизонтальному пикселю:

$$x_{i+1} = x_i + 1,$$

$$y_{i+1} = y_i,$$

$$\begin{aligned} \Delta_{i+1} &= (x_{i+1} + 1)^2 + (y_{i+1} - 1)^2 - R^2 = (x_{i+1})^2 + 2x_{i+1} + 1 + (y_i - 1)^2 - R^2 = \\ &= (x_i + 1)^2 + (y_i - 1)^2 - R^2 + 2x_{i+1} + 1 = \Delta_i + 2x_{i+1} + 1. \end{aligned}$$

При переходе к диагональному пикселю:

$$x_{i+1} = x_i + 1,$$

$$y_{i+1} = y_i - 1,$$

$$\Delta_{i+1} = \Delta_i + 2x_{i+1} - 2y_{i+1} + 2.$$

При переходе к вертикальному пикселю:

$$x_{i+1}=x_i,$$

$$y_{i+1}=y_i-1,$$

$$\Delta_{i+1} = \Delta_i - 2y_{i+1} + 1.$$

Начальное значение для $x=0$, $y=R$ будет равно

$$\Delta_i = (0+1)^2 + (R-1)^2 - R^2 = 2(1-R).$$

Алгоритм Брезенхема для генерации окружности в первом квадранте может быть записан в следующем виде:

1. Ввод исходных данных R (радиус окружности) и при необходимости x_0 , y_0 (координаты центра окружности).
2. Задание начальных значений текущих координат пикселя $x:=0$, $y:=R$, параметров $\text{delta}:=2-2*R$, $d1:=0$, $d2:=0$.
3. Проверка окончания работы: если $x>y$, то переход к п.10.
4. Вывод текущего пикселя (x,y) Здесь же можно вывести остальные семь симметричных.
5. Вычисление $d1:=2*(\text{delta} + y)-1$, $d2:=2*(\text{delta} - x)-1$.
6. Анализ значений параметров:
 - если $\text{delta}<0$ и $d1\leq 0$, то переход к п.7;
 - если $\text{delta}>0$ и $d2>0$, то переход к п.8;
 - в остальных случаях переход к п.9.
7. Вычисление параметра delta и горизонтальный шаг:
 - $x:=x+1$;
 - $\text{delta}:=\text{delta}+2*x+1$;
 - переход к п. 3.
8. Вычисление параметра delta и вертикальный шаг:
 - $y:=y-1$;
 - $\text{delta}:=\text{delta}-2*y+1$;
 - переход к п. 3.
9. Вычисление параметра delta и диагональный шаг:
 - $x:=x+1$;
 - $y:=y-1$;
 - $\text{delta}:=\text{delta} + 2*(x-y)$;
 - переход к п. 3.
10. Конец.

Учёт ненулевых координат центра окружности означает фактически выполнение операции переноса, поэтому координаты точек окружности (x,y) будут вычисляться по формуле: (x_0+x, y_0+y) .

Задание

1. Написать программу (на языке высокого уровня), реализующую алгоритмы растеризации окружности с их последующей прорисовкой. Координаты центра и радиуса должны задаваться пользователем. Реализовать:

- построение окружности по уравнению окружности;
- построение окружности по параметрическому уравнению;
- алгоритм Брезенхема для построения окружности;
- встроенные средства языка программирования.

2. Проиллюстрировать разницу между результатами или её отсутствие.

3. Ответить на контрольные вопросы.

4. Оформить отчёт.

Содержание отчёта

1. Титульный лист.
2. Задание.
3. Схемы алгоритмов.
4. Листинг программы.
5. Пример работы программы.
6. Ответы на контрольные вопросы.

Порядок выполнения работы

1. Ознакомление с целью работы, заданием, содержанием отчёта порядком выполнения работы, индивидуальным вариантом, примером выполнения работы, примером оформления отчёта.

2. Ознакомление со средствами системы программирования для чтения, обработки и записи графики на примере и с помощью технической документации.

3. Ознакомление с теоретическими материалами по теме.

4. Разработка алгоритма в соответствии с вариантом задания. Алгоритм должен предусматривать:

- 1) чтение файла изображения;
- 2) растеризацию окружностей в соответствии с заданием по:
 - уравнению окружности;

- параметрическому уравнению;
- алгоритму Брезенхема;
- встроенные средства языка программирования;
- 3) запись полученного изображения в файл.
- 5. Разработка и отладка программы для решения задачи.
- 6. Ответы на контрольные вопросы.
- 7. Оформление отчёта.

Индивидуальные задания

Варианты задания предусматривают вывод нескольких окружностей или их дуг. Для каждой из окружностей нужно в соответствии с условием вычислить значения исходных данных.

1. Вывести вписанную и описанную окружности квадрата по его длине стороны.
2. Вывести вписанную и описанную окружности по заданной стороне правильного пятиугольника.
3. Вывести вписанную и описанную окружности по заданной стороне правильного шестиугольника.
4. Вывести арбеллос.
5. Вывести салион.
6. Вывести трикветр.
7. Вывести цветок жизни.
8. Вывести треугольник Рёло заданной ширины.
9. Вывести вписанную и описанную окружности по заданной стороне правильного семиугольника.
10. Вывести вписанную и описанную окружности по заданной стороне правильного восьмиугольника.
11. По заданным вершинам вывести треугольник и его вписанную и описанную окружности.
12. По заданным длинам сторон вывести треугольник и его вне-вписанные окружности.
13. По двум заданным радиусам вывести яйцо Мосса.
14. По двум заданным осям вывести стадиум.
15. По заданным сторонам треугольника вывести окружности Мальфатти.
16. По радиусу и длине оси вывести овал.
17. По заданным радиусам кривизны и толщине нарисовать линзу.

18. По заданным радиусам и расстоянию между центрами нарисовать луночку.
19. Вывести трифолий.
20. Вывести Инь-Ян.

Контрольные вопросы

1. Что такое пиксель?
2. Что такое растровое изображение?
3. Для чего нужны алгоритмы растеризации окружностей?
4. Какие ещё алгоритмы растеризации окружностей существуют?

Пример выполнения работы

- 0 По двум заданным длинами осей вывести овал.

Для растеризации овала, заданного длинами осей, реализуем алгоритмы растеризации дуг окружностей по уравнению окружности, параметрическому уравнению и Брезенхема в виде отдельных функций, куда будем передавать координаты центра, радиусы, координаты лучей, определяющих начало и конец дуги и холст.

На форме (рис. 4) предусматриваем 2 объекта Tbutton для нового изображения и сохранения результата, диалог TsavePictureDialog для цивилизованного сохранения файла, объект TImage для хранения и отображения картинок, 2 объекта Tedit для ввода длин осей, 2 компонента TLabel для подписи Tedit.

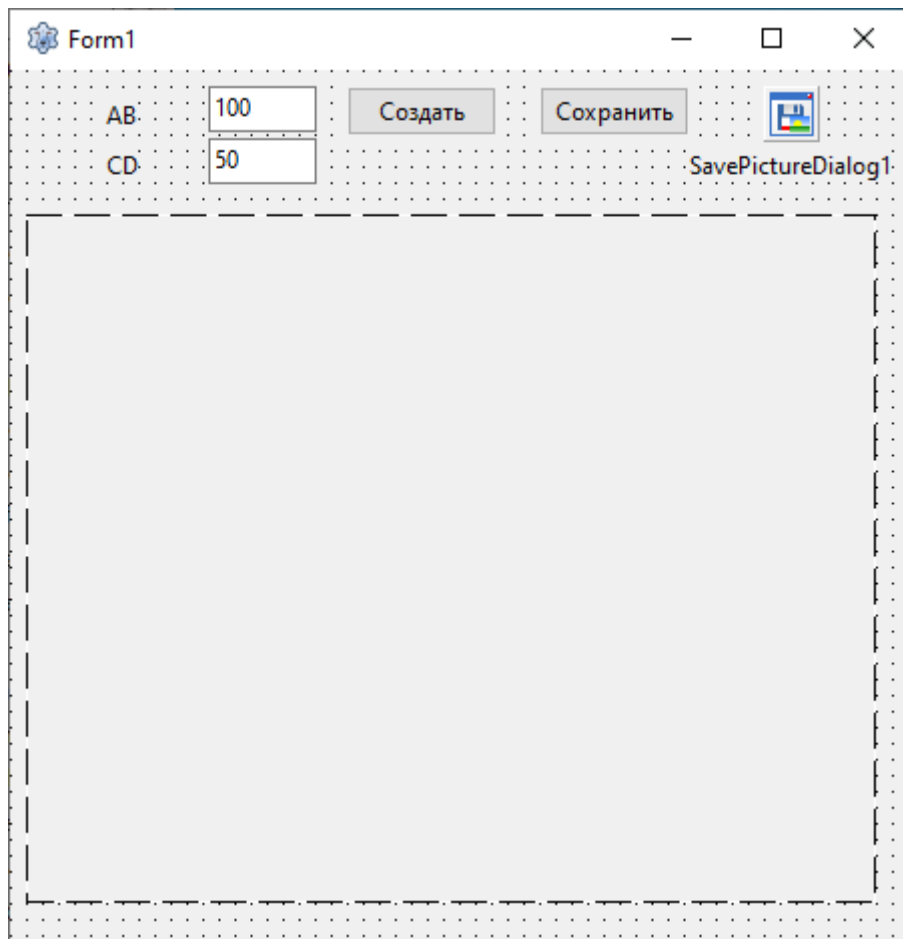


Рис. 4. Форма программы прорисовки окружностей

При нажатии на кнопку «Создать» из компонентов Tedit считываются длины осей, для битмапа Image1 выполняется очистка, задаётся формат пикселей pf24bit, задаются ширина и высота по размерам длин осей + 50, чтобы поместился овал, обрамлённый пустотой, выполняется заполнение белым фоном.

Далее вычисляются параметры дуг окружностей и разным цветом прорисовываются дуги по уравнению окружности, параметрическому уравнению, алгоритму Брезенхема и встроенным методом Tcanvas.arc(). Методика построения четырёхцентрального овала по заданным длинам осей приведена в [5]. Примеры построения дуг окружностей приведены в [6].

Получили модуль приблизительно следующего содержания:

```
unit Unit1;
```

```
{ $mode objfpc } { $H+ }
```

interface

uses

Classes, SysUtils, FileUtil, Forms, Controls, Graphics, Dialogs,
StdCtrls,
GraphType, IntfGraphics, LCLType, LCLProc, LCLIntf, ExtCtrls,
ExtDlgs,
fpImage, crt, math;
type

{ TForm1 }

TForm1 = class(TForm)

Button1: TButton;

Button3: TButton;

Edit6: TEdit;

Edit7: TEdit;

Label1: TLabel;

Label3: TLabel;

SavePictureDialog1: TSavePictureDialog;

Image1: TImage;

procedure Button1Click(Sender: TObject);

procedure Button3Click(Sender: TObject);

private

{ private declarations }

public

{ public declarations }

end;

var

Form1: TForm1;

implementation

{ \$R *.lfm }

{ TForm1 }

```

procedure arceq(x0, y0, r, x1,y1,x2,y2:real; canvas:TCanvas);
var x,y,kp,bp:real;
    inv:integer;
begin
    if x1>=x2 then inv:=-1 else inv:=1;//нужна ли инверсия
    if x1=x2 then x1:=x1+0.1/r;// плутовство исключить вертикали.
    kp:=(y1-y2)/(x1-x2);//угловой коэффициент секущей хорды
    bp:=y1-x1*kp;//свободный член секущей хорды
    //прорисовываем как функцию y=f(x) по точкам
    x:=0;// начиная с x=0
    y:=r;
    while x<y do //до угла 45
        begin //если точки с нужной стороны секущей
            y:=sqrt(sqr(r)-sqr(x));//вывод точек в симметричных секторах
            if inv*((y0+y)-(kp*(x0+x)+bp))<=0 then
                canvas.Pixels[floor(x0+x),floor(y0+y)]:=canvas.Pen.Color;
            if inv*((y0-y)-(kp*(x0+x)+bp))<=0 then
                canvas.Pixels[floor(x0+x),floor(y0-y)]:=canvas.Pen.Color;
            if inv*((y0+y)-(kp*(x0-x)+bp))<=0 then
                canvas.Pixels[floor(x0-x),floor(y0+y)]:=canvas.Pen.Color;
            if inv*((y0-y)-(kp*(x0-x)+bp))<=0 then
                canvas.Pixels[floor(x0-x),floor(y0-y)]:=canvas.Pen.Color;
            x:=x+1
        end;
    while y>=0 do //с угла 45 до y=0 выводим как функцию x=g(y)
        begin //если точки с нужной стороны секущей
            x:=sqrt(sqr(r)-sqr(y)); //вывод точек в симметричных секторах
            if inv*((y0+y)-(kp*(x0+x)+bp))<=0 then
                canvas.Pixels[floor(x0+x),floor(y0+y)]:=canvas.Pen.Color;
            if inv*((y0-y)-(kp*(x0+x)+bp))<=0 then
                canvas.Pixels[floor(x0+x),floor(y0-y)]:=canvas.Pen.Color;
            if inv*((y0+y)-(kp*(x0-x)+bp))<=0 then
                canvas.Pixels[floor(x0-x),floor(y0+y)]:=canvas.Pen.Color;
            if inv*((y0-y)-(kp*(x0-x)+bp))<=0 then
                canvas.Pixels[floor(x0-x),floor(y0-y)]:=canvas.Pen.Color;
            y:=y-1
        end;
    end;
end;

```

```

    end;
end;

```

```

procedure arcparam(x0, y0, r, x1,y1,x2,y2:real; canvas:TCanvas);
var x,y,tau,tau0,sa,ea:real;
begin
    if x1=x0 then if y1>y0 then sa:=pi/2 else sa:=3*pi/2
        else sa:=arctan((y1-y0)/(x1-x0)); //начальный угол
    if x2=x0 then if y2>y0 then ea:=pi/2 else ea:=3*pi/2
        else ea:=arctan((y2-y0)/(x2-x0)); //конечный угол
    if x1<x0 then sa:=pi+sa; //коррекция полуплоскости нач угла
    if x2<x0 then ea:=pi+ea; //коррекция полуплоскости кон угла
    if sa<0 then sa:=sa+2*pi; //коррекция знака начального угла
    if ea<0 then ea:=ea+2*pi; //коррекция знака конечного угла
    if sa>ea then //если начальный угол больше конечного
        ea:=ea+2*pi; //доворачиваем конечный угол на оборот
    tau0:=1/r; //чем больше радиус тем меньше шаг.
    tau:=sa; //начинаем с начального угла
    while tau<ea do //рисует до конечного
        begin
            x:=x0+r*cos(tau); // параметрическое уравнение для x
            y:=y0+r*sin(tau); // параметрическое уравнение для y
            canvas.Pixels[floor(x),floor(y)]:=canvas.Pen.Color; // точка
            tau:=tau+tau0; //увеличиваем угол
        end;
    end;
end;

```

```

procedure arcbrez(x0, y0, r, x1,y1,x2,y2:integer; canvas:TCanvas);
const pi=3.1415926;
var x,y,delta,spx,sp,spy,sp,sag,eag,sp,ep:integer;
    sa,ea,rs,re,t:real;
    dr,inv:boolean;
begin
    x:=0; //начальное значение x для алгоритма Брезенхема
    y:=R; //начальное значение y для алгоритма Брезенхема
    delta:=2-2*r; //ошибка

```

```

rs:=sqrt(sqr(x1-x0)+sqr(y1-y0));//длина луча начального угла
re:=sqrt(sqr(x2-x0)+sqr(y2-y0));//длина луча конечного угла
sa:=arcsin((y1-y0)/rs);//начальный угол
ea:=arcsin((y2-y0)/re);//конечный угол
if x1<x0 then sa:=pi-sa;//коррекция полуплоскости нач угла
if x2<x0 then ea:=pi-ea;//коррекция полуплоскости кон угла
if sa<0 then sa:=sa+2*pi;//коррекция знака начального угла
if ea<0 then ea:=ea+2*pi;//коррекция знака конечного угла
inv:=false;//нужна ли инверсия
if sa>ea then//если начальный угол больше конечного
begin
  t:=sa;
  sa:=ea;
  ea:=t; //меняем их местами
  inv:=true;//и инвертируем вывод
end;
sag:=round(sa*360/2/pi);//начальный угол в градусах
eag:=round(ea*360/2/pi);//конечный угол в градусах
spx:=round(x0+r*cos(sa));//абсцисса начала сектора
spy:=round(y0+r*sin(sa));//ордината начала сектора
epx:=round(x0+r*cos(ea));//абсцисса конца сектора
epy:=round(y0+r*sin(ea));//ордината конца сектора
while x<=y do //вывод по алгоритму Бразенхейма 8 секторов
begin
  dr:=false;//если есть все условия вывода для каждого сектора
  if (sag<=45)and(eag>=0) then
begin
  if (sag>=0) then sp:=spy else sp:=y0;
  if (eag<45) then ep:=epy else ep:=y0+r;
  if (y0+x>=sp) and (y0+x<=ep) then dr:=true;
end;
  if (dr and (not inv)) or (inv and (not dr)) then
    canvas.Pixels[x0+y,y0+x]:=canvas.Pen.Color;//1 (обп)
  dr:=false;
  if (sag<90) and (eag>=45) then
begin
  if (sag>=45) then sp:=spx else sp:=x0+r;
  if (eag<90) then ep:=epx else ep:=x0;

```

```

    if (x0+x <= sp) and (x0+x >= ep) then dr:=true;
end;
if (dr and (not inv)) or (inv and (not dr)) then
    canvas.Pixels[x0+x,y0+y]:=canvas.Pen.Color;//2
dr:=false;
if (sag<135) and (eag>=90) then
begin
    if (sag>=90) then sp:=spx else sp:=x0;
    if (eag<135) then ep:=epx else ep:=x0-r;
    if (x0-x <= sp) and (x0-x >= ep) then dr:=true;
end;
if (dr and (not inv)) or (inv and (not dr)) then
    canvas.Pixels[x0-x,y0+y]:=canvas.Pen.Color;//3 (o6p)
dr:=false;
if (sag<180) and (eag>=135) then
begin
    if (sag>=135) then sp:=spy else sp:=y0+r;
    if (eag<180) then ep:=epy else ep:=y0;
    if (y0+x <= sp) and (y0+x >= ep) then dr:=true;
end;
if (dr and (not inv)) or (inv and (not dr)) then
    canvas.Pixels[x0-y,y0+x]:=canvas.Pen.Color;//4
dr:=false;
if (sag<225) and (eag>=180) then
begin
    if (sag>=180) then sp:=spy else sp:=y0;
    if (eag<225) then ep:=epy else ep:=y0-r;
    if (y0-x <= sp) and (y0-x >= ep) then dr:=true;
end;
if (dr and (not inv)) or (inv and (not dr)) then
    canvas.Pixels[x0-y,y0-x]:=canvas.Pen.Color;//5 (o6p)
dr:=false;
if (sag<270) and (eag>=225) then
begin
    if (sag>=225) then sp:=spx else sp:=x0-r;
    if (eag<270) then ep:=epx else ep:=x0;
    if (x0-x >= sp) and (x0-x <= ep) then dr:=true;
end;

```

```

if (dr and (not inv)) or (inv and (not dr)) then
    canvas.Pixels[x0-x,y0-y]:=canvas.Pen.Color;//6
dr:=false;
if (sag<315) and (eag>=270) then
    begin
        if (sag>=270) then sp:=spx else sp:=x0;
        if (eag<315) then ep:=epx else ep:=x0+r;
        if (x0+x >= sp) and (x0+x <= ep) then dr:=true;
    end;
if (dr and (not inv)) or (inv and (not dr)) then
    canvas.Pixels[x0+x,y0-y]:=canvas.Pen.Color;//7 (обр)
dr:=false;
if (sag<360) and (eag>=315) then
    begin
        if (sag>=315) then sp:=spy else sp:=y0-r;
        if (eag<360) then ep:=epy else ep:=y0;
        if (y0-x >= sp) and (y0-x <= ep) then dr:=true;
    end;
if (dr and (not inv)) or (inv and (not dr)) then
    canvas.Pixels[x0+y,y0-x]:=canvas.Pen.Color;//8
// Переход к следующей точке
if (delta<0) and (2*(delta+y)-1<=0) then
    begin
        x:=x+1;
        delta:=delta+2*x+1
    end
else
    if (delta>0) and (2*(delta-x)-1>0) then
        begin
            y:=y-1;
            delta:=delta-2*y+1
        end
    else
        begin
            x:=x+1;
            y:=y-1;
            delta:=delta+2*(x-y)
        end;
end;

```

```
end;  
end;
```

```
procedure TForm1.Button1Click(Sender: TObject); //Кнопка Создать1  
var ab,cd,bord:integer;  
    oa,oc,xa,ya,xb,yb,xc,yc,xd,yd,xo,yo:real;  
    ce,ca,ae,xe,ye,xf,yf,kac,bac:real;  
    kgl,bgl,xk,yk,xl,yl:real;  
    r1,r2,xm,ym,xn,yn,sxg,xg,yg,xh,yh,xi,yi,xj,yj:real;  
begin  
    with Image1.Picture.Bitmap do //С битмапом на форме TImage  
    begin  
        Clear; //а вдруг там мусор  
        PixelFormat:=pf24bit; //формат пикселей  
        AB:=StrToInt(Edit6.Text); //большая ось овала  
        CD:=StrToInt(Edit7.Text); //малая ось овала  
        bord:=50; //бордюр  
        Width:=AB + 2*bord; //Ширина  
        Height:=CD + 2*bord; //Высота  
        Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder); //Белый фон  
        canvas.Pen.Color:=RGB(127,127,127); //цвет точек  
        OA:=AB/2; //половина большой оси  
        OC:=CD/2; //половина малой оси  
        xa:=0; //абсцисса точки A  
        ya:=OC; //ордината точки A  
        xb:=xa+ab; //абсцисса точки B  
        yb:=ya; //ордината точки B  
        xc:=OA; //абсцисса точки C  
        yc:=0; //ордината точки C  
        xd:=xc; //абсцисса точки D  
        yd:=yc+cd; //ордината точки D  
        xo:=xc; //абсцисса точки O  
        yo:=ya; //ордината точки O  
        CA:=sqrt(sqr(OA)+sqr(OC)); //длина отрезка CA  
        CE:=OA-OC; //длина отрезка CE  
        AE:=CA-CE; //Длина отрезка AE  
        xe:=(xa*CE+xc*AE)/CA; //абсцисса точки E
```

```

ye:=(ya*CE+yc*AE)/CA;//ордината точки E
xf:=(xe+xa)/2;//абсцисса точки F, AF=FE
yf:=(ye+ya)/2;//ордината точки F
кас:=-oc/oa;//угловой коэффициент прямой AC
bas:=ya-xa*кас;//свободный член прямой AC
kgl:=-1/кас;//угловой коэффициент прямой GL
bgl:=xf/кас+yf;//свободный член прямой GL
//центры малых окружностей овала – в точке K на пересече-
нии АВ и GL и в точке М симметрично.
yk:=ya; //ордината точки K
xk:=(yk-bgl)/kgl; //абсцисса точки K
r1:=xk-xa; //радиус малой окружности
xm:=xb-r1; //абсцисса точки М
ym:=yb; //ордината точки М
//центр большой окружности овала - в точке L на пересечении
CD и GL и в точке N симметрично.
xl:=xc; //абсцисса точки L
yl:=kgl*xl+bgl; //ордината точки L
r2:=yl-yc; //радиус большой окружности
xn:=xd; //абсцисса точки N
yn:=yo-(yl-yo); //ордината точки N
//Координаты точки G сопряжения овала
sxg:=-sqr(yk)+(2*kgl*xk+2*bgl)*yk-sqr(kgl)*sqr(xk)-
2*bgl*kgl*xk+(sqr(kgl)+1)*sqr(r1)-sqr(bgl);
xg:=min((sqr(sxg)+kgl*yk+xk-bgl*kgl)/(sqr(kgl)+1),(-
sqr(sxg)+kgl*yk+xk-bgl*kgl)/(sqr(kgl)+1));
yg:=kgl*xg+bgl; //ордината точки G
// точки сопряжения H, J, и I находятся симметрично.
xh:=xg; //абсцисса точки H
yh:=yk+(yk-yg); //ордината точки H
xi:=xl+(xl-xg); //абсцисса точки I
yi:=yg; //ордината точки I
xj:=xi; //абсцисса точки J
yj:=yh; //ордината точки J
canvas.Pen.Color:=RGB(191,191,191); //вывод осей серым
can-
vas.line(round(bord+xa),round(bord+ya),round(bord+xb),round(bord+yb))
;

```

```

can-
vas.line(round(bord+xc),round(bord+yc),round(bord+xd),round(bord+yd))
;
canvas.Pen.Color:=RGB(191,191,255);
//Вывод овала по уравнениям окружностей - голубым
arceq(bord+xk,bord+yk,r1,bord+xh,bord+yh,bord+xg,bord+yg,canvas);
arceq(bord+xm,bord+ym,r1,bord+xi,bord+yi,bord+xj,bord+yj,canvas);
arceq(bord+xl,bord+yl,r2,bord+xg,bord+yg,bord+xi,bord+yi,canvas);
arceq(bord+xn,bord+yn,r2,bord+xj,bord+yj,bord+xh,bord+yh,canvas);
canvas.Pen.Color:=RGB(255,191,191);
//Вывод овала по параметрическому уравнению - розовым
arcparam(bord+xk,bord+yk,r1,bord+xh,bord+yh,bord+xg,bord+yg,can-
vas);
arcparam(bord+xm,bord+ym,r1,bord+xi,bord+yi,bord+xj,bord+yj,canvas);
arcparam(bord+xl,bord+yl,r2,bord+xg,bord+yg,bord+xi,bord+yi,canvas);
arcparam(bord+xn,bord+yn,r2,bord+xj,bord+yj,bord+xh,bord+yh,canvas);
    canvas.Pen.Color:=RGB(191,255,191);
    //Вывод овала по алгоритму Брезенхайма - зелёным
arcbrez(round(bord+xk),round(bord+yk),round(r1),
round(bord+xh),round(bord+yh),round(bord+xg),round(bord+yg),canvas);
arcbrez(round(bord+xm),round(bord+ym),round(r1),
round(bord+xi),round(bord+yi),round(bord+xj),round(bord+yj),canvas);
arcbrez(round(bord+xl),round(bord+yl),round(r2),
round(bord+xg),round(bord+yg),round(bord+xi),round(bord+yi),canvas);
arcbrez(round(bord+xn),round(bord+yn),round(r2),
round(bord+xj),round(bord+yj),round(bord+xh),round(bord+yh),canvas);
    canvas.Pen.Color:=RGB(191,191,191);
    //Вывод овала встроенными средствами - серым.
canvas.Arc(round(bord+xa),round(bord+ya-
r1),round(bord+xa+2*r1),round(bord+ya+r1),
round(bord+xg),round(bord+yg),round(bord+xh),round(bord+yh));
canvas.Arc(round(bord+xb-2*r1),round(bord+yb-
r1),round(bord+xb),round(bord+yb+r1),
round(bord+xj),round(bord+yj),round(bord+xi),round(bord+yi));
canvas.Arc(round(bord+xc-
r2),round(bord+yc),round(bord+xc+r2),round(bord+yc+2*r2),
round(bord+xi),round(bord+yi),round(bord+xg),round(bord+yg));

```

```

canvas.Arc(round(bord+xd-r2),round(bord+yd-
2*r2),round(bord+xd+r2),round(bord+yd),
round(bord+xh),round(bord+yh),round(bord+xj),round(bord+yj));
    end;
end;

procedure TForm1.Button3Click(Sender: TObject);
begin
    if SavePictureDialog1.Execute then
        Image1.Picture.SaveToFile(SavePictureDialog1.FileName);
end;

end.

```

Получен результат (рис. 5):

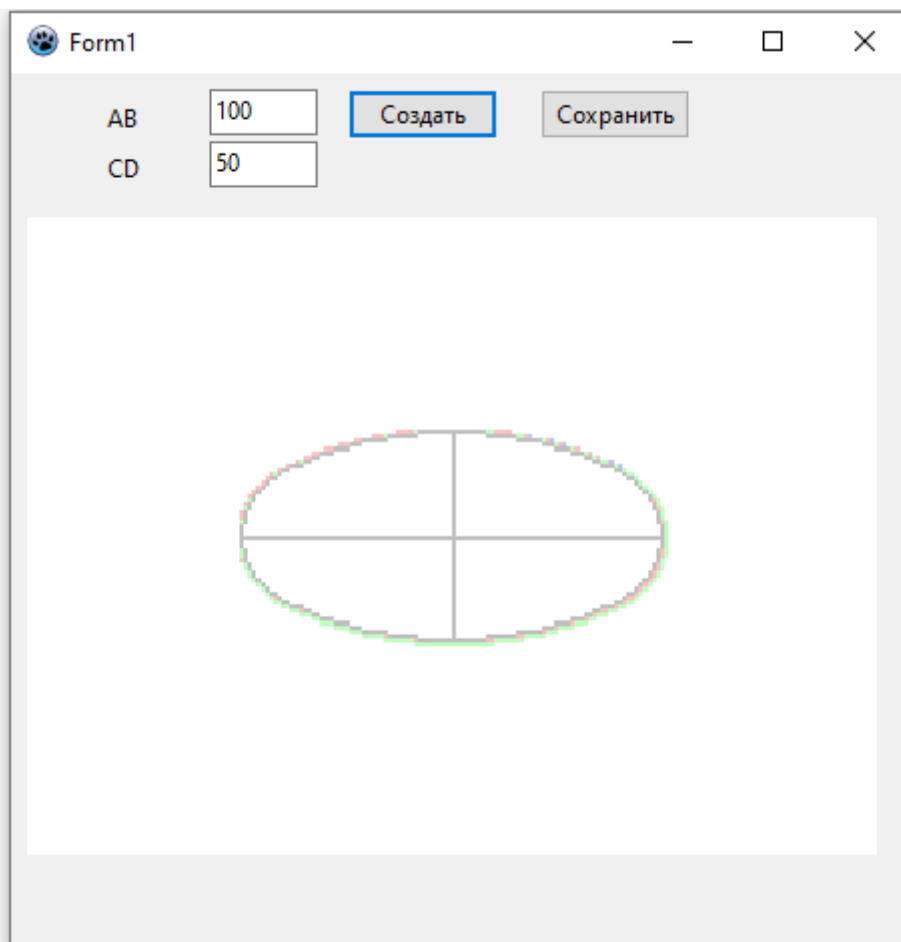


Рис. 5. Результат работы программы прорисовки окружностей

Лабораторная работа № 5.

Попиксельные преобразования

Цель работы – освоение навыков обработки растровых изображений с помощью алгоритмических языков программирования путём изменения характеристик отдельных пикселей.

Краткие теоретические сведения

Для восприятия цвета используются три типа фоточувствительных элементов, так называемых «колбочек», реагирующих на излучение в трёх широких перекрывающихся диапазонах оптического излучения: один – преимущественно в синей области, второй – в зелёной и третий – в красной. Любой цвет соответствует соотношению реакций фоторецепторов трёх типов на спектр излучения. Цветовая модель, соответствующая восприятию человека, называется **LMS**. Так как человеческий глаз формирует цветовые ощущения на основании всего трёх сигналов, любой цвет можно представить в виде взвешенной суммы всего трёх различных цветов, называемых основными. Для вычисления количественных соотношений между реакцией фоторецепторов и цветом были получены функции подбора цвета $\bar{g}(\lambda)$, $\bar{b}(\lambda)$, которые связывали все спектральные цвета ($\lambda \in (380, 780)$) с линейной комбинацией трёх монохроматических основных цветов с длинами волн 700 нм, 546,1 нм и 435,8 нм так, чтобы для каждой λ фоторецепторы выдавали соответствующее соотношение LMS. Выяснилось, что человеческое зрение использует как сложение, так и вычитание сигналов с рецепторов, и если взять любые (красный, зелёный и синий) компоненты, то для представления некоторых спектральных цветов некоторые компоненты нужно вычитать из общей суммы, т. е. задавать отрицательную интенсивность источников основных цветов, что физически невозможно. А если обойтись без вычитания, невозможно представить все воспринимаемые человеком цвета. Для описания всего цветового охвата, на который способно человеческое зрение, с помощью трёх неотрицательных составляющих необходимо, чтобы эти составляющие сами по себе имели невозможные цвета. Так была предложена цветовая модель XYZ на базе функций подбора цвета: $\bar{x}(\lambda)$, $\bar{y}(\lambda)$, $\bar{z}(\lambda)$.

$$\begin{bmatrix} \bar{x}(\lambda) \\ \bar{y}(\lambda) \\ \bar{z}(\lambda) \end{bmatrix} = \begin{bmatrix} 2,768892 & 1,757748 & 1,130160 \\ 1 & 4,590700 & 0,0601 \\ 0 & 0,056508 & 5,594292 \end{bmatrix} \begin{bmatrix} \bar{r}(\lambda) \\ \bar{g}(\lambda) \\ \bar{b}(\lambda) \end{bmatrix}$$

В этой модели Z условно обозначает синюю компоненту, Y – яркость, а X – сумму всех цветовых компонент с такими весами, что XYZ везде неотрицательны.

С помощью этой модели можно представить все возможные и некоторые невозможные цвета, поэтому её используют в качестве эталонной для описания цвета и в качестве промежуточной для преобразования между другими цветовыми моделями. Позже были найдены соотношения для преобразования LMS в XYZ.

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 1,91020 & -1,11212 & 0,20191 \\ 0,37095 & 0,62905 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} L \\ M \\ S \end{bmatrix}$$

Восприятие цвета зависит поля зрения и учитываются в модели XYZ путём ввода параметров стандартного наблюдателя. Стандартный наблюдатель CIE 1931 2° характеризует восприятие цвета в пределах поля зрения 2°. Стандартный наблюдатель CIE 1964 10° характеризует восприятие цвета в пределах поля зрения 10° и рекомендован к применению, если поле зрения превышает 4° (эксперимент был с длинами волн $r=645,16\text{nm}$, $g=526,32\text{nm}$, $b=444,44\text{nm}$).

Зависимость значений XYZ_{2° от длины волны (рис. 1) хорошо аппроксимируется следующими функциями [7]:

$$\begin{aligned} \bar{x}(\lambda) &= 1,056g(\lambda, 599,8, 0,0264, 0,0323) + \\ &+ 0,362g(\lambda, 442, 0,0624, 0,0374) - \\ &- 0,065g(\lambda, 501,1, 0,0490, 0,0382), \\ \bar{y}(\lambda) &= 0,821g(\lambda, 568,8, 0,0213, 0,0247) + \\ &+ 0,286g(\lambda, 530,9, 0,0613, 0,0322), \\ \bar{z}(\lambda) &= 1,217g(\lambda, 437, 0,0845, 0,0278) + 0,681g(\lambda, 459, 0,0385, 0,0725), \\ g(\lambda, \mu, \tau_1, \tau_2) &= \begin{cases} e^{\frac{-\tau_1^2(\lambda-\mu)^2}{2}}, & \lambda < \mu, \\ e^{\frac{-\tau_2^2(\lambda-\mu)^2}{2}}, & \lambda \geq \mu. \end{cases} \end{aligned}$$

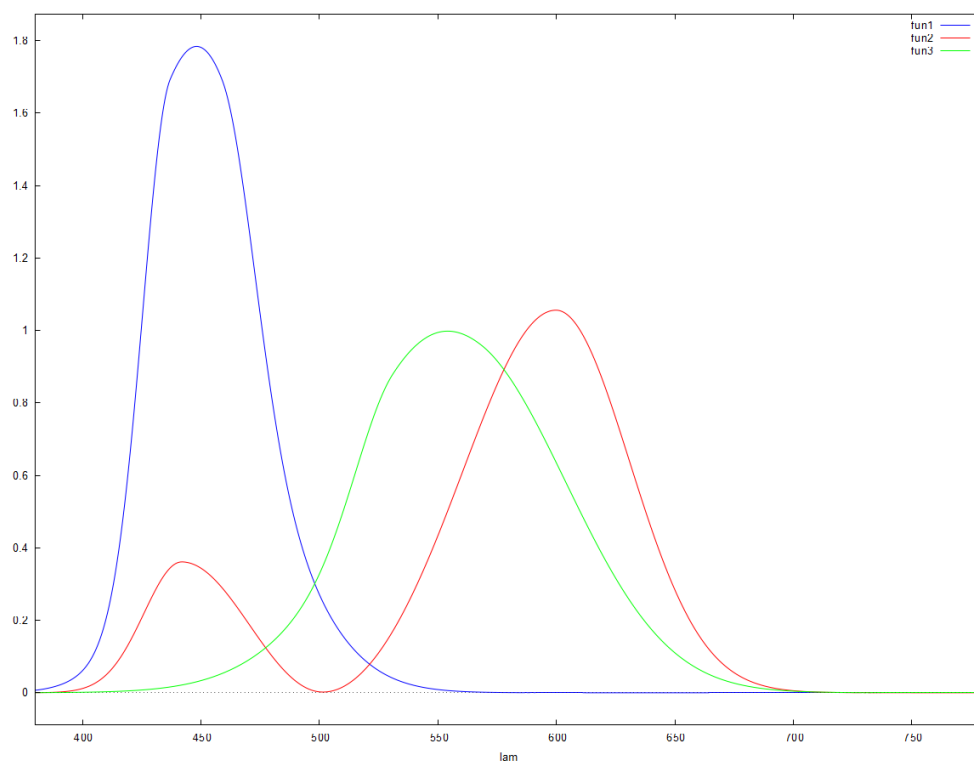


Рис. 1. Функции подбора цвета XYZ

Отообразим составляющие XYZ в декартовой системе координат и рассмотрим плоскость единичной интенсивности $X+Y+Z=1$ (рис. 2).

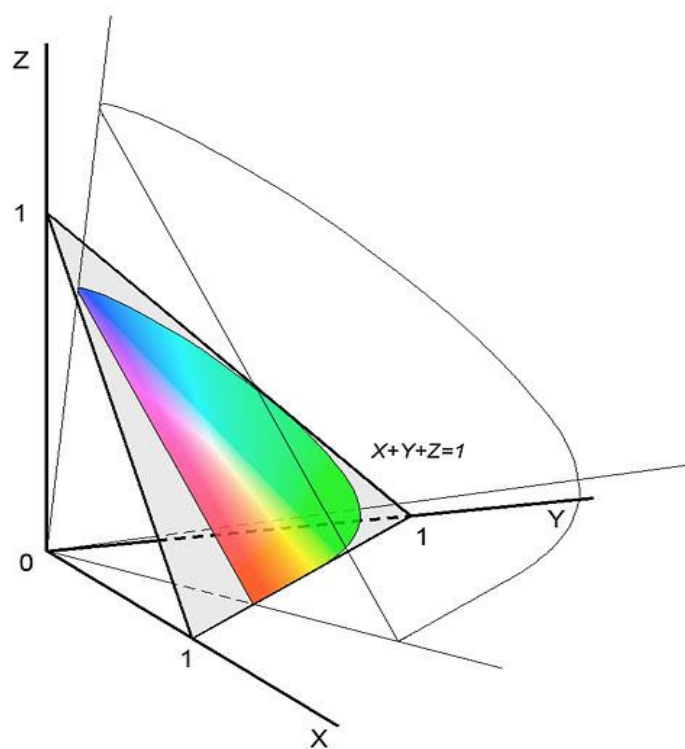


Рис. 2. Цветовое пространство XYZ

Значительную часть этой плоскости будут занимать реальные цвета. Рассмотрим плоскость внимательнее (рис. 3).

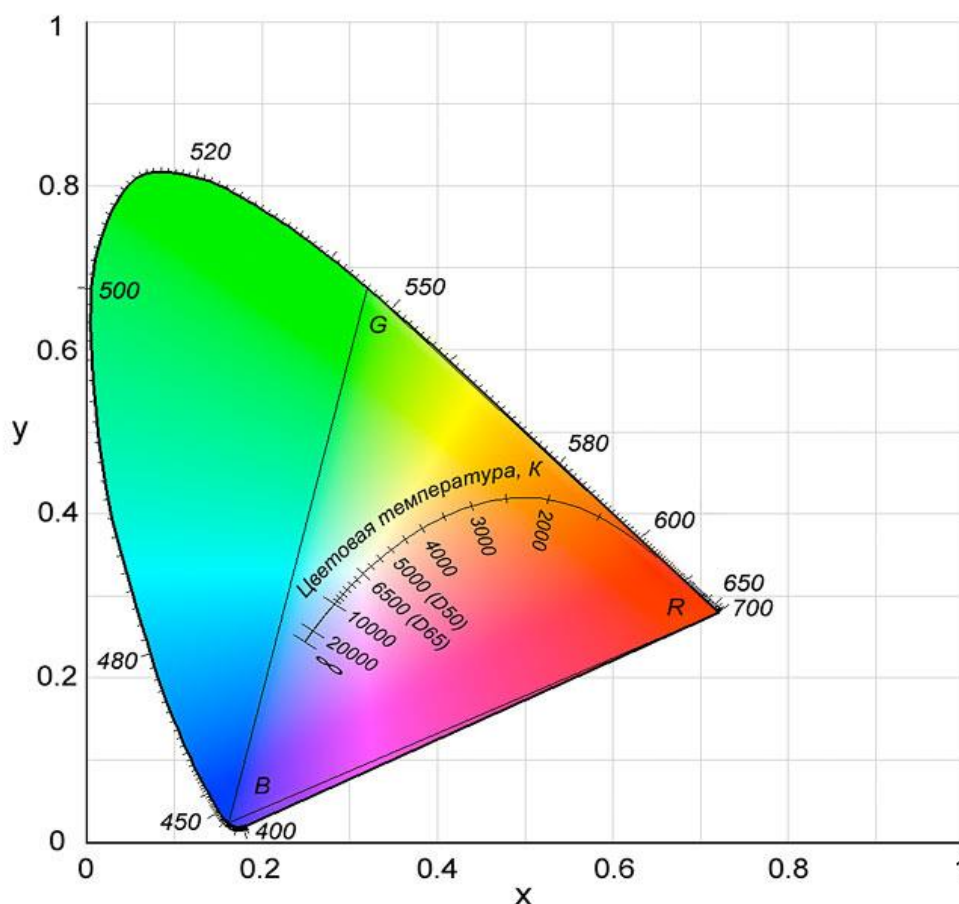


Рис. 3. Хроматическая диаграмма

Монохроматические цвета расположены по границе области реальных цветов. Внутри области представлены все цвета единичной интенсивности, воспринимаемые человеком. Если взять два источника света разных цветов, воспринимаемых человеком, они будут представлены двумя различными точками внутри области. Если эти цвета смешать, получившийся в результате цвет будет расположен на отрезке, соединяющем эти точки. В зависимости от соотношения интенсивностей источников результат будет ближе к тому или иному концу отрезка. Так как область выпуклая, смесь любых реальных цветов будет реальным цветом. Если взять три разных по цвету источника, все цвета, которые можно получить их смешиванием, будут расположены внутри треугольника, образуемого соответствующими точками.

Например, точки RGB на графике соответствуют монохромным источникам с длинами волн 700, 546,1 и 435,8 нм. Очевидно, никакие три реальных цвета, взятых за основные, не позволяют в результате смешивания получить всю совокупность реальных цветов. Множество представимых цветов называется **гамутом**. На деле не всегда удаётся использовать монохромные источники. Как следствие, вершины фигуры, описывающей гамут, находятся внутри области реальных цветов, и гамут получается сравнительно узким (рис. 4).

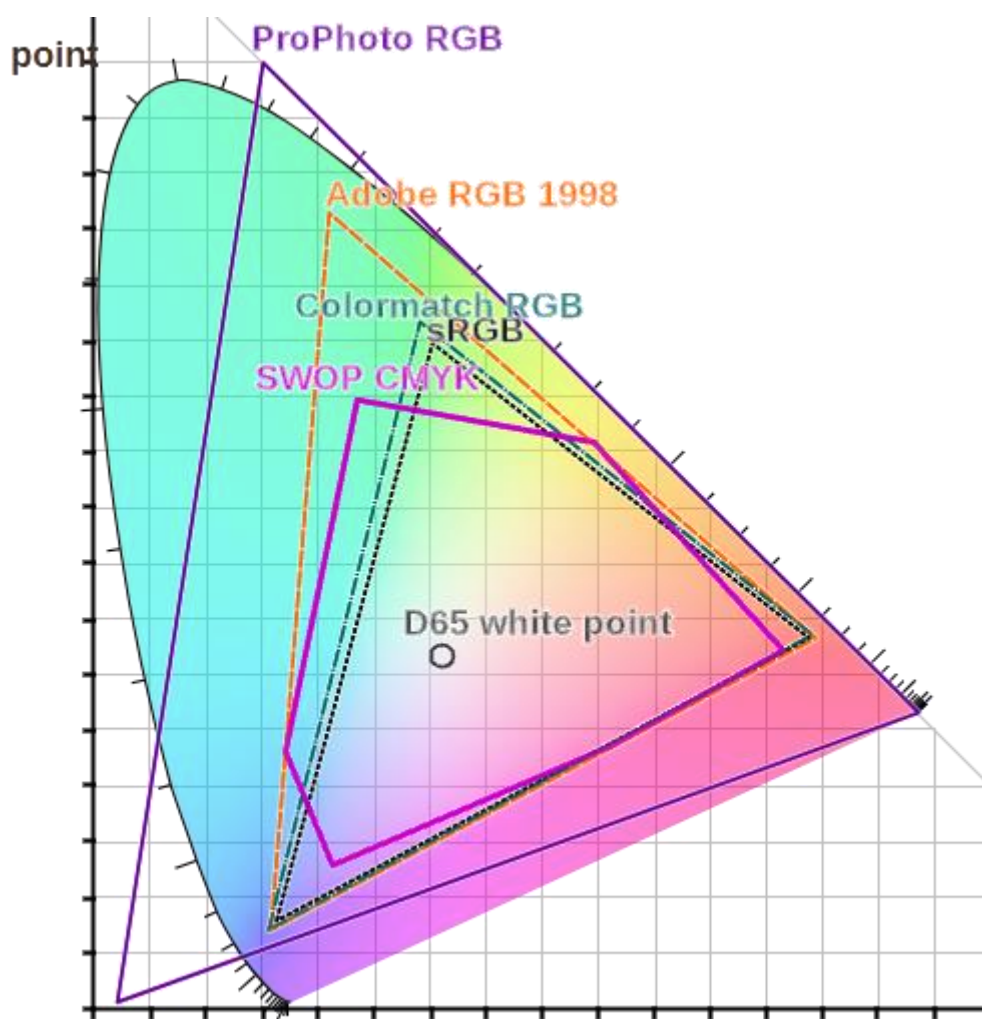


Рис. 4. Гамуты разных цветовых пространств

Любой цвет, представленный в RGB или в рамках другой цветовой модели, можно представить в XYZ, но не наоборот. Поскольку RGB ориентируется на реальное воспроизведение цвета с учётом физических свойств материалов, преобразование RGB в XYZ и наоборот может быть нелинейным, поэтому не всегда достаточно использовать

линейную комбинацию компонентов. Кроме того, необходимо учитывать координаты белой точки, т. е. цвет, который в модели RGB принят за белый, чтобы было ясно, каким значениям XYZ соответствует RGB(1,1,1). Для ясности используется один из стандартных осветителей, чаще всего – D65, соответствующий дневному свету с цветовой температурой около 6500K. Стандартные осветители характеризуются координатами белой точки, которые различаются для стандартных наблюдателей (табл. 1).

Таблица 1

Белые точки осветителей для наблюдателя CIE 1931 2°

Осветитель	X _n	Y _n	Z _n
A	109,85	100	35,58
C	98,07	100	118,22
D50	96,4	100	82,5
D55	95,68	100	92,14
D65	95,04	100	108,88
D75	94,97	100	122,61
E	100	100	100

Цветовые модели разрабатывались в разных целях, часто несовместимых друг с другом. XYZ позволяет задать любой существующий цвет в виде трёх неотрицательных чисел, RGB позволяет физически получить множество цветов из цветового треугольника, HSV – задать цвет одним параметром, Lab – приблизиться к линейному отображению цветов так, чтобы одинаковое изменение значений параметров приводило к одинаковому изменению цветовых ощущений, LCh – то же, но цвет задаётся одним параметром.

sRGB<->XYZ. sRGB использует источник D65.

$$\begin{bmatrix} R_{linear} \\ G_{linear} \\ B_{linear} \end{bmatrix} = \begin{bmatrix} 3,2406 & -1,5372 & -0,4986 \\ -0,9689 & 1,8758 & 0,0415 \\ 0,0557 & -0,2040 & 1,0570 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

Для получения sRGB выполняется гамма-коррекция.

$$C_{srgb} = \begin{cases} 12,92C_{linear}, & C_{linear} \leq 0,0031308, \\ 1,055C_{linear}^{1/2,4} - 0,055, & C_{linear} > 0,0031308. \end{cases}$$

Для каждого компонента RGB выполняется линеаризация.

$$C_{linear} = \begin{cases} C_{srgb}/12,92, & C_{srgb} \leq 0,04045, \\ \left(\frac{C_{srgb}+0,055}{1,055}\right)^{2,4} & C_{srgb} > 0,04045. \end{cases}$$

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 0,4124 & 0,3576 & 0,1805 \\ 0,2126 & 0,7152 & 0,0722 \\ 0,0193 & 0,1192 & 0,9502 \end{bmatrix} \begin{bmatrix} R_{linear} \\ G_{linear} \\ B_{linear} \end{bmatrix}$$

XYZ<->Lab.

$$L = 116f(Y/Y_n) - 16,$$

$$a = 500(f(X/X_n) - f(Y/Y_n)),$$

$$b = 200(f(Y/Y_n) - f(Z/Z_n)),$$

где $f(t) = \begin{cases} \sqrt[3]{t} & t > \varepsilon, \\ \frac{kt+16}{116} & t \leq \varepsilon, \end{cases}$ X_n, Y_n, Z_n – координаты белой точки,
 $\varepsilon=(6/29)^3, k=(29/3)^3$.

L принимает значения в диапазоне $[0,100]$, a – в диапазоне $[0,500]$, b – в диапазоне $[0,200]$. Не все комбинации L, a, b будут обозначать воспроизводимый цвет.

$$\begin{aligned} X &= xX_n \\ Y &= yY_n, \text{ где } x = \begin{cases} f_x^3 & f_x^3 > \varepsilon, \\ \frac{116f_x-16}{k} & f_x^3 \leq \varepsilon, \end{cases} y = \begin{cases} \left(\frac{L+16}{116}\right)^3 & L > k\varepsilon, \\ \frac{L}{k} & L \leq k\varepsilon, \end{cases} \\ Z &= zZ_n \\ z &= \begin{cases} f_z^3 & f_z^3 > \varepsilon, \\ \frac{116f_z-16}{k} & f_z^3 \leq \varepsilon, \end{cases} \end{aligned}$$

где $f_x = \frac{a}{500} + f_y; f_z = f_y - \frac{b}{200}; f_y = \frac{L+16}{116}$.

Lab<->LCh.

$$L = L,$$

$$C = \sqrt{a^2 + b^2},$$

$$h = \begin{cases} \arctan(b/a), & a > 0, b \geq 0, \\ \arctan(b/a) + 180^\circ, & a < 0, \\ \arctan(b/a) + 360^\circ, & a > 0, b < 0, \\ 90^\circ, & a = 0, b > 0, \\ 270^\circ, & a = 0, b < 0, \\ 0^\circ, & a = 0, b = 0. \end{cases}$$

Комбинация $a=0, b=0$ означает серый цвет без какой-либо хроматической составляющей и в этом случае величина h не имеет смысла и

может иметь любое значение, ноль в формуле указан для определённости. L , обозначающая яркость, соответствует Lab, C может принимать любые неотрицательные значения, h принимает значения в пределах $[0, 360^\circ)$.

$$L = L,$$

$$a = C \cosh,$$

$$b = C \sinh.$$

Преобразование Lab $\leftrightarrow$ LCh взаимно однозначно, и в LCh, как и в Lab, и в XYZ могут быть заданы невоспроизводимые цвета.

RGB $\leftrightarrow$ HSV.

$\max = \max(R, G, B), \quad \min = \min(R, G, B), \quad H \in [0, 360^\circ),$
 $S, V, R, G, B \in [0, 1),$

$$H = \begin{cases} 0, & \max = \min, \\ 60 \frac{G-B}{\max-\min}, & \max = R, G \geq B, \\ 60 \frac{G-B}{\max-\min} + 360, & \max = R, G < B, \\ 60 \frac{B-R}{\max-\min} + 120, & \max = G, \\ 60 \frac{R-G}{\max-\min} + 240, & \max = B, \end{cases}$$

$$S = \begin{cases} 0, & \max = 0, \\ 1 - \frac{\min}{\max}, & \max \neq 0, \end{cases}$$

$$V = \max$$

$$H_i = \left[\frac{H}{60} \right] \bmod 6, f = \frac{H}{60} - \left[\frac{H}{60} \right], \text{ где } [\] - \text{ взятие целой части.}$$

В зависимости от значения H_i компоненты R, G, B принимают значения V, p, q, t (табл. 2).

$$p = V(1 - S), q = V(1 - fS), t = V(1 - (1 - f)S).$$

Таблица 2

Выбор значений R, G, B при преобразовании из HSV

H_i	R	G	B	H_i	R	G	B
0	V	t	p	3	p	q	V
1	q	V	p	4	t	p	V
2	p	V	t	5	V	p	q

Цветовое расстояние

Международный комитет CIE задаёт определение цветовой разницы через ΔE_{ab}^* . Буква «Е» обозначает Empfindung – нем. ощущение.

CIE76: Используя координаты в цветовом пространстве Lab:

$$\Delta E_{ab} = \sqrt{(L_1 - L_2)^2 + (a_1 - a_2)^2 + (b_1 - b_2)^2}.$$

$\Delta E_{ab} \approx 2.3$ соответствует минимально различимому для человеческого глаза отличию между цветами.

CIE94: ΔE (1994) задавалось в цветовом пространстве LCh:

$$\Delta E_{94} = \sqrt{\left(\frac{L_1 - L_2}{K_L}\right)^2 + \left(\frac{C_1 - C_2}{1 + K_1 C_1}\right)^2 + \left(\frac{h_1 - h_2}{1 + K_2 C_1}\right)^2},$$

где весовые коэффициенты зависят от области применения:

Искусство: $K_L=1, K_1=0,045, K_2=0,015$

Промышленность: $K_L=2, K_1=0,048, K_2=0,014$

Ввиду того, что определение 1994 г. не полностью устранило неоднородности восприятия цветового различия, комитет CIE разработал новый стандарт, который включал пять дополнений:

- 1) поворот цветового угла тона (R_T), чтобы устранить проблемы в синей области (угол Hue 275°);
- 2) компенсация для нейтральных цветов;
- 3) компенсация для светлоты (S_L);
- 4) компенсация для цветности (S_C);
- 5) компенсация для тона (S_H).

Используя координаты (L_1^*, a_1^*, b_1^*) и (L_2^*, a_2^*, b_2^*) в цветовом пространстве Lab и весовые коэффициенты k_L, k_C, k_H (если весовые коэффициенты неизвестны, можно все их принять равными единице):

$$\Delta E_{00}^* = \sqrt{\left(\frac{\Delta L'}{k_L S_L}\right)^2 + \left(\frac{\Delta C'}{k_C S_C}\right)^2 + \left(\frac{\Delta H'}{k_H S_H}\right)^2 + R_T \left(\frac{\Delta C'}{k_C S_C}\right)^2 \left(\frac{\Delta H'}{k_H S_H}\right)^2},$$

$$\Delta L' = L_2^* - L_1^*,$$

$$\Delta C' = C_2' - C_1',$$

где

$$C_1' = \sqrt{a_1'^2 + b_1'^2}, C_2' = \sqrt{a_2'^2 + b_2'^2},$$

$$a_1' = (1 + G)a_1^*, a_2' = (1 + G)a_2^*,$$

$$G = 0,5 \left(1 - \sqrt{\frac{\bar{C}_{ab}^{*7}}{\bar{C}_{ab}^{*7} + 25^7}} \right),$$

$$\begin{aligned}
\bar{C}_{ab}^* &= (C_{1,ab}^* + C_{2,ab}^*)/2, \\
C_{1,ab}^* &= \sqrt{a_1^{*2} + b_1^{*2}}, C_{2,ab}^* = \sqrt{a_2^{*2} + b_2^{*2}}, \\
\Delta H' &= 2\sqrt{C_1' C_2'} \sin(\Delta h'/2), \\
\Delta h' &= \begin{cases} 0, & C_1' C_2' = 0, \\ h_2' - h_1', & C_1' C_2' \neq 0; |h_2' - h_1'| \leq 180^\circ, \\ (h_2' - h_1') - 360, & C_1' C_2' \neq 0; (h_2' - h_1') > 180^\circ, \\ (h_2' - h_1') + 360, & C_1' C_2' \neq 0; (h_2' - h_1') < -180^\circ, \end{cases} \\
S_L &= 1 + \frac{0,015(\bar{L}' - 50)^2}{\sqrt{20 + (\bar{L}' - 50)^2}}, \\
\bar{L}' &= (L_1^* + L_2^*)/2, \\
S_C &= 1 + 0,045\bar{C}', \\
\bar{C}' &= (C_1' + C_2')/2, \\
S_H &= 1 + 0,015\bar{C}' T, \\
T &= 1 - 0,17 \cos(\bar{h}' - 30^\circ) + 0,24 \cos(2\bar{h}') + \\
&\quad + 0,32 \cos(3\bar{h}' + 6^\circ) - 0,2 \cos(4\bar{h}' - 63^\circ), \\
\bar{h}' &= \\
&= \begin{cases} (h_1' + h_2')/2, & |h_2' - h_1'| \leq 180^\circ, C_1' C_2' \neq 0, \\ (h_1' + h_2' + 360^\circ)/2, & |h_2' - h_1'| > 180^\circ, (h_1' + h_2') < 360^\circ; C_1' C_2' \neq 0, \\ (h_1' + h_2' - 360^\circ)/2, & |h_2' - h_1'| > 180^\circ, (h_1' + h_2') \geq 360^\circ; C_1' C_2' \neq 0, \\ h_1' + h_2', & C_1' C_2' = 0, \end{cases} \\
R_T &= -2 \sqrt{\frac{\bar{C}'^7}{\bar{C}'^7 + 25^7}} \sin\left(60e^{-\left[\frac{\bar{h}' - 275^\circ}{25}\right]^2}\right).
\end{aligned}$$

Попиксельные преобразования

Попиксельные или поэлементные преобразования предполагают выполнение одного и того же преобразования каждого пикселя вне зависимости от его позиции и характеристик других пикселей изображения, т. е. результат преобразования для каждого элемента изображения будет зависеть только от этого элемента.

1. *Изменение яркости.* Для максимально возможного увеличения яркости надо найти максимальное значение яркости среди всех обрабатываемых пикселей, вычислить, сколько не хватает до максимально возможного, и прибавить полученное значение всем пикселям. Для уменьшения яркости надо найти минимальное значение и вычесть его из каждого пикселя. Для этого преобразования следует пользоваться моделью, имеющей отдельный яркостной канал, например HSV.

2. *Изменение контраста.* Для максимально возможного увеличения яркостного контраста надо уменьшить яркость до минимума, найти максимальное значение яркости, вычислить, во сколько раз его надо увеличить до максимально возможного значения и увеличить яркость всех пикселей в полученное число раз. Если контраст нужно изменить так, чтобы яркость V изображения $[0,1]$ не выходила за пределы определённого диапазона: $[a, b]$, $0 < a < b < 1$, нужно её преобразовать, соответственно $V_{res} = a + (b - a)V$.

3. *Преобразование в чёрно-белое.* Заданное в RGB — изображение переводится в чёрно-белое с помощью эмпирической формулы: $V = 0,2125R + 0,7154G + 0,0721B$. Каждому компоненту RGB присваивается полученное значение.

4. *Баланс белого по блику.* Необходимо, если источник нестандартный. Если есть блик, обычно в нём присутствует самая яркая точка, цвет которой соответствует цвету источника. Нужно сделать её белой (или серой), а цвет остальных точек скорректировать соответствующим образом. Для этого нужно найти координаты самой яркой точки, определить значения RGB, выбрать минимальное значение из RGB, определить коэффициенты для двух оставшихся каналов так, чтобы при умножении они сравнялись с минимальным. Коэффициент для канала с минимальным значением будет равен 1. Далее значения всех пикселей этих каналов умножаются на соответствующие коэффициенты. После этого можно скорректировать яркость.

5. *Баланс белого методом серого мира.* Предполагая, что в среднем изображение серое, надо найти среднее значение красной, зелёной и синей компонент по каждому из каналов RGB и аналогично предыдущему пункту определить коэффициенты, такие, чтобы при умножении на них средние стали одинаковыми. Далее значения всех пикселей этих каналов умножаются на соответствующие коэффициенты. После этого можно скорректировать яркость.

6. *Негатив*. Вычитаем значения каждого канала RGB из максимально возможного.

7. *Постеризация (изогелия, изополихромия)* – уменьшение количества полутонов. Выполняется путём квантования соответствующего канала на меньшее количество уровней. Для этого можно умножить значение нужного канала каждого пикселя на такое число, чтобы максимальное значение результата было равно заданному количеству уровней, отбросить дробную часть и разделить результат на то же число.

8. *Бинаризация* — частный случай постеризации, когда количество полутонов сокращается до двух. Может выполняться сравнением уровня соответствующего канала с заданным пороговым уровнем, значение которого лежит на интервале от минимального до максимального значения канала. Пиксели, уровень обрабатываемого канала которых меньше порога, получают в соответствующий канал 0, в противоположном случае – 100%

9. *Функциональное преобразование яркости*. После вычисления значения функции для каждого пикселя результат надо нормализовать в пределы допустимого. Например, гамма-коррекция выполняется по формуле $V_{res}=V^{\gamma}$.

Наложение изображений

Изображения накладывают друг на друга, чтобы в результате получить некоторую композицию. Результат зависит от физики процесса. При описании положим, что компоненты RGB принимают значения от 0 до 1. Ряд способов наложения применяются в художественных целях и используют различные цветовые модели.

1. *Аддитивное наложение*. Физическую реализацию легко себе представить, если вообразить два проектора, формирующие разные изображения на одном экране. Яркость каждой компоненты каждого пикселя результата будет вычисляться просто как взвешенная сумма соответствующих пикселей исходных изображений: $C_{res}=aC_1+bC_2$, где a и b – яркость первого и второго проекторов; C_1 – значение каждой цветовой составляющей первого проектора; C_2 – второго. Чтобы результат не вышел за пределы допустимого, a и b в сумме должны давать

единицу, либо результат нужно нормировать. Ничто не мешает добавить ещё проекторов в модель — это только добавит количество слагаемых в сумму.

2. *Прозрачность*. Физически можно представить себе, если формировать изображение, накладывая полупрозрачные слайды друг на друга. Этот случай сложнее, так как в данном случае более яркий цвет не обязательно лучше будет пропускать излучение на просвет. Так, на слайде могут быть тёмные полупрозрачные фрагменты и белые, вовсе не прозрачные. Строго говоря, прозрачность пикселей задаётся специальным способом (например, альфа-каналом — четвёртым байтом в 32х-битном представлении изображения). В простейшем случае можно представить себе, что нижний слой не прозрачен, а прозрачность верхнего для всех пикселей одинакова и равна $0 < t < 1$. Тогда $C_{res} = C_1 t + C_2 (1 - t)$. Чем больше прозрачность или коэффициент пропуска слайда, тем в большей степени на результат будет влиять нижний слой и, соответственно, в меньшей степени — верхний. Единственное разнообразие, которое можно здесь внести, — разные коэффициенты пропуска для разных каналов для моделирования цветных плёнок слайдов. Ничто не мешает последовательно добавить ещё несколько слайдов.

3. *Разность*. Результат в принципе сходен с суммой позитивного и негативного изображения. На некоторых видах картинок даёт интересный эффект. Например, при наблюдении неподвижной сцены с движущимся объектом на разностном изображении двух последовательно снятых кадров будут заметны только контуры движущегося объекта. Похожий способ детектирования движения реализован в человеческом зрении. Как вариант можно взять разность по модулю. Результат будет заметно иным.

4. *Затемнение*. В каждом канале из точек двух изображений в качестве результата берётся та, что темнее.

5. *Умножение*. В качестве результата в каждой точке берётся произведение яркости каждого канала двух исходных изображений.

6. *Затемнение основы*: $C_{res} = 1 - (1 - C_1) / C_2$ для каждого цветового канала.

7. *Линейный затемнитель*: $C_{res} = C_1 + C_2 - 1$ для каждого цветового канала.

8. *Осветление*. В каждом канале из точек двух изображений в качестве результата берётся та, что светлее.

9. *Экранирование*: $C_{res}=1-((1-C_1)(1-C_2))$ для каждого цветового канала.

10. *Осветление основы*: $C_{res}=C_1/(1-C_2)$ для каждого цветового канала.

11. *Линейный осветлитель*: $C_{res}=C_1+C_2$ для каждого цветового канала.

12. *Перекрытие*. Если верхний слой темнее 50%, выполняется «Умножение», иначе – «Экранирование».

13. *Мягкий свет*. Если верхний слой темнее 50%, выполняется «Затемнение», иначе – «Экранирование».

14. *Жёсткий свет*. Если верхний слой темнее 50%, выполняется «Умножение», если светлее 50%, выполняется «Осветление».

15. *Яркий свет*. Если верхний слой темнее 50%, выполняется «Затемнение основы», если светлее 50%, выполняется «Осветление основы».

16. *Линейный свет*. Если верхний слой темнее 50%, выполняется «Линейный затемнитель», если светлее 50%, выполняется «Линейный осветлитель».

17. *Точечный свет*. Если верхний слой темнее 50%, выполняется «Затемнение», иначе – «Осветление».

18. *Жёсткое смешение*. Для каждого пикселя суммируются значения по каждому из каналов RGB, затем, если сумма по каналу превысила 1, канал в результате получает значение 1, иначе – 0.

19. *Цветовой тон*. Результат составляется из цветового тона верхнего слоя и насыщенности и яркости нижнего.

20. *Насыщенность*. Результат составляется из насыщенности верхнего слоя и цветового тона и яркости нижнего.

21. *Цветность*. Результат составляется из цветового тона и насыщенности верхнего слоя с яркости нижнего.

Задание

1. Разработать алгоритм и программу для чтения из файла, преобразования и записи в файл графической информации по варианту. Программа включает в себя:

- чтение пары изображений из файла;
- попиксельная обработка изображения;
- наложение одного изображения на другое;

- сохранение созданных изображений в файл на диске.
- 2. Ответить на контрольные вопросы.
- 3. Оформить отчёт.

Содержание отчёта

1. Титульный лист.
2. Формулировка задачи.
3. Алгоритм.
4. Программа на алгоритмическом языке программирования.
5. Форма.
6. Исходное и преобразованное изображения.
7. Ответы на контрольные вопросы.

Порядок выполнения работы

1. Ознакомление с целью работы, заданием, содержанием отчёта, порядком выполнения работы, индивидуальным вариантом, примером выполнения работы, примером оформления отчёта.

2. Ознакомление со средствами системы программирования для чтения, обработки и записи графики на примере и с помощью технической документации.

3. Ознакомление с теоретическими материалами по теме.

4. Разработка алгоритма в соответствии с вариантом задания. Алгоритм должен предусматривать:

- чтение файла изображения;
 - преобразование в подходящую цветовую модель;
 - преобразование изображения для достижения указанного результата;
 - наложение одного изображения на другое по заданному правилу;
 - запись преобразованного изображения в файл.
5. Разработка и отладка программы для решения задачи.
6. Ответы на контрольные вопросы.
7. Оформление отчёта.

Индивидуальные задания

Каждое индивидуальное задание (табл. 3) включает в себя преобразование изображения и наложение изображений.

Таблица 3

Варианты индивидуальных заданий

№	Преобразование	Наложение
1	Увеличить на 30% насыщенность точек, яркость которых превышает 50%	Аддитивное наложение
2	Увеличить на 30% насыщенность точек, яркость которых меньше 50%	Прозрачность
3	Увеличить яркостной контраст до максимально возможного	Разность
4	Увеличить контраст по насыщенности до максимально возможного	Затемнение
5	Увеличить до максимума контраст точек, яркость которых превышает 20%	Умножение
6	Преобразовать в негатив точки, яркость которых меньше 50%	Затемнение основы
7	Преобразовать в ч/б точки, насыщенность которых меньше 75%	Линейный затемнитель
8	Отрегулировать баланс белого по блику	Осветление
9	Отрегулировать баланс белого методом серого мира	Экранирование
10	Постеризовать изображение по цветовому каналу	Осветление основы
11	Выполнить гамма-коррекцию яркости	Линейный осветлитель
12	Выполнить логарифмирование яркости	Перекрытие
13	Записать в канал насыщенности синус тона	Мягкий свет
14	Обесцветить на 50%	Жёсткий свет
15	Постеризовать по интенсивности	Яркий свет

№	Преобразование	Наложение
16	Бинаризовать по насыщенности	Линейный свет
17	Убрать синий компонент у точек, яркость которых больше 75%	Точечный свет
18	Увеличить на 100% красный компонент у точек, яркость которых меньше 25%	Жёсткое смешение
19	Уменьшить контраст так, чтобы яркость изображения находилась в пределах от 50% до 90% возможного диапазона	Цветовой тон
20	Плавно уменьшить насыщенность точек до 0 в направлении сверху вниз	Насыщенность

Контрольные вопросы

1. Что такое цвет?
2. Каковы основные параметры цвета?
3. В чём преимущества цветковых моделей разного типа?
4. Как спектр перевести в цвет и наоборот?
5. Как изменить цветовую температуру изображения?
6. Зачем баланс белого?
7. Как описать функцию яркости?

Пример выполнения работы

0	Всё, что отличается от наиболее близкого из присутствующих на картинке к атому более чем на заданный процент, сделать ч/б	Цветность
---	---	-----------

Отличие в цвете будем вычислять по формуле CIE94, чтобы не усложнять. Для этого изображение должно быть представлено с использованием цветовой модели LCh. Алым в координатах RGB: (255,36,0). Режим наложения означает, что яркость L пикселя верхнего слоя совмещается с цветностью C и тоном h нижнего. Этот режим тоже легко реализовать, используя LCh.

На форме (рис. 5) предусматриваем 6 объектов Tbutton — по кнопке для считывания каждого из двух изображений, обработки и сохранения каждого из двух результатов, диалоги TopenPictureDialog и TsavePictureDialog для цивилизованного чтения файлов, 4 объекта Tgraphics для хранения и отображения картинок, 1 Tedit для задавания процента близости, 2 компонента TLabel для вывода размера картинки.

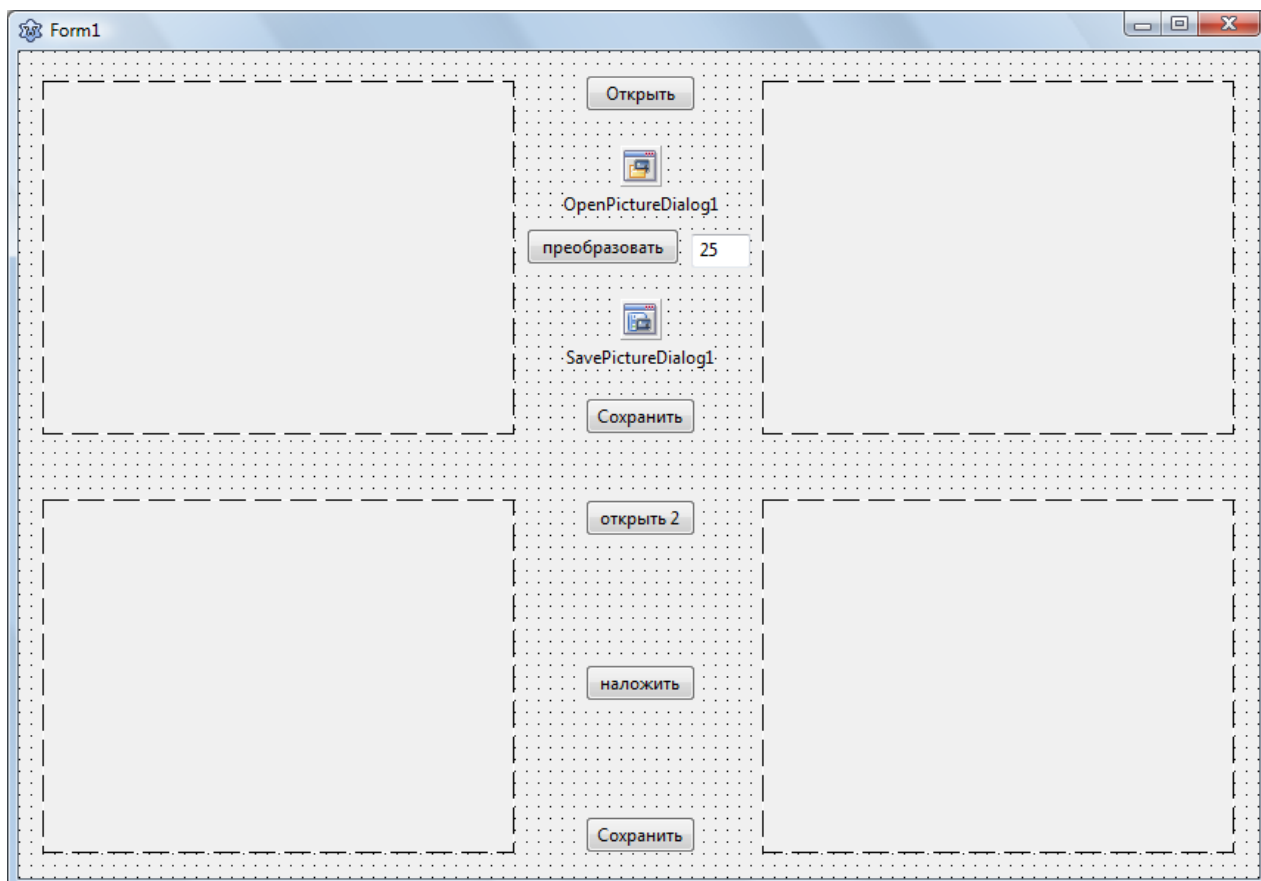


Рис. 5. Форма программы для попиксельной обработки

Алгоритм преобразования:

1. Загрузить изображение в Timage.
2. Записать RGB-составляющие каждого пикселя изображения в двумерные массивы sr, sg, sb, каждый массив размером с изображение.
3. Преобразовать в LCh координаты алого цвета из RGB.
4. Преобразовать из RGB и записать LCh-компоненты каждого пикселя изображения в двумерные массивы h, c, l, каждый массив размером с изображение.

5. Найти координаты $i_{\min}$ и $j_{\min}$ точки изображения, цветовое расстояние цвета которой до заданного алого цвета минимально. Также запомнить минимальное $\min d$ и максимальное $\max d$ значение цветового расстояния до заданного алого.

6. Считать из $Tedit$ процент отличия от алого цвета.

7. Вычислить значение cd цветового расстояния до наиболее близкого к алому цвета как введенный процент разницы между наиболее близким к алому и наиболее далёким от алого среди присутствующих на изображении цветов.

8. Для каждого пикселя изображения, если его цвет отличается от наиболее алого на изображении не более, чем на cd , записать в массивы rr , rg , rb , представляющие результат, RGB-компоненты пикселя исходного изображения, в противном случае записать в результат монохромное значение.

9. Вывести результат на форму.

10. Сохранить результат в файл.

Алгоритм наложения.

1. Загрузить первое изображение в $Timage$, если оно ещё не загружено.

2. Загрузить второе изображение в другой $Timage$.

3. Найти ширину w_i и высоту h_e результата как минимальные значения ширины и высоты считанных изображений.

4. Преобразовать из RGB и записать HSV-компоненты каждого пикселя каждого изображения в двумерные массивы: $h1$, $s1$, $v1$, $h2$, $s2$, $v2$, каждый массив — размером w_i на h_e .

5. Преобразовать из HSV и записать RGB-компоненты в массивы $rr2$, $rg2$, $rb2$ результата, причём для каждого преобразуемого пикселя компоненты тона и насыщенности H и S брать из массивов $h2$ и $s2$, соответствующих второму изображению, а компонент яркости V брать из массива $v1$, соответствующего первому изображению.

6. Вывести результат на форму.

7. Сохранить результат в файл.

Получаем модуль приблизительно следующего содержания:

```
unit Unit1;

{$mode objfpc}{$H+}

interface

uses
  Classes, SysUtils, FileUtil, Forms, Controls, Graphics, Dialogs, StdCtrls,
  GraphType, IntfGraphics, LCLType, LCLProc, LCLIntf, ExtCtrls, ExtDlgs,
  fpImage, math, crt;

type

  { TForm1 }
  TA = Array of array of real; //тип двумерного массива.
  TForm1 = class(TForm)
    Button1: TButton;
    Button2: TButton;
    Button3: TButton;
    Button4: TButton;
    Button5: TButton;
    Button6: TButton;
    Edit3: TEdit;
    Image1: TImage;
    Image2: TImage;
    Image4: TImage;
    Label3: TLabel;
    Label4: TLabel;
    OpenPictureDialog1: TOpenPictureDialog;
    SavePictureDialog1: TSavePictureDialog;
    t: TImage;
    procedure Button1Click(Sender: TObject);
    procedure Button2Click(Sender: TObject);
    procedure Button4Click(Sender: TObject);
    procedure Button3Click(Sender: TObject);
    procedure Button5Click(Sender: TObject);
```

```

    procedure Button6Click(Sender: TObject);
    procedure FormClose(Sender: TObject; var CloseAction: TCloseAction);
private
    { private declarations }
public
    { public declarations }
end;

var
    Form1: TForm1;
// массивы для хранения компонентов изображений:
    l,h,c,sr,sg,sb,rr,rg,rb,r2,g2,b2,s2,h2,v2,h1,s1,v1,rr2,rg2,rb2: TA;
implementation

{$R *.lfm}

{ TForm1 }

// функция преобразования изображения в формат 24 бита на пиксель.
function ConvertTo24Bit(aValue: TBitmap;BackColor: TColor): TBitmap;
begin
    Result := Tbitmap.Create; //результат делаем
    with Result do //с результатом
        if aValue.PixelFormat <> pf24bit then //если параметр не какой надо
            begin
                PixelFormat := pf24bit; //результат будет формата какого надо
                SetSize(aValue.Width,aValue.Height); //исходного размера
                Canvas.FloodFill(0,0,BackColor,fsBorder); //заполнен чем дали
                Canvas.Draw(0,0,aValue); //и нарисовано что дали
            end
        else //а если параметр какого надо формата
            Assign(AValue); //просто сразу в результат его
    end;

    //процедура записи массивов компонентов в Timage
    //с возможностью нормализации в диапазон 0..255.
    procedure
    show3TA(rta,gta,bta:TA;p:timage;w,h:integer;normalized:boolean);

```

```

var i,j:integer;minr,ming,minb,maxr,maxg,maxb:real; cr,cg,cb:byte;
begin
with p.Picture.Bitmap do //компонент Timage какой дали
begin
Clear; //чистим
PixelFormat:=pf24bit; //делаем нужного формата
Width:=w; //какой дали ширины
Height:=h; //какой дали высоты
Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder); //белым если что
if normalized then begin //если дали нормировать
minr:=rta[0,0];maxr:=rta[0,0]; //ищем минимальные
ming:=gta[0,0];maxg:=gta[0,0]; //и максимальные значения
minb:=bta[0,0];maxb:=bta[0,0]; //каждой компоненты
for j:=0 to h-1 do //в цикле по всем пикселям
for i:=0 to w-1 do //в массивах какие дали
begin
if rta[i,j]<minr then minr:=rta[i,j];
if gta[i,j]<ming then ming:=gta[i,j];
if bta[i,j]<minb then minb:=bta[i,j];
if rta[i,j]>maxr then maxr:=rta[i,j];
if gta[i,j]>maxg then maxg:=gta[i,j];
if bta[i,j]>maxb then maxb:=bta[i,j];
end;
for j:=0 to h-1 do //нормируем каждую компоненту независимо
for i:=0 to w-1 do begin //в диапазон 0..255
cr:=round((rta[i,j]-minr)*255/(maxr-minr));
cg:=round((gta[i,j]-ming)*255/(maxg-ming));
cb:=round((bta[i,j]-minb)*255/(maxb-minb));
Canvas.pixels[i,j]:=RGB(cr,cg,cb); //и выводим на холст битмапа
end;
end
else //а если дали не нормировать
for j:=0 to h-1 do //просто выводим каждый пиксель
for i:=0 to w-1 do //округляя до целого
Canvas.pixels[i,j]:=RGB(round(rta[i,j]),round(gta[i,j]),round(bta[i,j]));
end;
end;
end;

```

```
//процедура преобразования RGB->XYZ
// для D65 и RGB без гамма коррекции.
procedure rgb2xyz(var xc:real; var yc:real; var zc:real; cr,cg,cb:real);
begin
  xc:=(0.4124564*cr+0.3575761*cg+0.1804375*cb)/2.55;
  yc:=(0.2126729*cr+0.7151522*cg+0.0721749*cb)/2.55;
  zc:=(0.0193339*cr+0.1191920*cg+0.9503041*cb)/2.55;
end;
```

```
//процедура преобразования XYZ->RGB
//для D65 и RGB без гамма коррекции.
procedure xyz2rgb(var cr:real; var cg:real; var cb:real; xc,yc,zc:real);
begin
  cr:=(3.2404542*xc-1.5371385*yc-0.4985314*zc)*2.55;
  cg:=(1.8760108*yc-0.9692660*xc+0.0415560*zc)*2.55;
  cb:=(0.0556434*xc-0.2040259*yc+1.0572252*zc)*2.55;
end;
```

```
//процедура преобразования XYZ->Lab для D65.
procedure xyz2lab(var cl:real; var ca:real; var cb:real; xc,yc,zc:real);
var eps,k,xr,yr,zr,fx,fy,fz,p,q: real;
begin
  eps:=power(6/29,3);k:=4/29;p:=1/3;q:=29/6;
  xr:=xc/95.047;yr:=yc/100;zr:=zc/108.883;
  if xr>eps then fx:=power(xr,p) else fx:=sqr(q)*xr*p+k;
  if yr>eps then fy:=power(yr,p) else fy:=sqr(q)*yr*p+k;
  if zr>eps then fz:=power(zr,p) else fz:=sqr(q)*zr*p+k;
  cl:=116*(fy-k);
  ca:=500*(fx-fy);
  cb:=200*(fy-fz);
end;
```

```
//процедура преобразования Lab->XYZ для D65.
procedure lab2xyz(var xc:real; var yc:real; var zc:real; cl,ca,cb:real);
var eps,k,xr,yr,zr,fx,fy,fz: real;
begin
  eps:=6/29;k:=4/29;
  fy:=(cl+16)/116;fx:=ca/500+fy;fz:=fy-cb/200;
```

```

if fx>eps then xr:=power(fx,3) else xr:=3*sqr(eps)*(fx-k);
if fy>eps then yr:=power(fy,3) else yr:=3*sqr(eps)*(fy-k);
if fz>eps then zr:=power(fz,3) else zr:=3*sqr(eps)*(fz-k);
xc:=xr*95.047;yc:=yr*100;zr:=zr*108.883;
end;

```

```

//процедура преобразования Lab->LCh.
procedure lab2LCH(var lc:real; var cc:real; var hc:real; cl,ca,cb:real);
begin
lc:=cl;
cc:=sqrt(sqr(ca)+sqr(cb));
hc:=arctan2(cb,ca)/pi*180;
if hc<0 then hc:=hc+360 else if hc>=360 then hc:=hc-360;
end;

```

```

//процедура преобразования LCh->Lab.
procedure LCH2lab(var cl:real; var ca:real; var cb:real; lc,cc,hc:real);
begin
cl:=lc;
ca:=cc*cos(hc/180*pi);
cb:=cc*sin(hc/180*pi);
end;

```

```

//процедура преобразования RGB->HSV.
procedure rgb2hsv(var hc:real; var sc:real; var vc:real; cr,cg,cb:real);
var maxa,mina:real;
begin
maxa:=max(max(cr,cg),cb);mina:=min(min(cr,cg),cb);
vc:=maxa/255;
if maxa=0 then sc:=0 else sc:=1-mina/maxa;
if maxa=mina then hc:=0
else if maxa=cr then
begin
hc:=60*(cg-cb)/(maxa-mina);
if cg<cb then hc:=hc+360;
end
else
begin

```

```

    if maxa=cg then
        hc:=60*(cb-cr)/(maxa-mina)+120
    else
        hc:=60*(cr-cg)/(maxa-mina)+240;
    end;
end;

```

//процедура преобразования HSV->RGB.

```

procedure hsv2rgb(var cr:real; var cg:real; var cb:real; hc,sc,vc:real);
var hi:byte;f,p,q,tt:real;
begin
    hi:=round(int(hc/60))mod 6;
    f:=hc/60-int(hc/60);
    p:=vc*(1-sc);
    q:=vc*(1-f*sc);
    tt:=vc*(1-(1-f)*sc);
    case hi of
        0: begin cr:=vc;cg:=tt;cb:=p; end;
        1: begin cr:=q;cg:=vc;cb:=p; end;
        2: begin cr:=p;cg:=vc;cb:=tt; end;
        3: begin cr:=p;cg:=q;cb:=vc; end;
        4: begin cr:=tt;cg:=p;cb:=vc; end;
        5: begin cr:=vc;cg:=p;cb:=q; end;
    end;
    cr:=cr*255;cg:=cg*255;cb:=cb*255;
end;

```

//Функция вычисления цветового расстояния CIE94

//на основе компонентов Lch.

```

function cie94cd(h1,c1,l1,h2,c2,l2,kl:real):real;
var s1,s2,s3,k1,k2:real;
begin
    if kl=1 then begin k1:=0.045; k2:=0.015; end
    else begin k1:=0.048; k2:=0.014 end;
    s1:=(l2-l1)/kl; s2:=(c2-c1)/(1+k1*c1); s3:=(h2-h1)/(1+k2*c1);
    result:=sqrt(sqr(s1)+sqr(s2)+sqr(s3));
end;

```

```

//функция преобразования в монохром.
function rgb2bw(cr,cg,cb:real):real;
begin
    result:=0.2125*cr+0.7154*cg+0.0721*cb;
end;

//чтение первого изображения
procedure TForm1.Button1Click(Sender: TObject);
var i,j:integer;
begin
    if OpenPictureDialog1.Execute then //запускаем диалог
        t.Picture.LoadFromFile(OpenPictureDialog1.FileName); //читаем файл
    Label3.Caption:=IntToStr(t.Picture.Bitmap.Width); //выводим ширину
    Label4.Caption:=IntToStr(t.Picture.bitmap.Height); //выводим высоту
    if t.Picture.bitmap.PixelFormat <> pf24bit then //если не 24бита
        begin //делаем чтобы точно был 24 бита
            showmessage('invalid pixel color format. trying to convert');
            t.picture.bitmap.assign(convertto24bit(t.Picture.bitmap,RGB(255,255,255)));
        end;
    setlength(sr,t.picture.bitmap.width,t.picture.Bitmap.height); //задаём раз-
    setlength(sg,t.picture.bitmap.width,t.picture.Bitmap.height); //мер массива
    setlength(sb,t.picture.bitmap.width,t.picture.Bitmap.height); //вов.
    with t.Picture.Bitmap do //считанное в Timage изображение
        for j:=0 to Height-1 do //строка за строкой
            for i:=0 to Width-1 do begin //пиксель за пикселем
                sr[i,j]:=getrvalue(canvas.pixels[i,j]); //записываем в
                sg[i,j]:=getgvalue(canvas.pixels[i,j]); //соответствующий массив
                sb[i,j]:=getbvalue(canvas.pixels[i,j]);
            end;
        end;
end;

//преобразование изображения
procedure TForm1.Button2Click(Sender: TObject);
var la,ca,ha,x1,y1,z1,mind,maxd,cd,bwc:real; i,j,imin,jmin:integer;
begin
    rgb2xyz(la,ca,ha,255,36,0); //преобразуем алый из RGB в XYZ
    xyz2lab(la,ca,ha,la,ca,ha); //преобразуем алый из XYZ в Lab

```

```

lab2lch(la,ca,ha,la,ca,ha); //преобразуем алый из Lab в LCh
setlength(h,t.picture.bitmap.width,t.picture.Bitmap.height); //задаём
setlength(c,t.picture.bitmap.width,t.picture.Bitmap.height); //размер
setlength(l,t.picture.bitmap.width,t.picture.Bitmap.height); //динамичес-
setlength(rr,t.picture.bitmap.width,t.picture.Bitmap.height); //ких масси-
setlength(rg,t.picture.bitmap.width,t.picture.Bitmap.height); //вов для
setlength(rb,t.picture.bitmap.width,t.picture.Bitmap.height); //компонент
for j:=0 to t.picture.Bitmap.height-1 do //в каждой строке
  for i:=0 to t.picture.bitmap.width-1 do begin //каждый пиксель
    rgb2xyz(x1,y1,z1,sr[i,j],sg[i,j],sb[i,j]); //преобразуем через XYZ
    xyz2lab(x1,y1,z1,x1,y1,z1); //через Lab
    lab2lch(l[i,j],c[i,j],h[i,j],x1,y1,z1); // в LCh
  end;
imin:=0; //инициализируем поиск позиции минимально
jmin:=0; //отличающегося от алого элемента изображения, а также
mind:=cie94cd(ha,ca,la,h[0,0],c[0,0],l[0,0],1); //минимального и
maxd:=mind; //максимального цветового расстояния до алого
for j:=0 to t.picture.Bitmap.height-1 do //для каждой строки
  for i:=0 to t.picture.bitmap.width-1 do begin //для каждого пикселя
    cd:=cie94cd(ha,ca,la,h[i,j],c[i,j],l[i,j],1); //разница с алым
    if cd<mind then begin mind:=cd; imin:=i; jmin:=j; end; //минимум
    if cd>maxd then maxd:=cd; //максимум
  end;
cd:=strtoint(edit3.text)*(maxd-mind)/100; //процент разницы в edit3
for j:=0 to t.picture.Bitmap.height-1 do //для каждой строки
  for i:=0 to t.picture.bitmap.width-1 do begin //для каждого пикселя
    if cie94cd(h[i,j],c[i,j],l[i,j],h[imin,jmin],c[imin,jmin],l[imin,jmin],1)<cd
then //если расстояние до самого алого элемента изображения меньше
  begin
    rr[i,j]:=sr[i,j]; //записываем в результат
    rg[i,j]:=sg[i,j]; //исходные RGB цвета
    rb[i,j]:=sb[i,j];
  end
else //а если больше
  begin
    bwc:=rgb2bw(sr[i,j],sg[i,j],sb[i,j]); //преобразуем в чёрно-белое
    rr[i,j]:=bwc;rg[i,j]:=bwc;rb[i,j]:=bwc; //записываем его в результат
  end;

```

```

    end;
show3ta(rr,rg,rb,image4,t.Picture.Bitmap.width,t.Picture.Bit-
map.height,false);//результат выводим на форму в компонент Image4 без
нормировки
end;

//процедура считывания второго изображения
procedure TForm1.Button4Click(Sender: TObject);
var i,j:integer;
begin
    if OpenPictureDialog1.Execute then //запускаем диалог и читаем файл
        image1.Picture.LoadFromFile(OpenPictureDialog1.FileName);
    if image1.Picture.bitmap.PixelFormat <> pf24bit then //если не формат
        begin
            showmessage('invalid pixel color format. trying to convert');//пугаемся
            image1.picture.bitmap.assign(convertto24bit(image1.Picture.bit-
map,RGB(255,255,255))); //и конвертируем
        end;
    with image1.Picture.Bitmap do begin //со считанным изображением
        setlength(r2,width,height); //задаём размеры динамических массивов
        setlength(g2,width,height); //для хранения компонентов RGB
        setlength(b2,width,height); //
        for j:=0 to Height-1 do //для каждой строки
            for i:=0 to Width-1 do begin //для каждого пикселя
                r2[i,j]:=getrvalue(canvas.pixels[i,j]); //пишем в массивы
                g2[i,j]:=getgvalue(canvas.pixels[i,j]); //компоненты RGB
                b2[i,j]:=getbvalue(canvas.pixels[i,j]);
            end;
        end;
    end;
end;

//процедура наложения
procedure TForm1.Button5Click(Sender: TObject);
var i,j,he,wi:integer;
begin
    he:=min(image1.picture.Bitmap.height,t.picture.Bitmap.height);//высота
    wi:=min(image1.picture.Bitmap.width,t.picture.Bitmap.width);//ширина
    setlength(rr2,wi,he); setlength(rg2,wi,he); setlength(rb2,wi,he);//задаём

```

```

setlength(h2,wi,he); setlength(s2,wi,he); setlength(v2,wi,he); //размеры
setlength(h1,wi,he); setlength(s1,wi,he); setlength(v1,wi,he); //9 массивов
for j:=0 to he-1 do //для каждой строки
  for i:=0 to wi-1 do begin //для каждого пикселя
    rgb2hsv(h1[i,j],s1[i,j],v1[i,j],sr[i,j],sg[i,j],sb[i,j]); // 1 из RGB в HSV
    rgb2hsv(h2[i,j],s2[i,j],v2[i,j],r2[i,j],g2[i,j],b2[i,j]); //2 из RGB в HSV
    hsv2rgb(rr2[i,j],rg2[i,j],rb2[i,j],h2[i,j],s2[i,j],v1[i,j]); //результат в RGB
  end;
show3ta(rr2,rg2,rb2,image2,wi,he,false); //выв. в image2 без нормировки
end;

//сохранение результата обработки
procedure TForm1.Button3Click(Sender: TObject);
begin
  if SavePictureDialog1.Execute then
    Image2.Picture.SaveToFile(SavePictureDialog1.FileName);
end;

//сохранение результата наложения
procedure TForm1.Button6Click(Sender: TObject);
begin
  if SavePictureDialog1.Execute then
    Image4.Picture.SaveToFile(SavePictureDialog1.FileName);
end;

//с закрытием формы всё ликвидируем
procedure TForm1.FormClose(Sender: TObject; var CloseAction: TClose-
Action);
begin
  t.Free;
  image1.free;
  image2.free;
  image4.Free;
end;

end.

```

Форма с результатами (рис. 6):

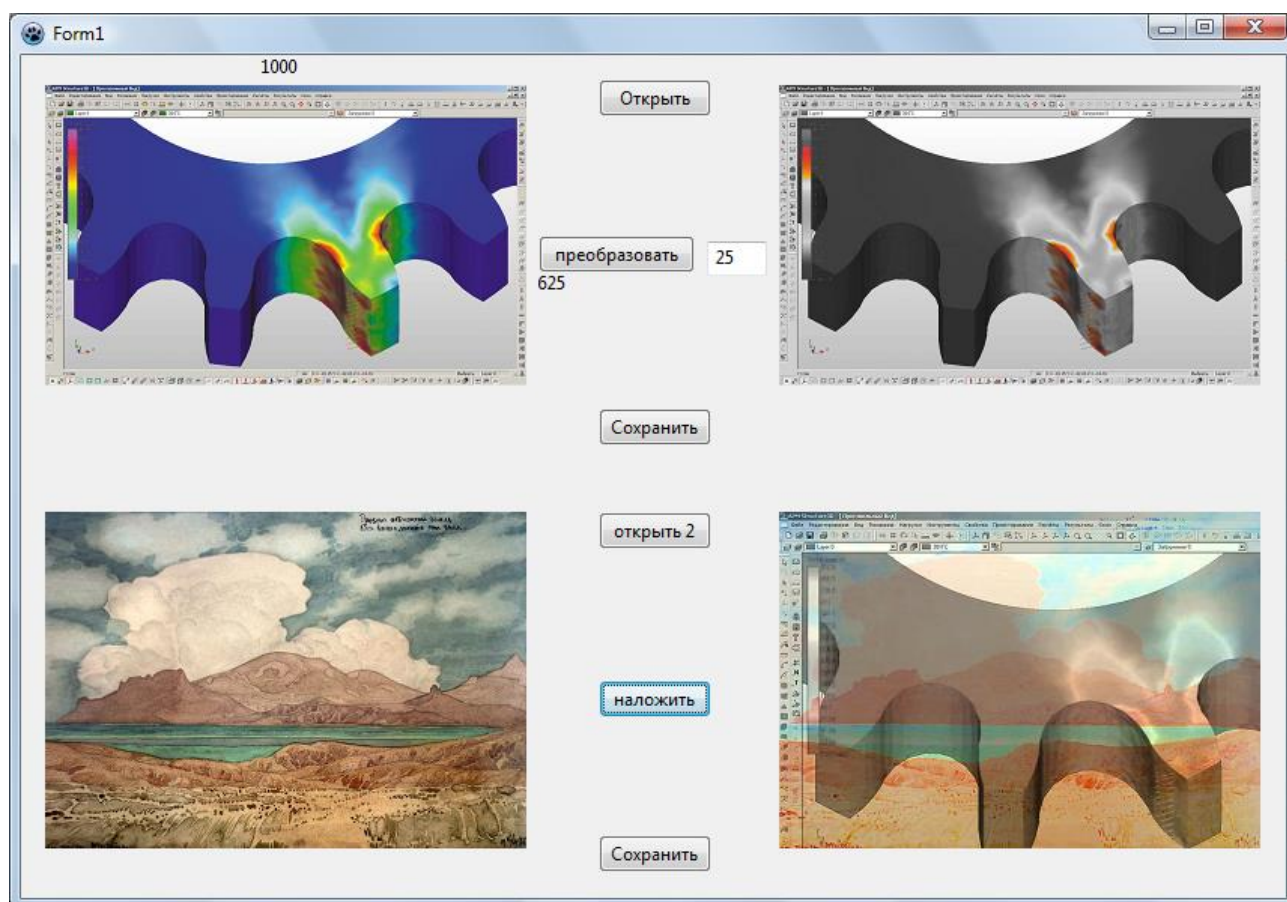


Рис. 6. Результат работы программы для попиксельной обработки

Лабораторная работа № 6.

Геометрические преобразования

Цель работы – освоение навыков обработки растровых изображений с помощью алгоритмических языков программирования путём преобразования координат пикселей.

Краткие теоретические сведения

Положение объектов сцены в 3-мерном евклидовом пространстве может быть задано с помощью 3-х пространственных координат. Функция яркости изображения, воспринимаемого камерой, будет определяться светимостью каждой видимой с позиции камеры точки объекта сцены в направлении камеры. Каждая из этих точек определяется тремя координатами, а функция яркости будет функцией трёх переменных. В связи с особенностями человеческого зрения, состоящими в том, что функция яркости сцены воспринимается практически из одной точки пространства в пределах ограниченного телесного угла, формирующего пирамиду видимости, оптическое изображение 3-мерной сцены можно представить в виде центральной перспективной проекции функции яркости на плоскость, перпендикулярную главной оптической оси. В рамках этой проекционной плоскости любая точка, соответствующая видимой точке сцены, будет задана двумя координатами, а функция яркости физически будет описывать величину потока через данную точку в направлении от видимой точки объекта сцены к камере. Эта величина будет зависеть от светимости точки объекта сцены и расстояния от этой точки до камеры. Преобразование трёхмерных координат объектов сцены в двумерные координаты проекционной плоскости называются проективным преобразованием.

Проективные преобразования

Если координаты трёхмерного объекта задавать в системе координат, начало которой совпадает с точкой, из которой производится наблюдение и которая, соответственно, является центром проекции, главную оптическую ось совместить с OZ , а проекционную плоскость расположить параллельно XOY в $z=const$, то координаты проекции точки объекта сцены (x_1, y_1, z_1) на проекционной плоскости будут (x'_1, y'_1)

(рис. 1). Координаты проекции вычисляются с помощью проективных преобразований: $x'_1 = \frac{const \cdot x_1}{z_1}$, $y'_1 = \frac{const \cdot y_1}{z_1}$. Если проекционную плоскость расположить в $z=1$, координаты изображения точки на плоскости будут $x'_1 = \frac{x_1}{z_1}$, $y'_1 = \frac{y_1}{z_1}$.

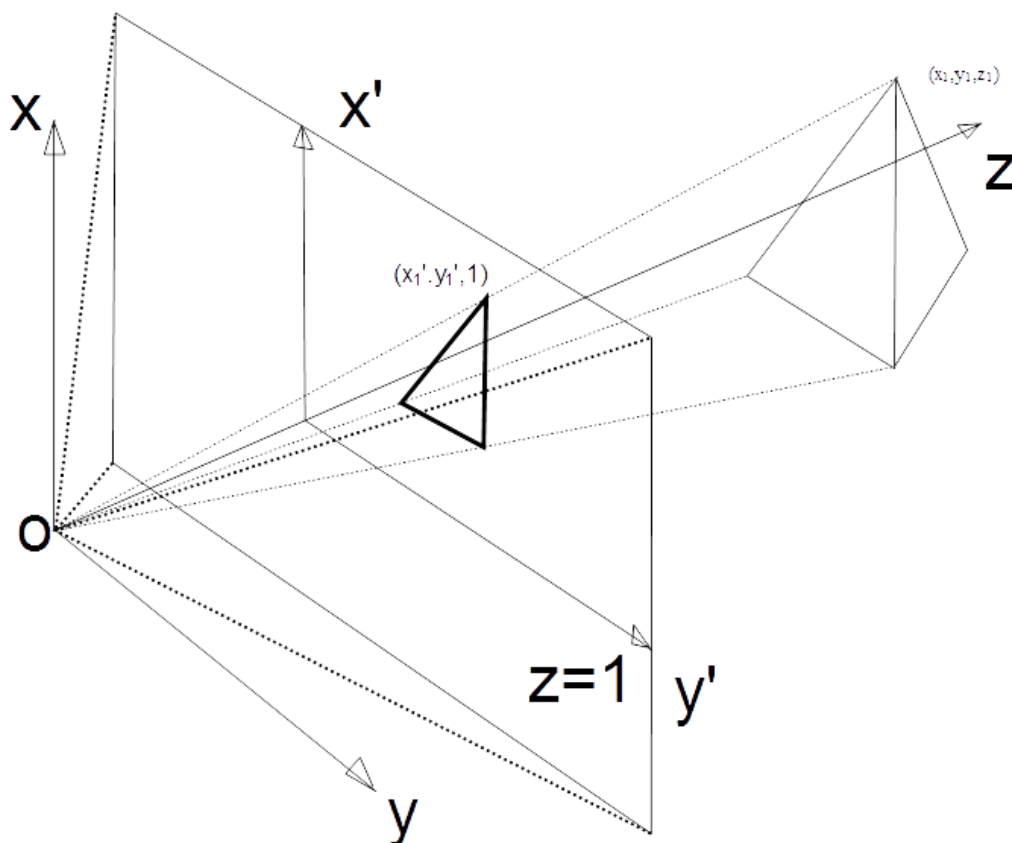


Рис. 1. Построение проекции

Аффинные преобразования

При использовании камеры в конечном счёте функция яркости регистрируется как освещённость чувствительной поверхности фотоприёмника, на которую проецируется оптическое излучение объектов сцены. Освещённость зависит от потока через проекционную плоскость, но координаты функции яркости в плоскости фотоприёмника не обязательно будут соответствовать координатам на проекционной плоскости в связи с тем, что фотоприёмник может быть сдвинут, повернут в пространстве и удалён относительно системы координат проекционной плоскости и главной оптической оси (рис. 2). Для приведе-

ния в соответствие выполняются преобразования координат, обеспечивающих перенос, поворот, наклон, масштабирование и отражение изображения. Преобразования такого рода называются аффинными и обладают рядом свойств: любое аффинное преобразование имеет обратное, которое также является аффинным. Произведение прямого и обратного преобразований даёт единичное преобразование, оставляющее все на месте. Аффинное преобразование является самым общим взаимно однозначным отображением плоскости на плоскость, при котором сохраняются прямые линии. Сохраняются также отношения длин отрезков, лежащих на одной прямой (или на параллельных прямых), и отношения площадей фигур. Параллельные прямые переходят в параллельные.

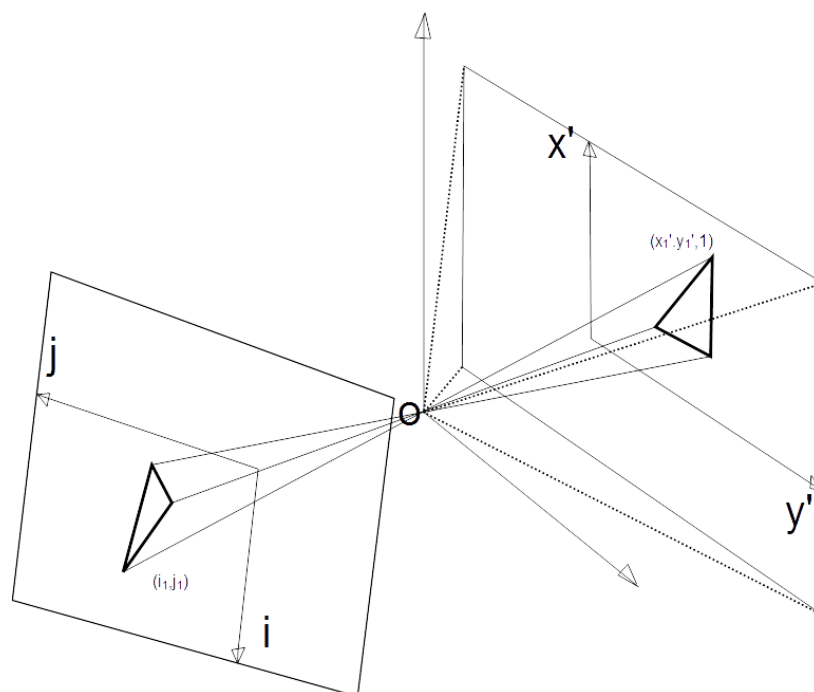


Рис. 2. Возникновение аффинных искажений

Преобразования координат линейны и описываются полиномами первой степени: $i = ax' + by' + c$, $j = dx' + ey' + f$, где i, j – новые координаты; x', y' – старые координаты; a, b, c, d, e, f – параметры преобразования.

Для определения параметров произвольного аффинного преобразования необходимо решить систему из 6 линейных уравнений. Для этого надо задать координаты трёх точек исходного изображения и координаты трёх точек, в которые эти исходные точки должны перейти при преобразовании.

Некоторые частные случаи аффинных преобразований:

- масштабирование по осям: $i=ax', j=ey'$;
- поворот на угол φ : $i=x'\cos \varphi - y'\sin \varphi, j=x'\sin \varphi + y'\cos \varphi$;
- перенос: $i=x'+c, j=y'+f$;
- скос по x : $i=x'+by', j=y'$, по y : $i=x', j=dx'+y'$;
- отражение: $i=x', j=-y'$ и $i=-x', j=y'$.

Зная параметры аффинного преобразования, можно вычислить непосредственно и параметры обратного преобразования: $x'=a^{-1}i+b^{-1}j+c^{-1}$, $y'=d^{-1}i+e^{-1}j+f^{-1}$, $\det=ae-bd$, $a^{-1}=e/\det$, $b^{-1}=-b/\det$, $c^{-1}=(bf-ec)/\det$, $d^{-1}=-d/\det$, $e^{-1}=a/\det$, $f^{-1}=(cd-af)/\det$.

Полиномиальные преобразования

Помимо аффинных геометрических искажений в изображение на поверхности фотоприёмника вносятся искажения, вызванные несовершенством реальных оптических систем и среды распространения оптического излучения. Искажения оптических систем, или аберрации, описываются полиномами более высоких степеней, чем аффинные, и не являются линейными. Например, дисторсия (рис. 3): $i=c_1+x'+k_1(x'^2+y'^2)$, $j=c_2+x'+k_2(x'^2+y'^2)$.

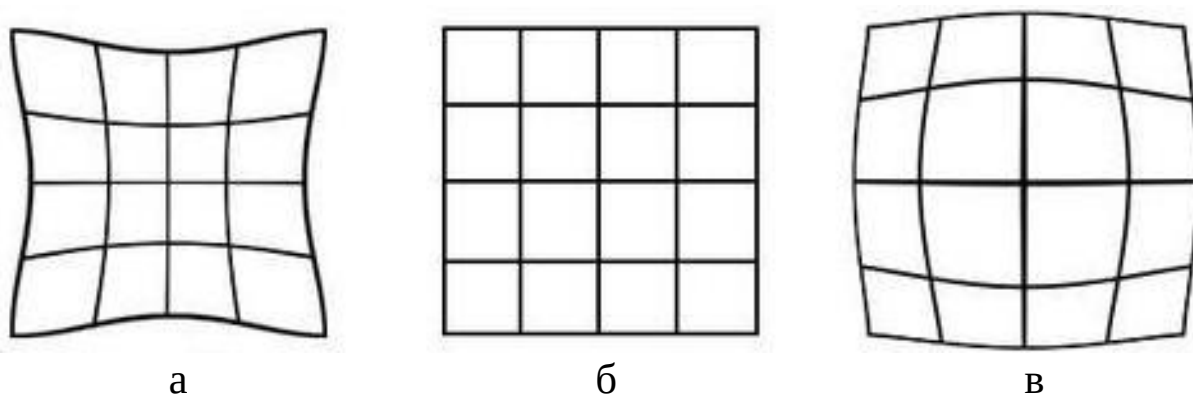


Рис. 3. Дисторсия: а – подушкообразная; б – отсутствует; в – бочкообразная

В описании дисторсии всего четыре параметра: c_1, k_1, c_2, k_2 , для получения их значений нужно взять координаты двух точек на исходном и искажённом изображении и решить систему из четырёх линейных уравнений. Произвольный полином второй степени описывается двенадцатью параметрами: $i=a_{20}x'^2+a_{02}y'^2+a_{11}x'y'+a_{10}x'+a_{01}y'+a_{00}$,

$j=b_{20}x'^2+b_{02}y'^2+b_{11}x'y'+b_{10}x'+b_{01}y'+b_{00}$. Для их определения по искажённой картинке следует определить координаты пар шести точек и решить систему из двенадцати линейных уравнений.

Степень полинома можно и повысить. Количество точек для оценки параметров полинома зависит от количества ненулевых коэффициентов. Для оценки большого количества параметров стараются взять избыточное количество точек и вычислить значения параметров статистическими методами, например методом наименьших квадратов.

Функциональные преобразования

Изображение может быть геометрически преобразовано с помощью любых функций для достижения требуемого эффекта.

Поскольку цифровое изображение дискретно и задано в изолированных узлах, при выполнении преобразования координат на некоторые точки изображения результата может не попасть ни одна точка с исходного изображения. Чтобы избежать пропусков, при обработке цифровых изображений для функции преобразования координат находят обратную функцию, а затем с помощью обратной функции для каждой точки изображения результата определяют значение яркости соответствующей точки исходного изображения.

Например, если $i=x'^2$, то, рассматривая пиксели с координатой $x'=1$ на исходном изображении, будем заполнять пиксели $i=1$ на преобразованном изображении. Переход к следующему пикселю с $x'=2$, приведёт к заполнению пикселей с координатами $i=x'^2=4$ на преобразованном изображении, при этом пиксели с координатами $i=2$ и $i=3$ будут пропущены. Чтобы избежать пропусков, следует найти обратную функцию $x'=i^{1/2}$, перебирать все координаты i на изображении результата и заполнять их пикселями с соответствующими координатами на исходном изображении.

Задание

1. Разработать алгоритм и программу для реализации индивидуального задания. Программа должна выполнять:

- загрузку изображения из файла;
- прямые и обратные аффинные преобразования;
- функциональное преобразование координат;

- сохранение результата в файл.
- 2. Ответить на контрольные вопросы.
- 3. Оформить отчёт.

Содержание отчёта

1. Титульный лист.
2. Формулировка задачи.
3. Алгоритм.
4. Программа на алгоритмическом языке программирования.
5. Исходное и преобразованное изображения.
6. Ответы на контрольные вопросы.

Порядок выполнения работы

1. Ознакомление с целью работы, заданием, содержанием отчёта порядком выполнения работы, индивидуальным вариантом, примером выполнения работы, примером оформления отчёта.
2. Ознакомление с теоретическими материалами по теме.
3. Разработка алгоритма в соответствии с вариантом задания. Алгоритм должен предусматривать:
 - чтение файла изображения;
 - аффинные преобразования изображения указанным способом;
 - восстановление исходного изображения из преобразованного путём обратного преобразования;
 - нелинейные преобразования указанным способом;
 - запись преобразованного изображения в файл.
4. Разработка и отладка программы для решения задачи.
5. Ответы на контрольные вопросы.
6. Оформление отчёта.

Индивидуальные задания

Каждый вариант задания (табл. 1) включает в себя два аффинных преобразования и функциональное преобразование координат, выраженное нелинейной зависимостью.

Таблица 1

Варианты индивидуальных заданий

№	Аффинные преобразования	Функция преобразования
1	Масштабирование, поворот	$i = \frac{x'^3}{1+x'^2}, j = y'$
2	Масштабирование, перенос	$i = 2^{x'}, j = y'$
3	Масштабирование, скос	$i = x'^{\frac{3}{5}}, j = y'$
4	Масштабирование, отражение	$i = \exp(x'), j = y'$
5	Поворот, масштабирование	$i = \sinh(x'), j = y'$
6	Поворот, перенос	$i = \operatorname{asinh}(x'), j = y'$
7	Поворот, скос	$i = \sqrt{\exp(x')}, j = y'$
8	Поворот, отражение	$i = x'^{\frac{5}{3}}, j = y'$
9	Перенос, масштабирование	$i = \frac{1}{\exp(-x')}, j = y'$
10	Перенос, поворот	$i = x'^{\frac{3}{7}}, j = y'$
11	Перенос, скос	$i = x'^{\frac{7}{3}}, j = y'$
12	Перенос, отражение	$i = \frac{x'^2}{1+x'}, j = y'$
13	Скос, масштабирование	$i = \cosh(x') - 1, j = y'$
14	Скос, поворот	$i = \operatorname{acosh}(x' + 1), j = y'$
15	Скос, перенос	$i = \log_2(x'), j = y'$
16	Скос, отражение	$i = \ln(x'), j = y'$
17	Отражение, масштабирование	$i = \sqrt{x'}, j = y'$
18	Отражение, поворот	$i = x' \cdot y', j = y'$
19	Отражение, перенос	$i = x' - \exp(-y'), j = y'$
20	Отражение, скос	$i = \frac{x'^2}{y'}, j = y'$

Контрольные вопросы

1. Какие преобразования координат происходят в процессе формирования на поверхности фотоприёмника двумерного изображения трёхмерной сцены?
2. Где физически находится центр проекции при формировании цифрового изображения?
3. Какие из преобразований необратимы и почему?
4. Какие эффекты фотошопа реализованы путём преобразования координат?
5. Какие искажения наблюдаются при выполнении обратного аффинного преобразования в этой работе и почему?

Пример выполнения работы

0	Повернуть изображение относительно центра на заданный угол, уменьшить вдвое и записать в центр результата	$i = x' + 20 \sin\left(\frac{2\pi y'}{128}\right), j = y'$
---	---	--

На форме предусматриваем по кнопке и диалогу для считывания изображения и сохранения каждого из трёх результатов, 4 объекта Tgraphics для хранения и отображения картинок, кнопки для трёх преобразований, поле ввода для угла поворота.

Последовательность действий для преобразования:

1. Загрузка изображения, преобразование в полноцветное.
2. Инициализация нового изображения с теми же параметрами, что и исходное.
3. Ввод угла поворота ϕ , преобразование в радианы: $\phi = \pi * \phi / 180$
4. Обозначим i, j – координаты пикселей на новом изображении, $i1, j1$ – на исходном. Так как координаты являются целыми числами, там, где необходимо, будем использовать целочисленное деление $\div$ и округление результата round . Для поворота относительно центра надо выполнить перенос в центр: $i = i1 - \text{width} \div 2, j = j1 - \text{height} \div 2$. Выполнение поворота: $i = i1 * \cos(\phi) - j1 * \sin(\phi), j = i1 * \sin(\phi) + j1 * \cos(\phi)$. Выполнение масштабирования: $i = i1 \div 2, j = j1 \div 2$. Выполнение переноса результата в центр: $i = i1 - \text{width} \div 2, j = j1 - \text{height} \div 2$. Все преобразования можно совместить и записать в обратном виде:

```
i1=round((((i-width/2)*cos(phi))-((j-height/2)*sin(phi)))*2+width/2),
j1=round((((i-width/2)*sin(phi))+((j-height/2)*cos(phi)))*2+height/2).
```

Для всех (i,j) результата вычисляем (i1,j1) и, проверяя на попадание координат в пределы изображения, копируем соответствующую точку с исходного изображения на новое.

5. Для восстановления исходного изображения строим обратную функцию и выполняем обратное аффинное преобразование:

```
i1=round((((i-width/2)*cos(-phi))-((j-height/2)*sin(-phi)))/2+width/2),
j1=round((((i-width/2)*sin(-phi))+((j-height/2)*cos(-phi)))/2+height/2).
```

6. Строим обратную функцию для нелинейного преобразования и выполняем нелинейные преобразования. Для этого выражаем x' и y' как в системе уравнений: $j1=j$, $i1=i-20*\sin(j*\pi/64)$.

7. Предусматриваем возможность записи результатов.

Получаем модуль приблизительно следующего содержания:

```
unit Unit1;
```

```
{ $mode objfpc } { $H+ }
```

```
interface
```

```
uses
```

```
Classes, SysUtils, FileUtil, Forms, Controls, Graphics, Dialogs, StdCtrls,
GraphType, IntfGraphics, LCLType, LCLProc, LCLIntf, ExtCtrls, Ex-
tDlgs, Menus,
fpImage, math, crt;
```

```
type
```

```
{ TForm1 }
```

```
TForm1 = class(TForm)
```

```
Button1: TButton;
```

```
Button2: TButton;
```

```
Button3: TButton;
```

```
Button4: TButton;
```

```
Button5: TButton;
```

```
Button6: TButton;
```

```
Button7: TButton;
```

```

Edit3: TEdit;
Image1: TImage;
Image2: TImage;
Image4: TImage;
Label15: TLabel;
Label16: TLabel;
Label17: TLabel;
Label3: TLabel;
Label4: TLabel;
OpenPictureDialog1: TOpenPictureDialog;
SavePictureDialog1: TSavePictureDialog;
t: TImage;
procedure Button1Click(Sender: TObject);
procedure Button2Click(Sender: TObject);
procedure Button3Click(Sender: TObject);
procedure Button4Click(Sender: TObject);
procedure Button5Click(Sender: TObject);
procedure Button6Click(Sender: TObject);
procedure Button7Click(Sender: TObject);
procedure FormClose(Sender: TObject);
private
  { private declarations }
public
  { public declarations }
end;

```

var

```
Form1: TForm1;
```

implementation

```
{ $R *.lfm }
```

```
{ TForm1 }
```

//из предыдущих работ

```

function ConvertTo24Bit(aValue: TBitmap;BackColor: TColor): TBitmap;
begin
  Result := TBitmap.Create;
  with Result do

```

```

if aValue.PixelFormat <> pf24bit then
begin
    PixelFormat := pf24bit;
    SetSize(aValue.Width,aValue.Height);
    Canvas.FloodFill(0,0,BackColor,fsBorder);
    Canvas.Draw(0,0,aValue);
end
else
    Assign(AValue);
end;

procedure TForm1.Button1Click(Sender: TObject); //считываем из файла.
begin
    if OpenPictureDialog1.Execute then
        t.Picture.LoadFromFile(OpenPictureDialog1.FileName);
    Label3.Caption:=IntToStr(t.Picture.Bitmap.Width);
    Label4.Caption:=IntToStr(t.Picture.bitmap.Height);
    if t.Picture.bitmap.PixelFormat <> pf24bit then
        begin
            showmessage('invalid pixel color format. trying to convert');
            t.picture.bitmap.assign(convertto24bit(t.Picture.bitmap,RGB(255,255,255)));
        end;
    end;

procedure TForm1.Button2Click(Sender: TObject);
var i,j,i1,j1:integer;phi:real;
begin
    phi:=strtoint(edit3.text)/180*pi;//читаем угол, переводим в радианы
    with Image4.Picture.Bitmap do//результат — в Image4
        begin
            Clear;
            PixelFormat:=pf24bit;
            Width:=t.Picture.Bitmap.Width;
            Height:=t.Picture.Bitmap.Height;
            Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder);
            for i:=0 to width-1 do //все пиксели результата
                for j:=0 to height-1 do

```

```

begin //аффинные преобразования в виде обратных функций.
i1:=round(((i-width div 2)*cos(phi)-(j-height div 2)*sin(phi))*2+width div
2);
j1:=round(((i-width div 2)*sin(phi)+(j-height div 2)*cos(phi))*2+height div
2);
//проверим, не вышли ли за пределы исходного изображения
if i1>=0 then if j1 >=0 then if i1 <Width then if j1 < height then
//переносим пиксель с преобразованными координатами.
canvas.pixels[i,j]:=t.Picture.Bitmap.Canvas.Pixels[i1,j1];
end
end
end;

```

```

//процедура записи
procedure TForm1.Button3Click(Sender: TObject);
begin
if SavePictureDialog1.Execute then
Image2.Picture.SaveToFile(SavePictureDialog1.FileName);
end;

```

```

//нелинейное преобразование координат.
procedure TForm1.Button4Click(Sender: TObject);
var i,j,i1,j1:integer;
begin
with Image1.Picture.Bitmap do //результат — в Image1
begin
Clear;
PixelFormat:=pf24bit;
Width:=t.Picture.Bitmap.Width;
Height:=t.Picture.Bitmap.Height;
Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder);
for i:=0 to width-1 do //для всех пикселей результата
for j:=0 to height-1 do
begin
i1:=round(i-20*sin(j*Pi/64)); //координаты по обратной функции.
j1:=round(j);
//Попадают ли вычисленные координаты в исходное изображение.

```

```

        if ((i1>=0) and (j1 >=0) and (i1 <Width) and (j1 < height)) then
canvas.pixels[i,j]:=t.Picture.Bitmap.Canvas.Pixels[i1,j1];
        end
    end
end;

```

//обратное аффинное преобразование.

```

procedure TForm1.Button5Click(Sender: TObject);
var i,j,i1,j1:integer;phi:real;
begin
phi:=strtoint(edit3.text)/180*pi;
with Image2.Picture.Bitmap do //результат — в Image2
begin
    Clear;
    PixelFormat:=pf24bit;
    Width:=image4.Picture.Bitmap.Width;//источник — в Image4
    Height:=image4.Picture.Bitmap.Height;
    Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder);
    for i:=0 to width-1 do
        for j:=0 to height-1 do
            begin
                i1:=round((((i-width div 2)*cos(-phi)-(j-height div 2)*sin(-
phi))/2+width div 2);
                j1:=round((((i-width div 2)*sin(-phi)+(j-height div 2)*cos(-
phi))/2+height div 2);
                if i1>=0 then if j1 >=0 then if i1 <Width then if j1 < height then
canvas.pixels[i,j]:=image4.Picture.Bitmap.Canvas.Pixels[i1,j1];
                end
            end
        end
    end;
end;

```

```

procedure TForm1.Button6Click(Sender: TObject);
begin
if SavePictureDialog1.Execute then
    Image4.Picture.SaveToFile(SavePictureDialog1.FileName);
end;

```

```

procedure TForm1.Button7Click(Sender: TObject);

```

```

begin
if SavePictureDialog1.Execute then
    Image1.Picture.SaveToFile(SavePictureDialog1.FileName);
end;

procedure TForm1.FormClose(Sender: TObject);
begin
t.Free;
image1.free;
image2.free;
image4.Free;
end;

end.

```

Результат работы программы (рис. 4):

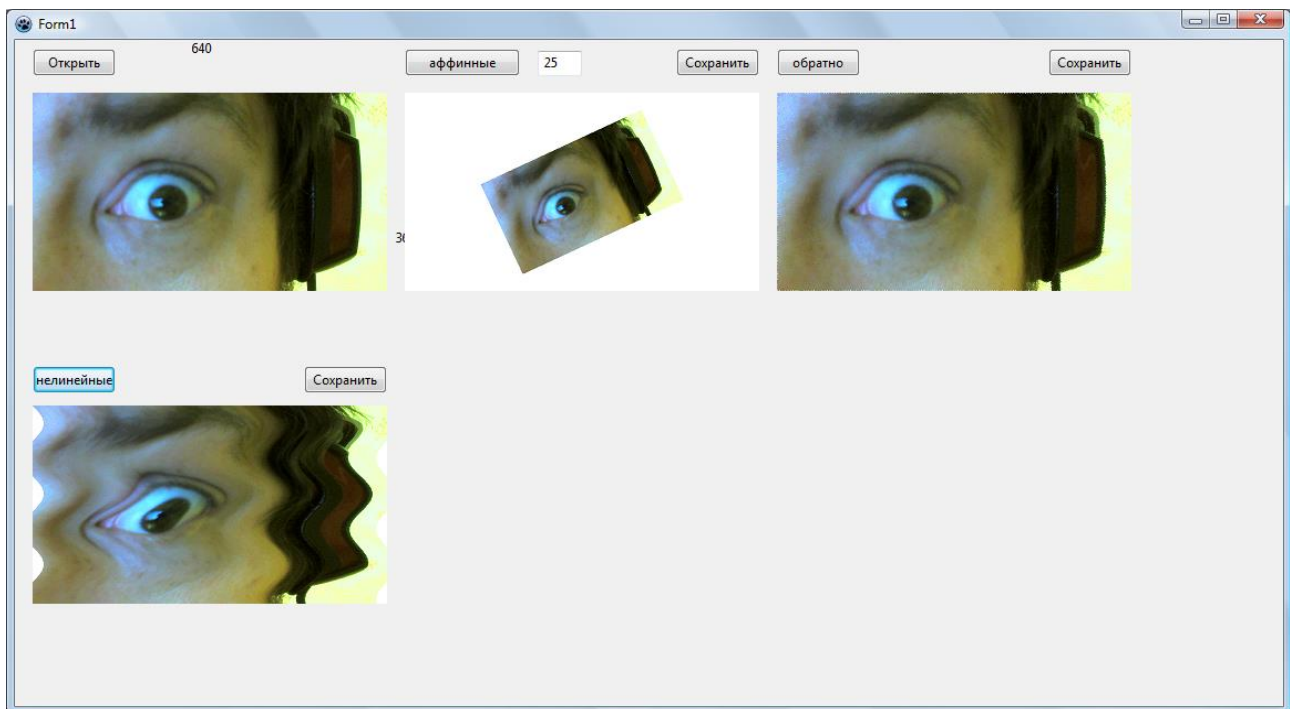


Рис. 4. Результат работы программы геометрических преобразования изображения

Лабораторная работа № 7.

Введение в фильтрацию изображений

Цель работы – освоение навыков фильтрации растровых изображений с помощью алгоритмических языков программирования.

Краткие теоретические сведения

В процессе формирования изображения оптический сигнал подвергается искажениям, которые проявляются тем, что точечный источник сцены при проецировании на поверхность фотоприёмника точкой выглядеть уже не будет, а превратится в размытое пятно. Функция яркости этого пятна $h(x',y')$ называется функцией рассеяния точки (ФРТ). Если представить сцену как совокупность точек различной яркости, то с учётом этих искажений изображение каждой из этих точек на фотоприёмнике будет размытым, а в результате и изображение сцены на фотоприёмнике как совокупности этих точек будет размытым. Характер такого рода искажений зависит от условий получения изображения и характеристик среды распространения оптического сигнала, оптической системы и датчика изображения.

Вследствие свойств оптического излучения и применяемых материалов, за исключением специальных случаев, такие преобразования относятся к линейным, так как они обладают свойствами однородности и аддитивности. Преобразование называется однородным, если изменение сигнала на входе порождает пропорциональное изменение сигнала на выходе. Преобразование называется аддитивным, если преобразование суммы сигналов равно сумме преобразований сигналов. Такие преобразования можно рассматривать как фильтрацию сигнала в рамках теории обработки сигналов. Компоненты оптической системы, обуславливающие эти искажения, могут быть описаны как фильтры. В качестве сигнала рассматривается пространственно-зависимая функция яркости $f(x,y)$. Результат линейного преобразования будет выражаться суперпозицией функции яркости сцены и функции рассеяния точки:

$$g(x',y') = \iint_{-\infty}^{\infty} f(x,y)h(x,y,x',y')dxdy,$$

где x', y' – координаты на плоскости фотоприёмника; $g(x', y')$ – функция яркости на плоскости фотоприёмника; x, y – координаты сцены,

$f(x, y)$ – функция яркости сцены; $h(x, y, x', y')$ – функция рассеяния точки, форма которой описывается как функция (x', y') и в общем случае может зависеть от положения (x, y) на сцене, другими словами, $h(x', y')$ может быть разной для точек в центре сцены и на периферии сцены, соответственно, зависеть также и от значений (x, y) . В большинстве случаев принято допускать, что это не так, преобразование является пространственно-инвариантным, функция рассеяния точки везде одинаковая и выражается как функция двух переменных, — зависимость $h(x'-x, y'-y)$ от разности координат на сцене и на изображении, а интеграл суперпозиции превращается в интеграл свёртки:

$$g(x', y') = (f * h)(x', y') = \iint_{-\infty}^{\infty} f(x, y) h(x - x', y - y') dx dy.$$

Свёртка обладает свойствами коммутативности, ассоциативности, дистрибутивности относительно сложения и умножения на константу. Дифференциал свёртки двух функций равен свёртке одной функции с дифференциалом другой.

Операция свёртки связана с преобразованием Фурье: образ Фурье результата свёртки равен произведению образов Фурье свёртываемых функций: $F\{f * h\} = F\{f\} \cdot F\{h\}$. Симметрично образ Фурье произведения функций равен свёртке образов Фурье функций: $F\{f \cdot h\} = F\{f\} * F\{h\}$.

Преобразование Фурье – это преобразование функции $f(x)$ вида

$$F(u) = F\{f(x)\} = \int_{-\infty}^{\infty} f(x) e^{-2\pi j u x} dx$$

при соблюдении условий существования интеграла. В результате преобразования получается образ Фурье $F(u)$ функции $f(x)$. Предполагается, что $f(x)$ и её образ — комплексные функции. Если $f(x)$ – действительная, то мнимая часть её значений будет везде равна нулю, но её образ Фурье будет действительным только тогда, когда $f(x)$ чётная.

Экспонента в формуле преобразования Фурье может быть выражена в виде $e^{-2\pi j u x} = \cos(2\pi u x) + j \sin(2\pi u x)$. Частота синусоиды задаётся u , а в процессе преобразования для каждого значения u измеряется интегральный вес синусоид этой частоты в сигнале $f(x)$. Синусоида каждой фиксированной частоты u называется гармоникой. Образ Фурье представляет собой функцию $F(u)$, которая показывает вес гармоники с частотой u в сигнале $f(x)$. Можно сказать, что Фурье образ представляет сигнал в виде совокупности синусоид различной частоты, каждая со своим весом. Эта совокупность называется также

спектром сигнала. Спектр содержит всю информацию о сигнале, и сигнал может быть выражен из спектра путём обратного преобразования Фурье: $f(x) = F^{-1}\{F(u)\} = \int_{-\infty}^{\infty} F(u)e^{2\pi jux} du$ с точностью до постоянного множителя.

Двумерное преобразование Фурье $F(u, v) = \iint_{-\infty}^{\infty} f(x, y)e^{-2\pi jux-2\pi jvy} dx dy$ обладает теми же свойствами, что и одномерное. Аналогично выполняется преобразование Фурье для любого количества измерений.

Вес гармоник зависит от скорости изменения сигнала. Медленные изменения сигнала соответствуют гармоникам низкой частоты, быстрые – высокой. На изображении резкий перепад яркости описывается быстрым изменением функции яркости, что добавляет веса гармоникам высокой частоты. Обычно изменения оптического сигнала такого рода возникают на контрастных границах объектов. Многие шумы на изображении также проявляются резкими скачками сигнала. Для представления плавных изменений функции яркости достаточно гармоник из ограниченного диапазона низких частот – от 0 до частоты среза $u_{гр}$.

В процессе фильтрации часть гармоник пропускается, другая часть — задерживается. Фильтры, пропускающие гармоники, частота которых находится в пределах от 0 до $u_{гр}$, т. е. только низкие частоты, называются фильтрами низких частот (ФНЧ). Фильтры, пропускающие только высокие частоты от $u_{гр}$ до бесконечности, называются фильтрами высоких частот (ФВЧ). При этом, рассматривая образ Фурье сигнала, вес тех гармоник, которые должны быть пропущены, умножается на 1, а вес тех, которые нужно убрать из сигнала, — на 0. При умножении на величину от 0 до 1 гармоника ослабляется, от 1 до бесконечности — усиливается. Функция $H(u)$, определяющая, какие компоненты сигнала и в какой степени должны пропускаться фильтром, называется частотной функцией. Спектр отфильтрованного сигнала вычисляется как $F(u) \cdot H(u)$. Обратным преобразованием Фурье получается сам сигнал.

Строго говоря, фильтр характеризуется передаточной функцией $H(s)=B(s)/A(s)$, где $A(s)$ и $B(s)$ — результаты преобразования Лапласа $F(s) = \int_{-\infty}^{\infty} f(x)e^{-2sx} dx$ входного и выходного сигналов соответственно, или $H(z)=B(z)/A(z)$, где $A(z)$ и $B(z)$ — результаты z -

преобразования входного и выходного сигналов соответственно – частного случая преобразования Лапласа. Преобразование Фурье тоже является частным случаем преобразования Лапласа, полученное путём замены s на ju , так, чтобы речь шла о частоте гармоник. Частотная функция фильтра $H(u)$ представляет собой частный случай передаточной функции.

Применение ФНЧ ликвидирует из сигнала резкие переходы, что приводит к размыванию границ, так как в сигнале остаётся информация только о плавных изменениях. В то же время, если шум выражается только резкими скачками величины сигнала, он будет удалён. Применение ФВЧ оставит в сигнале только резкие переходы, в результате чего останется информация только о границах. Шум в виде резких скачков также будет пропущен. Кроме ФНЧ и ФВЧ известны полосовые фильтры, пропускающие сигнал, содержащий частоты из определённого диапазона, и режекторные фильтры, которые пропускают все частоты, кроме частот из определённого диапазона.

Произведение образа Фурье сигнала на передаточную функцию $H(u)$ соответствует свёртке самого сигнала с функцией $h(x)$, которую можно получить обратным преобразованием Фурье передаточной функции. Функция h называется импульсной характеристикой фильтра. Для изображения ФРТ является импульсной характеристикой, реализующей ФНЧ. Описание работы фильтра в виде произведения образа Фурье сигнала на передаточную функцию и в виде свёртки сигнала с импульсной характеристикой эквивалентны. При выполнении программной фильтрации изображений применяется способ, наиболее удобный в данных условиях. В ряде случаев за счёт эффективных алгоритмов дискретного быстрого преобразования Фурье (БПФ) оказывается быстрее найти образы Фурье сигнала и фильтра, перемножить их и выполнить обратное преобразование Фурье для получения результата. В других случаях проще и быстрее использовать свёртку, для реализации которой также известны быстрые способы вычислений.

Дискретная двумерная свёртка выполняется по формуле $g_{k,l} = \sum_i \sum_j f_{i,j} h_{k-i,l-j}$. Цифровое изображение размера $M \times N$ представлено в виде дискретной функции яркости $f_{i,j} = f(i,j), i = 0, \dots, M-1, j = 0, \dots, N-1$. Двумерная импульсная характеристика (ядро) фильтра в дискретном виде представлена как

матрица $h_{k,l}=h(k,l)$ размера $P \times Q$. С учётом конечности массивов изображения и ядра, свойства коммутативности свёртки, положив, что центральный элемент матрицы свёртки (точка привязки) имеет координаты $(0,0)$, формулу можно преобразовать в более удобный для вычислений вид: $g_{i,j} = \sum_{k=-P/2}^{P/2} \sum_{l=-Q/2}^{Q/2} f_{i-k,j-l} h_{k,l}$.

Для однозначного определения позиции центрального элемента удобнее, когда ядро имеет нечётные размеры. В случае необходимости ядро можно дополнить нулями. Обычно ядро задаётся в виде матрицы размером от 2×1 до 9×9 , иногда и более. Размер и форма ядра (апертура) определяют количество пикселей входного изображения, участвующих в получении каждого пикселя выходного изображения. Увеличение размеров матрицы позволяет более точно определить характеристики фильтра, но приводит к увеличению объёма вычислений и усилению краевых эффектов, когда области, близкие к краю изображения, не могут быть полноценно обработаны.

Краевые эффекты возникают вследствие того, что при $i < P/2$, $j < Q/2$, $i > M-P/2$, $j > N-Q/2$ точек исходного изображения $f_{i-k,j-l}$ не существует, так что для выходного изображения $i=P/2, \dots, M-P/2-1$, $j=Q/2, \dots, N-Q/2-1$, то есть в результате свёртки изображение уменьшается на размер ядра по каждому измерению. Краевые эффекты можно снизить без уменьшения изображения, если, например, тем или иным манером добавить по краям изображения кайму из крайних пикселей изображения шириной в половину размера фильтра, но при этом качество фильтрации в этой области всё равно ухудшится из-за фактической нехватки информации. Краевые эффекты связаны с тем, что при выполнении преобразования Фурье либо оригинал, либо Фурье образ может быть финитной функцией, но не одновременно. При этом фактически как спектр оцифрованного сигнала ограничен, так и сам сигнал ограничен, так что, либо в сигнале, либо в спектре часть информации не представлена.

При реализации фильтра в виде свёртки, для получения значения каждого пикселя $g_{i,j}$ выходного сигнала, ядро фильтра накладывается центром на соответствующий пиксель $f_{i,j}$ входного сигнала. Значения пикселей входного сигнала, покрытые апертурой фильтра, перемножаются на соответствующие элементы матрицы ядра. Полученные произведения суммируются, и результат записывается в выходной сигнал.

Часто процесс свёртки рассматривается как движение окна, совпадающего с апертурой фильтра, которое выбирает на исходном изображении группу пикселей, участвующих в свёртке. Окно передвигается на один пиксель по горизонтали или вертикали. Пиксели, попавшие в окно, свёртываются с ядром фильтра и формируют один элемент выходного изображения.

Если ядро фильтра можно представить в виде произведения столбца на строку $h(k, l) = h_1(k) \cdot h_2(l)$, такой фильтр называется сепарабельным. Обработку таким фильтром можно производить отдельно: сначала столбцом обработать каждый столбец изображения, затем каждую строку результата обработать строкой фильтра.

К линейным преобразованиям относятся интегрирование (в том числе и с весом), дифференцирование, умножение функций, вычисление функции в одной точке сигнала и с некоторыми оговорками, аффинные преобразования. Линейные пространственно инвариантные преобразования могут быть описаны как фильтры и реализованы в виде свёртки. Например, дифференцирование представляет собой фильтр высоких частот.

Если преобразование не обладает свойством однородности или аддитивности, оно относится к нелинейным. Теоретически нелинейные преобразования исследованы в меньшей мере, чем линейные, что ограничивает разработку и применение соответствующих фильтров. Нелинейные преобразования невозможно описать такими формализованными средствами, как свёртка или суперпозиция. Нелинейная фильтрация выполняется по специальному алгоритму, который может быть достаточно простым.

Одним распространённых нелинейных фильтров является медианный. Для медианной фильтрации, как и при свёртке с матрицей, рассматривается окно вокруг пикселя. В качестве результата берётся медианное значение сигнала в окне. Частотная характеристика медианного фильтра зависит от входных данных: если там присутствуют единичные выбросы, они будут удалены, т. е. частотная характеристика обеспечит существенное снижение высоких частот; более однородные исходные данные будут фильтроваться более мягко.

Фильтры с конечной импульсной характеристикой (КИХ) реализуются в виде свёртки входного сигнала с ядром конечного размера (рис. 1). Так как финитной функцией может быть либо оригинал, либо

образ Фурье, у КИХ-фильтра частотная характеристика не будет нулевой на интервале от минус бесконечности до бесконечности. Следовательно, никакие частоты сигнала таким фильтром не будут подавляться полностью.

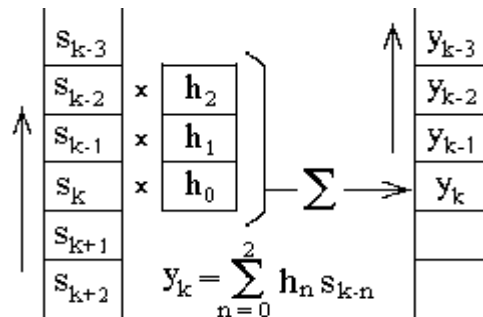


Рис. 1. Схема работы КИХ-фильтра

Если необходимо пропустить частоты из определённого диапазона и полностью задержать все остальные частоты сигнала, требуется фильтр, частотная характеристика которого будет принимать за пределами области пропускания строго нулевые значения, т. е. будет финитной. Тогда импульсная характеристика должна быть бесконечной. Фильтры с бесконечной импульсной характеристикой (БИХ) реализуются путём свёртки за счёт введения обратной связи (рис. 2).

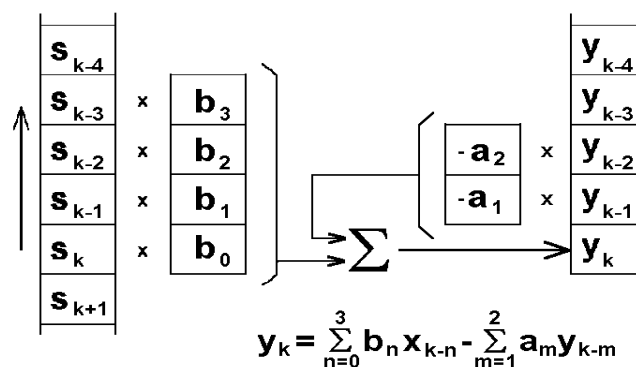


Рис. 2. Схема работы БИХ-фильтра

Такие фильтры лучше подавляют частоты, которые не должны пропускаться. Их труднее рассчитать и реализовать.

Для обработки изображений в большинстве случаев используются КИХ-фильтры. Это связано также и с тем, что по природе изоб-

ражения пиксели могут быть обработаны в любой последовательности. При использовании БИХ-фильтра результат будет зависеть от направления обработки.

Результат свёртки принимает максимальное значение в тех позициях, где форма сигнала совпадает с формой функции, описывающей ядро свёртки. Если ядро свёртки задать в соответствии с функцией яркости объекта, в результате свёртки в позиции объекта будет наблюдаться локальный максимум. Частотная характеристика такого фильтра будет равна произведению выражения, комплексно сопряжённого с выражением, описывающим спектр объекта на $e^{-j(ua+vb)}$, $a \times b$ – размер объекта.

Для определения свёрточного фильтра следует задать матрицу чисел определённого размера. Чаще всего используется размер 3×3 . Эти числа должны представлять функцию, являющуюся обратным преобразованием Фурье частотной функции фильтра.

Матрица может состоять из действительных чисел любой величины, и это может привести к тому, что у результата значения выйдут за пределы допустимого. В большинстве случаев яркость элемента находится в пределах от 0 до 255. При свёртке с ядром, в состав которого входят значения большой величины, результат может превысить 255, а при свёртке с ядром, содержащим отрицательные значения, результат тоже может оказаться меньше нуля. Известны по меньшей мере два способа решения этой проблемы.

1. Преобразовать ядро свёртки так, чтобы при сохранении формы импульсной характеристики все элементы были в пределах от 0 до 1, а сумма всех элементов ядра была равна единице.

2. Преобразовать результат нужным способом. В простейшем случае, если ядро не содержит отрицательных элементов, — разделить каждое выходное значение на сумму элементов ядра.

Один из простейших фильтров — усредняющий. По сути он применяется путём вычисления среднего значения некоторой окрестности точки или, можно сказать, среднего значения пикселей, попавших в окно и представляет собой определённый интеграл сигнала по области окна. Это преобразование можно определить как свёртку с ядром, состоящим из одних единиц с последующим делением на количество элементов ядра. Частотная характеристика такого фильтра будет иметь вид

$H(u, v) = \frac{\sin u}{u} \frac{\sin v}{v}$, т. е. это будет плохой фильтр низких частот, пропускающий, хоть и в меньшей мере, чем низкие, все частоты до бесконечности.

Хороший ФНЧ будет пропускать только низкие частоты до частоты среза u_{cp} , а остальные — задерживать. Частотная характеристика у него будет прямоугольной, а импульсная, соответственно, будет иметь вид $h(x, y) = \frac{\sin x}{x} \frac{\sin y}{y}$. Поскольку эта функция убывает медленно, соответствующее ядро преобразования должно быть большим, следовательно, путём свёртки такой фильтр реализовывать затратно, а краевые эффекты будут велики по-любому.

Удачной формой частотной характеристики оказался гауссиан $H(u) = e^{-u^2}$. Гауссиан является чётной функцией. В результате преобразования Фурье получается тоже гауссиан. Хотя эта функция нефинитна, она быстро убывает, что позволяет эффективно ограничить частоту при небольших размерах ядра свёртки. К достоинствам этой функции относится и то, что искажения оптической системы типа расфокусировки и аберраций, приводящие к снижению чёткости изображения, хорошо приближаются функцией именно такого вида. Двумерное ядро свёртки в форме гауссиана описывается формулой $h(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$. У гауссова фильтра всего один параметр σ , определяющий ширину полосы пропускания: чем он больше, тем уже полоса, тем больше степень расфокусировки. Матрицу для свёртки можно получить, вычислив её, например, с помощью табличного процессора или заполнив с помощью формулы массив в программе. Количество элементов матрицы обычно не берут больше, чем 64. Часто вещественные числа округляют до целых, предварительно взвешивая их удобными коэффициентами. На практике, если матрица вычисляется прямо в программе по формуле, в этом нет необходимости. Пример вычисления гауссовой матрицы свёртки — на рисунке 3, σ — в ячейке A1.

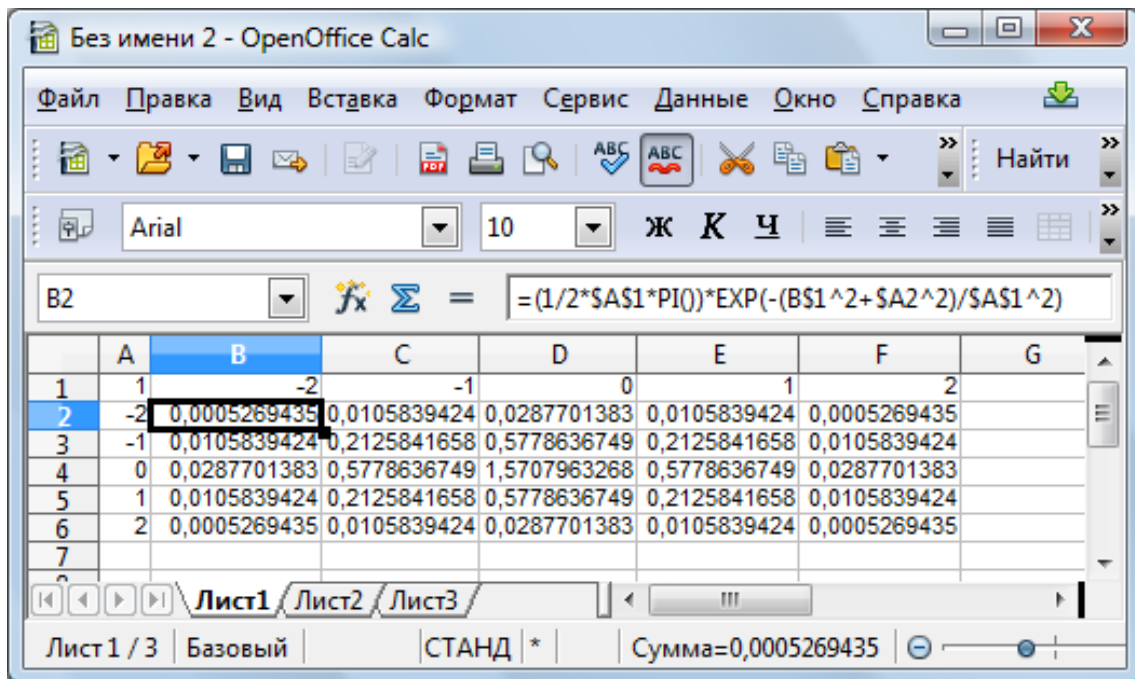


Рис. 3. Вычисление матрицы свёртки в табличном процессоре

Границы объектов на изображении часто характеризуются скачками функции яркости и представляют значительный интерес. Скорость изменения значения функции яркости показывает производная функции яркости. Численной оценкой частной производной двумерной функции по направлению слева направо может служить конечная разность $\frac{\partial f(x,y)}{\partial x} \approx f_{i+1,j} - f_{i,j}$ или симметричная конечная разность $\frac{\partial f(x,y)}{\partial x} \approx f_{i+1,j} - f_{i-1,j}$. Дискретное ядро свёртки для реализации конечной разности может иметь размер 2×1 , а для реализации центральной конечной разности — 3×1 . В отличие от ранее рассмотренных фильтров ядро дифференцирующего фильтра асимметрично и применение фильтра в разных направлениях будет приводить к разным результатам. Кроме того, невозможно определить производную в точке сразу по всем направлениям. Для оценки производной по другим направлениям соответствующим образом выбираются координаты точек. Для дискретизованного на регулярной прямоугольной решётке изображения можно определить четыре направления: слева направо, сверху вниз и две диагонали. Производные в этих направлениях можно оценить свёрткой с соответствующими ядрами размером 2×1 и 2×2 . Свёртка, реализующая центральную конечную разность, позволяет оценить производные по восьми направлениям. Есть и другие способы оценки производных.

Дифференцирующий фильтр хорошо будет пропускать границы, но ещё лучше — помехи, поэтому перед применением фильтра высоких частот изображение должно быть как можно лучше очищено от шума. Результат свёртки может быть как положительным, так и отрицательным в зависимости от того, возрастает или убывает функция яркости в заданном направлении, так что после его применения результат следует вогнать в пределы допустимого.

Логично предположить, что граница находится там, где скорость изменения функции яркости максимальна. Производная в этой позиции принимает максимальное или минимальное значение. Вторая производная в этих точках равна нулю. Численной оценкой второй частной производной по одной из переменных двумерной функции яркости может служить разность соседних значений первой производной $\frac{\partial^2 f(x,y)}{\partial x^2} \approx -f_{i+1,j} + 2f_{i,j} - f_{i-1,j}$. Ядро свёртки для такой оценки будет симметричным, а фильтр — изотропным. Сумма вторых частных производных двумерной функции является лапласианом функции яркости и принимает нулевое значение, когда первые производные достигают экстремума одновременно по направлениям слева направо и сверху вниз. Оценка лапласиана $\frac{\partial^2 f(x,y)}{\partial x^2} + \frac{\partial^2 f(x,y)}{\partial y^2} \approx 4f_{i,j} - f_{i+1,j} - f_{i-1,j} - f_{i,j+1} - f_{i,j-1}$ содержит пять слагаемых. Оценка суммы вторых производных по четырём направлениям будет содержать девять слагаемых. Все эти оценки легко вычисляются свёрткой с соответствующим ядром.

Частотная характеристика лапласиана имеет ещё более крутой подъём в области высоких частот, а это значит, что реакция лапласиана на помехи будет ещё более выраженной, а предварительная фильтрация от помех — ещё более актуальной. В какой-то мере решить проблему можно, предварительно обработав сигнал фильтром низких частот с помощью, например, гауссиана. Благодаря свойствам ассоциативности и коммутативности свёртки нет разницы, обрабатывать ли сигнал последовательной свёрткой с двумя ядрами или свернуть сигнал один раз с предварительно вычисленным результатом свёртки двух ядер друг с другом. Другими словами, вместо того, чтобы обработать изображение сначала ядром гауссиана, а потом ядром лапласиана, можно предварительно вычислить ядро лапласиана гауссиана, а потом

свернуть с ним изображение. Такое ядро будет одновременно отфильтровывать изображение от шума как гауссиан и определять положение границ как лапласиан.

Дифференцирующий фильтр, лапласиан и лапласиан гауссиана являются фильтрами высоких частот. Результатом их применения будут изображения, содержащие только области границ и лишённые подробностей в однородных областях из-за того, что в результате фильтрации спектр изображения остаётся без низких частот. Для того чтобы просто повысить резкость изображения, необходимо в частотной области усилить высокие частоты, сохранив при этом низкие. Этого можно добиться, сложив исходное изображение с результатом обработки фильтром высоких частот. Фильтрацию и сложение можно совместить, так что в результате применения этой пары операций каждый элемент на выходе будет вычисляться как $f(x, y) + \frac{\partial^2 f(x, y)}{\partial x^2} + \frac{\partial^2 f(x, y)}{\partial y^2} \approx 5f_{i,j} - f_{i+1,j} - f_{i-1,j} - f_{i,j+1} - f_{i,j-1}$ и породить соответствующую маску.

Линейные ФНЧ применяются:

1. Для очистки от помех, если спектр изображения в основном ниже частоты среза фильтра, а спектр шума — в основном выше.

2. Для сглаживания изображения перед его масштабированием, в частности, при построении пирамиды изображений для кратно-масштабного анализа.

3. С целью обеспечения выполнения условий теоремы Котельникова перед передискретизацией.

4. Для коррекции неравномерной освещённости или виньетирования, так как спектр связанного с этим изменения яркости лежит в области очень низких частот.

5. Для создания художественных эффектов.

Линейные ФВЧ применяются:

1. Для поиска границ объектов: детекторы границ Прюитт, Собела, Робертса, Лапласа, Марра, Щарра, Кэнни, Айверсона, Ротуэлла базируются на оценке производных, в т. ч. для вычисления градиентов, в большинстве случаев в комбинации с ФНЧ.

2. Для восстановления изображения, искажённого расфокусировкой и другими аберрациями, снижающими долю высоких частот.

3. Для создания художественных эффектов.

Режекторные фильтры применяются для подавления шума, когда сигнал помехи лежит в узком диапазоне частот, который перекрывается

полезным сигналом. Полосовые фильтры применяются, когда полезный сигнал лежит в узком диапазоне частот.

Матрицы свёртки некоторых фильтров приведены в таблице 1.

Нелинейные фильтры применяются в основном для очистки от помех. Кроме того, некоторые способы обработки, которые могут интерпретироваться как нелинейные, могут быть получены из линейных. Например, градиент, который можно оценить с помощью производных.

Согласованные фильтры применяются для поиска объектов. Это имеет смысл, если объект на изображениях не меняет размер и ориентацию и затруднено, когда размер объекта велик.

Таблица 1

Распространённые матрицы свёртки линейных фильтров

Название	Ядро											
1. Усредняющий	1				1				1			
	1				1				1			
	1				1				1			
2. Прямоугольный	0	0	0	0	1	1	1	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	-1	-2	-1	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0
	1	0	-1	0	4	6	4	0	-1	0	1	1
	1	0	-2	0	6	10	6	0	-2	0	1	1
	1	0	-1	0	4	6	4	0	-1	0	1	1
	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	-1	-2	-1	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	1	1	1	0	0	0	0	0
3. Гауссов	0			1			2			1		
	1			6			10			6		
	2			10			16			10		
	1			6			10			6		
	0			1			2			1		

Название	Ядро						
4. Дифференцирующий Робертса	по x		по y	по xy		по yx	
	1	-1	1	1	0	0	-1
			-1	0	-1	1	0
5. Дифференцирующий Прюитт	Вертикальный			Горизонтальный			
	-1	-1	-1	-1	0	1	
	0	0	0	-1	0	1	
	1	1	1	-1	0	1	
6. Дифференцирующий Собела	Вертикальный			Горизонтальный			
	-1	-2	-1	-1	0	1	
	0	0	0	-2	0	2	
	1	2	1	-1	0	1	
7. Лапласиан	2 напр.			4 напр.			
	0	-1	0	-1	-1	-1	
	-1	4	-1	-1	8	-1	
	0	-1	0	-1	-1	-1	
8. Лапласиан гауссиана	-1	-2	-3	-2	-1		
	-2	0	4	0	-2		
	-3	4	16	4	-3		
	-2	0	4	0	-2		
	-1	-2	-3	-2	-1		
9. Повышение резкости	2 напр.			4 напр.			
	0	-1	0	-1	-1	-1	
	-1	5	-1	-1	9	-1	
	0	-1	0	-1	-1	-1	
10. Тиснение	0		1		0		
	1		0		-1		
	0		-1		0		

Задание

1. Разработать алгоритм и программу для реализации индивидуального задания. Программа должна включать в себя:
 - обработку изображения свёрточным фильтром низких частот;
 - обработку изображения свёрточным фильтром высоких частот.
2. Ответить на контрольные вопросы.
3. Оформить отчёт.

Содержание отчёта

1. Титульный лист.
2. Формулировка задачи.
3. Алгоритм.
4. Программа на алгоритмическом языке программирования.
5. Исходное и преобразованные изображения.
6. Ответы на контрольные вопросы.

Порядок выполнения работы

1. Ознакомление с целью работы, заданием, содержанием отчёта, порядком выполнения работы, индивидуальным вариантом, примером выполнения работы, примером оформления отчёта.
2. Ознакомление с теоретическими материалами по теме.
3. Разработка алгоритма в соответствии с вариантом задания. Алгоритм должен предусматривать:
 - чтение файла изображения;
 - обработку фильтром низких частот в соответствии с заданием;
 - обработку фильтром высоких частот в соответствии с заданием;
 - запись преобразованного изображения в файл.
4. Разработка и отладка программы для решения задачи.
5. Ответы на контрольные вопросы.
6. Оформление отчёта.

Индивидуальные задания

Каждый вариант задания (табл. 2) включает в себя применение разных ФВЧ и ФНЧ к разным частям изображения.

Таблица 2

Варианты индивидуальных заданий

№	ФНЧ	ФВЧ
1	К левой половине изображения примените ФНЧ № 1, к правой – ФНЧ № 2	К левой половине изображения примените ФВЧ Прюитт, к правой — ФВЧ Собела.
2	Область изображения за пределами круга радиуса R размойте гауссианом	Повысьте резкость в районе пикселей, превышающих яркость T
3	Размойте пиксели, не превышающие порог яркости T	Повысьте резкость изображения после ФНЧ
4	Уменьшите размеры изображения вдвое, взяв в результат каждый второй пиксель по каждому направлению. Сделайте для сравнения то же самое после обработки исходного изображения ФНЧ	К левой половине изображения примените ФВЧ Робертса, к правой — ФВЧ Прюитт
5	Обработайте изображение двумя гауссианами с разными значениями параметра и найдите разность результатов	К левой половине изображения примените ФВЧ Робертса, к правой — ФВЧ Собела
6	Обработайте изображение ФНЧ № 1 и вычтите результат обработки из исходного изображения	К левой половине изображения примените ФВЧ Лапласа, к правой — лапласиан гауссиана
7	К левой половине изображения примените ФНЧ только для одного из цветовых каналов, к правой — только для другого	К левой половине изображения примените лапласиан гауссиана, к правой — его же, предварительно переведя изображение в оттенки серого
8	К левой половине изображения примените ФНЧ только для яркости, к правой — только для насыщенности	К левой половине изображения примените лапласиан гауссиана только для яркости, к правой — только для насыщенности

№	ФНЧ	ФВЧ
9	К левой половине изображения примените ФНЧ только для яркости, к правой — только для цветового тона	К левой половине изображения примените лапласиан гауссиана только для яркости, к правой — только для насыщенности
10	К левой половине изображения примените ФНЧ только для насыщенности, к правой — только для цветового тона	К левой половине изображения примените лапласиан гауссиана только для яркости, к правой — только для насыщенности
11	К левой половине изображения примените ФНЧ № 2, к правой — ФНЧ № 3	К левой половине изображения примените лапласиан гауссиана только для яркости и цветового тона, к правой — только для насыщенности и цветового тона
12	Область изображения за пределами прямоугольника $a \times b$ размойте гауссианом	К левой половине изображения примените лапласиан гауссиана только для яркости и цветового тона, к правой — только для яркости и насыщенности
13	Размойте пиксели, превышающие порог яркости T	К левой половине изображения примените лапласиан гауссиана только для яркости и насыщенности, к правой — только для насыщенности и цветового тона
14	Увеличьте размеры изображения вдвое, взяв в результат каждый пиксель по каждому направлению по два раза. Обработайте результат ФНЧ	К левой половине изображения примените лапласиан гауссиана, к правой — его же, предварительно переведя изображение в негатив, а после — обратно
15	Обработайте изображение двумя ФНЧ № 1 и ФНЧ № 3, найдите разность результатов	К изображению примените ФВЧ Лапласа и лапласиан гауссиана. Найдите разность результатов

№	ФНЧ	ФВЧ
16	Обработайте изображение ФНЧ № 3 и вычитите результат обработки из исходного изображения	К изображению примените ФВЧ Робертса и ФВЧ Собела. Найдите разность результатов
17	К левой половине изображения примените ФНЧ только для двух из трёх цветовых каналов, к правой — только для других двух.	К изображению примените ФВЧ Робертса и ФВЧ Прюитт. Найдите разность результатов.
18	К левой половине изображения примените ФНЧ только для яркости и цветового тона, к правой — только для насыщенности и цветового тона	Повысьте резкость изображения в пределах круга радиусом R
19	К левой половине изображения примените ФНЧ только для яркости и насыщенности, к правой — только для яркости и цветового тона	Повысьте резкость в районе пикселей, не превышающих яркость T
20	К левой половине изображения примените ФНЧ только для насыщенности и яркости, к правой — только для насыщенности и цветового тона	К изображению примените ФВЧ Прюитт, и ФВЧ Собела. Найдите разность результатов

Контрольные вопросы

1. Какие фильтры проходит оптический сигнал в процессе формирования на поверхности фотоприёмника двумерного изображения трёхмерной сцены и к какому классу относится каждый из этих фильтров?

2. Какие фильтры проходит оптический сигнал в процессе формирования цифрового изображения от поверхности фотоприёмника до памяти и какому классу относится каждый из этих фильтров?
3. Какие из потерь в процессе фильтрации необратимы и почему?
4. Какие из потерь в процессе фильтрации обратимы и как?
5. В чём преимущества и недостатки БИХ-фильтров по сравнению с КИХ в области обработки изображений?
6. Как реализовать БИХ-фильтр с помощью свёртки с окном?
7. Как реализовать попиксельные преобразования с помощью свёртки?
8. Как реализовать геометрические преобразования с помощью свёртки?
9. Какие эффекты фотошопа реализованы путём фильтрации?

Пример выполнения работы

0	Размыть изображение ФНЧ Гаусса с параметром, увеличивающимся по мере удаления от центра	По каждому каналу увеличить резкость точек, для которых контраст под окном превышает заданный порог
---	---	---

При выполнении первой части задания расстояние от обрабатываемой точки до центра будем вычислять по теореме Пифагора. В зависимости от расстояния будет изменяться параметр рассеяния. Чем больше параметр рассеяния, тем больший размер ядра необходим для представления гауссовского фильтра. Если ограничиться размером ядра 15x15, то параметр рассеяния следует брать около $15/6=2,5$. В крайнем случае до 6, при этом фильтр на большом удалении от центра будет не совсем гауссовским.

При выполнении второй части задания в качестве фильтра для повышения резкости возьмём четырёхнаправленный. Размер ядра 3x3, и контраст вычислим в пределах девяти точек. Контраст будем вычислять для каждой цветовой компоненты отдельно, как разницу между максимальным и минимальным значением яркости.

На форме (рис. 4) предусматриваем по кнопке и диалогу для считывания изображения и сохранения каждого из двух результатов, три объекта Tgraphics для хранения и отображения картинок, кнопки для двух преобразований, поле ввода для порога контраста.

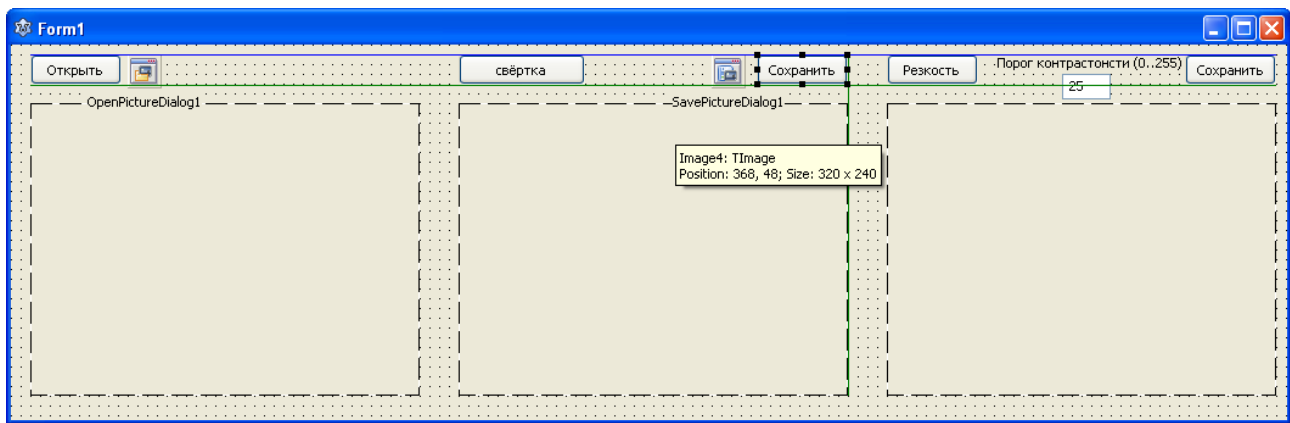


Рис. 4. Форма для программы фильтрации изображений

Последовательность действий для первого преобразования:

1. Считываем изображение, преобразовываем в полноцветное.
2. Инициализируем изображение с теми же параметрами, что и исходное.
3. Вычисляем коэффициент с которым будет увеличиваться параметр рассеяния фильтра при увеличении расстояния от центра: $kp := 6 / \sqrt{(wi/2 - kw/2)^2 + (he/2 - kh/2)^2}$, где wi , he — длина и ширина изображения; kw , kh — размеры ядра свёртки.
4. Для каждого обрабатываемого пикселя вычисляем параметр рассеяния: $s := kp * \sqrt{(i - wi/2 + kw/2)^2 + (j - he/2 + kh/2)^2}$, с этим параметром вычисляем значения ядра свёртки, выполняем свёртку окрестностей пикселя с вычисленным ядром.
5. Проверяем, не вышли ли результаты за пределы допустимых уровней яркости, и корректируем при необходимости.
6. Выводим результат.
7. Предусматриваем возможность записи результатов.

Последовательность действий для второго преобразования:

1. Инициализируем изображение с теми же параметрами, что и исходное.
2. Считываем пороговое значение контраста.
3. Формируем ядро преобразования.
4. Формируем массивы исходных данных и данных результата.
5. Для каждого обрабатываемого пикселя по каждому каналу в окрестностях, соответствующих окну преобразования, находим минимальное и максимальное значение. Если разница между этими значениями превышает заданный порог, осуществляем свёртку с ядром.

6. Проверяем, не вышли ли результаты за пределы допустимых уровней яркости, и корректируем при необходимости.

7. Выводим результат.

8. Предусматриваем возможность записи результатов.

Получаем модуль приблизительно следующего содержания:

```
unit Unit1;
```

```
{ $mode objfpc } { $H+ }
```

```
interface
```

```
uses
```

```
Classes, SysUtils, FileUtil, Forms, Controls, Graphics, Dialogs, StdCtrls,  
GraphType, IntfGraphics, LCLType, LCLProc, LCLIntf, ExtCtrls, Ex-  
tDlgs, Menus,  
Grids, fpImage, math, crt;
```

```
type
```

```
 { TForm1 }
```

```
TA = Array of array of real;
```

```
TForm1 = class(TForm)
```

```
  Button1: TButton;
```

```
  Button2: TButton;
```

```
  Button4: TButton;
```

```
  Button5: TButton;
```

```
  Button6: TButton;
```

```
  Button7: TButton;
```

```
  Button8: TButton;
```

```
  Edit1: TEdit;
```

```
  Edit2: TEdit;
```

```
  Edit3: TEdit;
```

```
  Image1: TImage;
```

```
  Image4: TImage;
```

```
  Label1: TLabel;
```

```
  Label3: TLabel;
```

```
  Label4: TLabel;
```

```

OpenPictureDialog1: TOpenPictureDialog;
SavePictureDialog1: TSavePictureDialog;
StringGrid1: TStringGrid;
t: TImage;
procedure Button1Click(Sender: TObject);
procedure Button2Click(Sender: TObject);
procedure Button5Click(Sender: TObject);
procedure Button6Click(Sender: TObject);
procedure Button7Click(Sender: TObject);
procedure FormClose(Sender: TObject);
private
  { private declarations }
public
  { public declarations }
end;

```

```

var
  Form1: TForm1;
implementation

```

```

{$R *.lfm}

```

```

{ TForm1 }
//процедура формирования ядра гауссовского ФНЧ
procedure gaussmask(var h:TA;kw,kh:integer;s:real;mustnormz:boolean);
//принимаем массив для маски, его размеры, параметр рассеивания и
нужно ли нормализовывать.
var i,j:integer;sum:real;
begin
  for i:=0 to kw-1 do
    for j:=0 to kh-1 do
      h[i,j]:=0;      //на всякий случай обнуляем
    if s<>0 then //если параметр рассеяния не равен 0
//вычисляем гауссоиду
      for i:=0 to kw-1 do
        for j:=0 to kh-1 do
          h[i,j]:=(1/(2*Pi*sqr(s)))*exp(-(sqr(i-kw div 2-1)+sqr(j-kh div 2-1))/(2*sqr(s)))

```

```
else h[kw div 2,kh div 2]:=1;//если параметр рассеяния 0, то маски как бы нет.
```

```
if mustnormz then //если нужно нормализовывать
```

```
begin
```

```
sum:=0;
```

```
//вычисляем сумму всех элементов
```

```
for i:=0 to kw-1 do
```

```
for j:=0 to kh-1 do
```

```
sum:=sum+h[i,j];
```

```
//и каждый элемент делим на неё
```

```
for i:=0 to kw-1 do
```

```
for j:=0 to kh-1 do
```

```
h[i,j]:=h[i,j]/sum;
```

```
end;
```

```
end;
```

```
//преобразование в формат pf24bit
```

```
function ConvertTo24Bit(aValue: TBitmap;BackColor: TColor): TBitmap;
```

```
begin
```

```
Result := TBitmap.Create;
```

```
with Result do
```

```
if aValue.PixelFormat <> pf24bit then
```

```
begin
```

```
PixelFormat := pf24bit;
```

```
SetSize(aValue.Width,aValue.Height);
```

```
Canvas.FloodFill(0,0,BackColor,fsBorder);
```

```
Canvas.Draw(0,0,aValue);
```

```
end
```

```
else
```

```
Assign(AValue);
```

```
end;
```

```
//открытие файла, конвертация если необходимо
```

```
procedure TForm1.Button1Click(Sender: TObject);
```

```
begin
```

```
if OpenPictureDialog1.Execute then
```

```
t.Picture.LoadFromFile(OpenPictureDialog1.FileName);
```

```
Label3.Caption:=IntToStr(t.Picture.Bitmap.Width);
```

```

Label4.Caption:=IntToStr(t.Picture.bitmap.Height);
if t.Picture.bitmap.PixelFormat <> pf24bit then
begin
  showmessage('invalid pixel color format. trying to convert');
  t.picture.bitmap.assign(convertto24bit(t.Picture.bitmap,RGB(255,255,255)));
end;
end;

```

//выполнение задания 1.

```

procedure TForm1.Button2Click(Sender: TObject);
var i,j,k,l,kerheight,kerwidth,he,wi:integer;min,max,kp,s:real;rr,rg,rb,h:ta;
begin
  kerwidth:=15;kerheight:=15;//размер ядра (не маленький)
  he:=t.Picture.Bitmap.Height;//количество пикселей по вертикали
  wi:=t.Picture.Bitmap.Width;//количество пикселей по горизонтали
  setlength(h,kerwidth,kerheight);//устанавливаем размер массива с ядром свёртки
  setlength(rr,wi,he);//устанавливаем размер массива с красным каналом изображения
  setlength(rg,wi,he);//устанавливаем размер массива с зелёным каналом изображения
  setlength(rb,wi,he);//устанавливаем размер массива с синим каналом изображения
  //во сколько раз увеличится параметр рассеяния к краю
  kp:=6/sqrt(sqr(wi div 2-kerwidth div 2)+sqr(he div 2-kerheight div 2));
  with t.Picture.Bitmap.Canvas do //с исходным изображением
  begin
    for i:=0 to wi-1-kerwidth do //по всей ширине
    begin
      for j:=0 to he-1-kerheight do //по всей высоте
      begin
        rr[i,j]:=0;rg[i,j]:=0;rb[i,j]:=0;//на всякий случай
        //вычисляем параметр рассеяния в зависимости от расстояния до центра
        s:=sqrt(sqr(i-wi div 2+kerwidth div 2)+sqr(j-he div 2+kerheight div 2))*kp;

```

```

    gaussmask(h,kerwidth,kerheight,s,true);//формируем массив с яд-
ром
    for k:=0 to kerwidth-1 do //осуществляется свёртка
        for l:=0 to kerheight-1 do
            begin
                rr[i,j]:=rr[i,j]+getrvalue(Pixels[i+k,j+l])*h[k,l];
                rg[i,j]:=rg[i,j]+getgvalue(Pixels[i+k,j+l])*h[k,l];
                rb[i,j]:=rb[i,j]+getbvalue(Pixels[i+k,j+l])*h[k,l];
            end;
        end;
        //выводим индикатор, свёртка с таким размером ядра медленная
        button2.caption:='свёртка' +inttostr(round(i/(width-ker-
width)*100))+'%';
    end;
end;
//на всякий случай вгоняем яркость в допустимый диапазон
min:=rr[0,0]; //для этого находим минимальное значение
for i:=0 to wi-1-kerwidth do
    for j:=0 to he-1-kerheight do
        begin
            if min>rr[i,j] then min:=rr[i,j];
            if min>rg[i,j] then min:=rg[i,j];
            if min>rb[i,j] then min:=rb[i,j];
        end;
    if min>=0 then min:=0;//если не выходим за пределы допустимого, из-
менений не будет
    max:=rr[0,0]-min;//находим максимальное значение
    for i:=0 to wi-1-kerwidth do
        for j:=0 to he-1-kerheight do
            begin
                if max<rr[i,j]-min then max:=rr[i,j]-min;
                if max<rg[i,j]-min then max:=rg[i,j]-min;
                if max<rb[i,j]-min then max:=rb[i,j]-min;
            end;
        //если не выходим за пределы допустимого, изменений не будет
        if max<=255 then max:=1 else max:=255/max;
        //формируем изображение с результатом
        with Image4.Picture.Bitmap do

```

```

begin
  Clear;
  PixelFormat:=pf24bit;
  Width:=wi;
  Height:=he;
  Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder);
  for i:=0 to wi-1-kerwidth do
    for j:=0 to he-1-kerheight do
      canvas.pixels[i+kerwidth div 2,j+kerheight div 2]:=RGB(round((rr[i,j]-min)*max),round((rg[i,j]-min)*max),round((rb[i,j]-min)*max));
    end
  end;
end;

```

```

//выборочное увеличение резкости
procedure TForm1.Button5Click(Sender: TObject);
var
  i,j,k,l,kerheight,kerwidth,he,wi:integer;minr,ming,minb,min,max,maxr,maxg,maxb,phi:real;ir,ig,ib,rr,rg,rb,h:ta;
;
begin
  phi:=strtofloat(edit3.text);//пороговая величина контраста
  kerwidth:=3;kerheight:=3; //размер ядра свёртки
  he:=t.Picture.Bitmap.Height;//размер изображения
  wi:=t.Picture.Bitmap.width;
  setlength(h,kerwidth,kerheight);//массив для ядра
  setlength(rr,wi,he); setlength(rg,wi,he); setlength(rb,wi,he); //массивы с
результатом
  setlength(ir,wi,he); setlength(ig,wi,he); setlength(ib,wi,he);//исходные
данные
  h[0,0]:=-1; h[1,0]:=-1; h[2,0]:=-1; //ядро свёртки
  h[0,1]:=-1; h[1,1]:=9; h[2,1]:=-1;
  h[0,2]:=-1; h[1,2]:=-1; h[2,2]:=-1;
  with t.Picture.Bitmap.Canvas do //с исходным изображением
    begin
      for i:=0 to wi-1 do //заполняем массивы с исходными данными и
результатом
        for j:=0 to he-1 do
          begin

```

```

    ir[i,j]:=getrvalue(Pixels[i,j]);
    ig[i,j]:=getgvalue(Pixels[i,j]);
    ib[i,j]:=getbvalue(Pixels[i,j]);
    rr[i,j]:=ir[i,j];rg[i,j]:=ig[i,j];rb[i,j]:=ib[i,j];
end;
for i:=0 to wi-1-kerwidth do //рассматриваем каждый элемент ре-
зультата
begin
    for j:=0 to he-1-kerheight do
    begin
        minr:=ir[i,j];ming:=ig[i,j];minb:=ib[i,j];//вычисляем минимумы
        maxr:=ir[i,j];maxg:=ig[i,j];maxb:=ib[i,j];//и максимумы для
        for k:=0 to kerwidth-1 do //вычисления контраста
            for l:=0 to kerheight-1 do
            begin
                if minr >ir[i+k,j+l] then minr:=ir[i+k,j+l];
                if ming >ig[i+k,j+l] then ming:=ig[i+k,j+l];
                if minb >ib[i+k,j+l] then minb:=ib[i+k,j+l];
                if maxr <ir[i+k,j+l] then maxr:=ir[i+k,j+l];
                if maxg <ig[i+k,j+l] then maxg:=ig[i+k,j+l];
                if maxb <ib[i+k,j+l] then maxb:=ib[i+k,j+l];
            end;
            for k:=0 to kerwidth-1 do //осуществляем свёртку
                for l:=0 to kerheight-1 do//если контраст больше порога
                begin
                    if (maxr-minr)>phi then rr[i,j]:=rr[i,j]+ir[i+k,j+l]*h[k,l];
                    if (maxg-ming)>phi then rg[i,j]:=rg[i,j]+ig[i+k,j+l]*h[k,l];
                    if (maxb-minb)>phi then rb[i,j]:=rb[i,j]+ib[i+k,j+l]*h[k,l];
                end;
            end;
        end;
    end;
end;
//если яркость выходит за пределы допустимого, вычисляем пара-
метры нормировки.
min:=rr[0,0];
for i:=0 to wi-1-kerwidth do
    for j:=0 to he-1-kerheight do
        begin

```

```

    if min>rr[i,j] then min:=rr[i,j];
    if min>rg[i,j] then min:=rg[i,j];
    if min>rb[i,j] then min:=rb[i,j];
end;
if min>=0 then min:=0;
max:=rr[0,0]-min;
for i:=0 to wi-1-kerwidth do
  for j:=0 to he-1-kerheight do
    begin
      if max<rr[i,j]-min then max:=rr[i,j]-min;
      if max<rg[i,j]-min then max:=rg[i,j]-min;
      if max<rb[i,j]-min then max:=rb[i,j]-min;
    end;
  if max<=255 then max:=1 else max:=255/max;
  with Image1.Picture.Bitmap do //выводим результат в Image1
    begin
      Clear;
      PixelFormat:=pf24bit;
      Width:=wi;
      Height:=he;
      Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder);
      for i:=0 to wi-1-kerwidth do
        for j:=0 to he-1-kerheight do
          canvas.pixels[i+kerwidth div 2,j+kerheight div
2]:=RGB(round((rr[i,j]-min)*max),round((rg[i,j]-
min)*max),round((rb[i,j]-min)*max));
        end;
      end;
//сохраняем размытое изображение
procedure TForm1.Button6Click(Sender: TObject);
begin
  if SavePictureDialog1.Execute then
    Image4.Picture.SaveToFile(SavePictureDialog1.FileName);
end;
//сохраняем изображение повышенной чёткости
procedure TForm1.Button7Click(Sender: TObject);
begin
  if SavePictureDialog1.Execute then

```

```
Image1.Picture.SaveToFile(SavePictureDialog1.FileName);  
end;
```

```
procedure TForm1.FormClose(Sender: TObject);  
begin  
  //освобождаем память  
  t.Free;  
  image1.free;  
  image4.Free;  
end;  
  
end.
```

Запускаем программу и получаем такой вот результат (рис.5):



Рис. 5. Форма для программы фильтрации изображений

Заключение

Спектр задач, в которых находит применение обработка изображений, необычайно широк. В первую очередь это связано с автоматизацией обработки информации с учётом того, что человек большую часть её получает с помощью зрения. Значительно распространены и широко применяются программные продукты для обработки изображений на персональных компьютерах, такие как Adobe Photoshop, GIMP, Corel Draw и др.

Практикум содержит лабораторные работы, формирующие начальные навыки обработки изображений. Для освоения этих навыков не требуется значительного объёма специальных знаний. В то же время уделено внимание определённой строгости изложения и разнообразию рассматриваемых задач. При этом умения, формируемые этими работами, позволяют разобраться в свойствах большинства способов обработки, применяемых в указанных программных продуктах.

В лабораторных работах практикума рассмотрены достаточно полезные и практически применимые способы обработки, которые способствуют пониманию логики обработки изображений вообще и которые можно улучшать до уровня решения задач, применяемых в перечисленных выше программных продуктах.

Библиографический список

1. Аммерал, Л. Принципы программирования в машинной графике: / Л. Аммерал – Москва : Сол Систем, 1992. – 224 с. – ISBN 5-85316-001-Х. – Текст : непосредственный.
2. Роджерс, Д. Алгоритмические основы машинной графики / Д. Роджерс – Москва : Мир, 1989. – 512 с. – ISBN 5-03-000476-9. – Текст : непосредственный.
3. Роджерс, Д. Математические основы машинной графики / Д. Роджерс, Дж. Адамс – Москва : Мир, 2001. – 604 с. – ISBN 5-03-002143-4. – Текст : непосредственный.
4. Шикин, Е. В. Начала компьютерной графики / Е. В. Шикин, А. В. Боресков, А. А. Зайцев. – Москва : ДИАЛОГ-МИФИ, 1993. – 138 с. – ISBN 5-86404-035-5. – Текст : непосредственный.
5. Модуль № 1. Основные правила оформления чертежей. Геометрические построения: учеб.-метод. Пособие по дисциплине «Инженерная графика» /сост. Н. И. Масакова [и др.]. – Тольятти: ТГУ, 2009г-58 с.
6. <https://www.scattergood.io/arc-drawing-algorithm/>
7. Chris Wyman, Peter-Pike Sloan, and Peter Shirley, Simple Analytic Approximations to the CIE XYZ Color Matching Functions, Journal of Computer Graphics Techniques (JCGT), vol. 2, no. 2, 1-11, 2013

Учебное издание

Ефремов Владислав Владиславович,
Ефремова Ирина Николаевна,
Петрик Елена Анатольевна

**КОМПЬЮТЕРНАЯ ГРАФИКА: ВВЕДЕНИЕ
В ОБРАБОТКУ ИЗОБРАЖЕНИЙ**

Практикум

Редактор *С. П. Тарасова*
Компьютерная вёрстка и макет

Подписано в печать . Формат 60х84 1/16. Бумага офсетная.
Усл. печ. л. . Уч.-изд. л. . Тираж экз. Заказ .
Юго-Западный государственный университет.
305040, г. Курск, ул. 50 лет Октября, 94.
Отпечатано в ЮЗГУ.